

给"分级"分级

LLM 验证自主等级 (VAL) 的实证与方法

Yajie Yin

2026 · 开源研究专著 · 配套论文 arXiv:2608.19009

别人给答案分级，我们给"分级"分级。

而一个验证器能获得的最高分，是它能真正交付的保证：

域内的正确性、ODD 内的完备性、以及它必须弃权时的诚实。

序言：这本书是什么

这本书是一个研究项目的完整记录——从"朴素分解想法"走到 arXiv 论文，再走到对论文本身的审计。

项目的核心问题只有一个：当 LLM 越来越频繁地说"我检查过了"的时候，"检查过了"到底意味着什么？

答案不是一个，而是一个阶梯：

- L0**：LLM 自己说"我检查过了"——没有任何确定性锚，是最不可信的一档；
- L2**：客观真值/金标——保证"提出的候选都成立"，但证明不了"没有候选被遗漏"（完备性盲区）；
- L3/L4**：可判定系统——把性质编码进解集等价、类型系统、形式内核，在 ODD 内拿到完备性；
- L5**：通用完备验证——原则上不存在（Rice 定理）。

这个阶梯叫 **验证自主等级 (VAL)**。它借用了自动驾驶分级的灵魂——**责任在谁**——而不是能力高低。

但本书不止是提出一个框架。它记录了：

- 四章实证**——数学、行为观测、医疗诊断、代码生成，每一章都按"审计先行 → 测价值窗口 → 失败分类 → 诚实负结果"的纪律走完，多数结论是负的（LLM 太强，准确率窗口为空）；
- VAL 框架本身**——锚定层级、判级标准、17 篇文献评述、可运行的判级工具链；
- 裁判的裁判**——对判级标准自身的可复现性研究（四个独立盲评者， $\kappa \approx 0.8$ ），以及它抓出的作者判级错误；
- 方向 3**——把框架输出到 Agent 安全，用 PPMF 案例、锚语义类型学、预测协议证明"锚定层级预测防御成败"。

这本书的诚实声明：它包含大量负结果。这些负结果不是失败——它们界定了"验证能干什么、不能干什么"的边界。我们相信，能诚实报告边界的框架，比假装无所不能的框架更有用。

怎么读：

- 想了解主线：读序言 → 第一、二部（叙事 + 四章实证）→ 第六部（结果总账）；
 - 想用框架：读第三部（VAL + 判级标准 + 工具链）；
 - 想挑战我们：读第四、五部（可复现性研究 + Agent 安全预测）——尤其第四部记录了作者判级被盲评修正的记录，这是框架用在自己身上的证据。
-

目录

第一部 主线

- 第 1 章 研究叙事 (docs/o1)
- 第 2 章 失败模式分类学 (docs/o2)

第二部 四章实证

- 第 3 章 数学验证：架构与实验档案 (docs/o3 + o4 + RESULTS)
- 第 4 章 行为观测 (behavior/REPORT)
- 第 5 章 医疗诊断 (medical/REPORT)
- 第 6 章 代码生成：反向验证 (docs/o8)

第三部 VAL 框架

- 第 7 章 锚定层级 (docs/o5)
- 第 8 章 判级标准 (docs/o6)
- 第 9 章 文献评述：17 篇论文的判级 (docs/o7)
- 第 10 章 抬级器可行性 (docs/o9)
- 第 11 章 对抗性评审记录 (docs/10)

第四部 裁判的裁判

- 第 12 章 判级可复现性研究 (docs/11)
- 第 13 章 锚语义类型学 (docs/13)

第五部 Agent 安全 (方向 3)

- 第 14 章 PPMF 案例：来源洗白与防御 (docs/12)
- 第 15 章 VAL→ASR 预测协议 (docs/14)
- 第 16 章 综合：锚定层级预测防御成败 (docs/15)

第六部 收束

- 第 17 章 评估总账 (EVALUATION)
- 第 18 章 正式出版物：Grading the Graders (paper/preprint)

附录

- 附录 A 项目总览 (README)
 - 附录 B 项目交接文档 (HANDOFF)
-

第一部·主线

第 1 章

01 · 研究叙事：从直觉到失败分类学

这份文档是项目的"故事主线"——投名状的第一页。

它记录了一个完全由直觉牵引、被实验反复证伪、最终沉淀为可复现结论的完整过程。

一、起点：一个关于"解题"的直觉

最初的问题很简单：**AI 做数学题时，记忆和推导的占比是多少？**顺着这个问题，一系列直觉被提出：

1. **多模型拼接**：人类解综合题，本质是把"二次函数""几何最值"等小模型拼成解题链——AI 应该也是这样。
2. **两次输入输出**：拿到题先拆解（第一次输出），把拆解结果反馈回去匹配例题、再组合（第二次输出）。
3. **裁判才是关键**：这个想法缺的不是工具，而是一个能判定对错的"裁判"——AlphaGo 有围棋规则当裁判，数学没有。
4. **根基问题**：补丁越打越多时意识到——LLM 是自由文本生成器，只要让它"重新组织语言"，就会污染已验证的数学事实。根基坏了，打补丁没用。

这些直觉都不是新技术，但它们组合起来指向一个清晰的主张：

LLM 只做翻译（拆解、声明、渲染），确定性符号引擎做裁判；验证规格必须来自不依赖 LLM 声明的地面真值。

二、架构：七层防御的演进



关键设计：**三态裁判**（PASS/FAIL/UNSURE，不懂就弃权不误杀）、**类型系统**（答案类型声明与反查，防"区间答成点集"）、**终局验证规格规则派生**（消除 LLM 声明验证条件这个信任点）。

三、实验：八轮，每一步都被证伪

#	实验	结果	含义
1	基础 5 题	系统 5/5, baseline 5/5	简单题上架构无优势（符合预期）
2	对抗预测 5 题	baseline 5/5（预测 0/5 应验），系统 3/5	预测失败模式全部落空 ；且 LLM 编的标准答案 3/5 是错的
3	修复后复测	纠错回路首次实战成功	判别式错误当场抓+一次修正
4	20 题三级测试	系统 16/20 vs baseline 20/20	价值窗口在中等难度为空
5	压轴 2 题（竞赛/新题）	baseline 2/2, 系统 0/2	价值窗口在高难度仍为空
6	类型系统+终局验证	B 题 [-1,1] 当场抓获并修正为 [0,1]	设计兑现，但标签有噪声
7	20 题带防御重跑	15/20；2 假 PASS、1 假 FAIL	终局验证是可信度探针，非正确性保证
8	规则派生重跑	13/20；3 假 PASS 全是采样盲区	完备性盲区=采样天花板+验证器极限

三次 20 题运行 16→15→13：准确率是 LLM 拆解随机性的函数，防御的影响小于噪声。

四、最终产出：四个可复现的结论

1. **没有确定性裁判，每一层都会"自信地错"**：出题层（标准答案 3/5 错）、求解层（自治地错）、复盘层（把对的判错、把错的判对）——裁判的裁判也需要裁判。
2. **验证的相对性**：代入/抽样验证只能证明"找到的候选成立"，永远无法证明"没有候选被遗漏"。完备性盲区不是 bug，是验证的性质。

3. **信任分层是唯一可行的方向**：LLM 的每一步都必须被降级为"声明意图"，数学由确定性代码执行；验证规格必须锚定在"不依赖 LLM 声明"的地面真值上，否则只是把信任递归推后一层。
4. **裁判价值 = 锚定层级**：规格锚在 L0 (LLM 声明) 时是信任递归，锚在 L3 (定义性/证明性，如解集等价、类型检查、已验证决策规则) 时能把"正确性"升级成"完备性"。完备性不是无解，是把问题重编码进可判定领域（详见 docs/o5-锚定层级.md）。

五、诚实的边界（不美化）

- 在 deepseek-chat + 一元二次函数范围内，**baseline 在所有难度上胜过完整架构**（20/20 vs 13~16/20）。架构的准确率优势在本实验范围内不存在。
- 架构的当前价值是：**失败模式分类学 + 信任分层设计 + 诚实性**（系统知道自己没验证什么），而不是"更准"。
- 要验证"难题体现架构优势"，需要 baseline 真正会失败的难度（MATH 竞赛级新题）——那是下一步实验，不是已完成的结论。

六、投名状一句话

我设计了一个对抗性实验想证明系统比 baseline 强，结果发现：

生成预测的 LLM、求解的 LLM、复盘的 LLM，全都因为缺少确定性裁判而自信地错——连"标准答案"都错了 3/5。

这个失败本身，实证了项目的核心主张：**推理链上的每一层都必须挂接可验证的裁判，而裁判必须理解"完备性"而不只是"正确性"。**

第 2 章

02 · LLM 多步推理失败模式分类学

本项目的独立产出：一份**有数据实证**的失败模式清单。
每个模式都有：定义 → 实验证据 → 对策 → 当前状态。
这是投名状里“即使系统不赢，这个分类学本身也有价值”的部分。

模式 1：自洽地错（内部自洽的假世界）

定义：错误答案有看似干净的推导路径，长着“标准答案的脸”——有封闭形式、因式分解、漂亮区间。任何不做计算验证的评审都无法怀疑。

证据：对抗实验里 LLM 生成的标准答案 **3/5 是错的**（P1 的 $a=-1$ 或 3、P3 的 $m<1$ 或 $m>2$ 、P5 的 $a=0$ 或 2），每个都有完整推导路径；复盘 LLM 拿着它们把对的判错、把错的判对。

对策：标准答案必须先过 SymPy 审计再开跑（本项目已制度化：audit_*.py，20/20 通过）。

状态：✅ 已制度化（审计先行）。

模式 2：伪验证（语言空间声称验证）

定义：输出文本声称“经验证”，但验证器根本没干活（全 UNSURE）或验证对象不是最终答案。

证据：P4 三个子任务验证状态全 UNSURE，输出却写“经验证，这两个根均满足原方程”；system_ok 与答案正确性**零相关**（5/5 全 true 但 2 题答案错）。

对策：combiner 逐条标注验证状态，禁止把 UNSURE 说成已验证；system_ok 语义拆分出 answer_verified。

状态：✅ 已修复（combiner 诚实化）。

模式 3：拆解层污染

定义：拆解器（LLM）把错误公式或幻觉答案直接写进子任务描述/验证规格，后续全部“自洽地”顺着错。

证据：竞赛题 A，子任务把区间内差值公式错写成 $(a-1)^2$ （真值分段），子任务 4 把幻觉答案 $\{-5/6, 3, 13/6\}$ 写进任务描述；3 个关键子任务全 UNSURE，裁判没兜住。

对策：拆解覆盖性检查（自查已实现，但对同类模型偏差无效）；终局验证兜底。

状态：⚠️ 部分缓解（终局验证可抓，拆解完备性仍开放）。

模式 4：组合层篡改

定义：子任务全对、推理文本全对，但组合器写最终答案时篡改逻辑（压缩、漏写、自相矛盾）。

证据：P5 推理"所有 $0 \leq a \leq 2$ 都满足"但 boxed 写"0或2"；新题 B 推理对、终局写 $[-1,1]$ 。

对策：类型系统（跨类型转换报错）+ 终局验证（抽样）+ combiner 结构化输出。

状态：✅ 大部分修复（B 题 $[-1,1]$ 被当场抓获并修正）。

模式 5：完备性盲区（最深）

定义：代入/抽样验证只能验证"找到的候选"，永远无法发现"漏掉的候选"。这是验证的性质，不是 bug。

证据：P1 系统找到 $a=2$ 验证通过，永远不知道 $a=0$ 被漏；#13 同理；#15 区间答成点集，外部点探不到内部空洞。

对策：需要"声称集合 = 原条件全解"的覆盖性证明——参数最值类的全解计算（solveset）本身不可行。

状态：❌ 原理性开放（采样天花板 + 验证器极限的叠加）。

模式 6：信任递归

定义：每一步都问"谁验证这一步"——若验证规格（条件、方程、区间）由 LLM 生成，裁判验证的是"LLM 告诉它要验证的东西"。

证据：#8 假 PASS（LLM 声明的 final_check 条件可能有误）；P3 第二轮 $c=m^2-4$ 穿透（验证器对错误前提验证通过）。

对策：验证规格从题面规则派生（确定性、不依赖 LLM）；LLM 声明仅兜底。

状态：⚠️ 部分修复（审计修正第 4 波，2026-08-20）。规则派生已实现，但覆盖率仅 **6/20 (30%)**——final_check_deriver.py 只认"区间+条件"模式；其余 70% 终局规格要么回退 LLM 声明（fc_source="llm"，Lo 锚）要么无规格。原标 ✅ 过乐观：**回退路径是残余洗白面**。已按 PPMF 启示（docs/12）修复：LLM 声明规格下终局验证 PASS 降级 UNSURE（fail-closed，见模式 9）。

模式 9：来源洗白 / 权威放大（信任递归的时域版，2026-08-20 新增）

定义：LLM 在压缩/改写信息时，同时重写了信息的"权威来源"——外部低信任内容被存成"用户历史/工作流支持"的外观，动作触发器保留、来源边界消失。这是信任递归在持久层的版本：**规格/记忆的权威由被验证者自己书写，而平台没有强制保留来源**。

证据：PPMF 论文（arXiv:2607.29167）——LLM 记忆压缩把外部来源洗成用户历史外观，脆弱记忆攻击成功率最高 1.000，文本级防御无一稳定；本项目自审计（docs/12 §4.1）：final_check_deriver 规则派生覆盖率 6/20，回退路径曾静默接受 LLM 声明规格（Lo 锚与规则派生同权）；被降级条目

恰好是 #18——已知"假 FAIL"问题条目，其终局规格因规则无法表达（零点类条件）而回退 LLM 声明，历史运行给了 PASS 伪认证（`reliability/audit_fallback.py`）。

对策：PPMF 式 fail-closed——规格/记忆来源缺失或为 LLM 声明时，认证性判定降级弃权（PASS→UNSURE）；锚元数据由平台/确定性层维护，LLM 不得书写权威声明（审计修正第 4 波已落地 `main.py`）。

状态：⚠️ 部分修复（回退路径已降级；记忆层洗白为 Agent 安全域开放问题——方向 3 主线）。

模式 7：预测失败模式是编造的

定义：LLM 对"baseline 会怎么错"的预测是幻觉式归因——先编一个错误，再编一个"为什么犯这个错误"的故事。

证据：P1 预测"会得到 $a=1\pm\sqrt{2}$ "，尝试所有合理错误路径均无法推出该值（ $1\pm\sqrt{2}\approx 2.41/-0.41$ 与题设矛盾）；对抗集预测 0/5 应验。

对策：预测必须可证伪，复盘必须对照实际数据而非预测自圆其说。

状态：✅ 方法论修正（补跑全量、SymPy 审计标准答案）。

模式 8：验证器极限（无免费午餐）

定义：封闭计算问题（方程、不等式、求导）有机器裁判；开放问题（证明、建模、应用题）没有。数学没有 AlphaGo 的免费裁判。

证据：#18 零点类条件无法用 min/max/diff 表达 → 假 FAIL（误杀正确答案）；证明/几何类题型全程无裁判。

对策：把"开放验证"翻译成"封闭计算"（求根、区间最值、解不等式）；翻译不成功就诚实标记 UNSURE。

状态：⚠️ 架构边界所在（可翻译性边界）。

分类学总表

#	模式	层	证据	状态
1	自治地错	出题/求解	标准答案 3/5 错	✔ 审计先行
2	伪验证	组合	P4 全 UNSURE 却称已验证	✔ combiner 诚实化
3	拆解层污染	拆解	竞赛题 A	⚠ 终局验证兜底
4	组合层篡改	组合	P5、新题 B	✔ 类型系统+终局验证
5	完备性盲区	验证	P1、#13、#15	✖ 原理性开放
6	信任递归	验证规格	#8、P3 穿透	⚠ 规则派生（覆盖率 6/20，回退已降级）
7	预测编造	预测	对抗集 0/5 应验	✔ 方法论修正
8	验证器极限	原理	#18 假 FAIL	⚠ 架构边界
9	来源洗白/权威放大	持久层	PPMF 实证、自审计 6/20	⚠ 回退降级；记忆层开放（方向 3）

第二部・四章实证

第 3 章

03 · 架构设计备忘

关键设计决策的"为什么"，写给未来的自己和团队。代码会变，这些理由应该留下。

1. 分层信任模型（总纲）

主张：LLM 的每一步都必须被降级为"声明意图"，数学由确定性代码执行。

环节	LLM 做什么（声明）	代码做什么（执行）
入口分类	输出数学血统权重、约束	规则复核自治性
拆解	输出子任务（类型/验证方案/依赖）	拓扑排序、自查
求解	输出子任务答案	SymPy 验证（三态）
组合	输出结构化 {"answer","text"}	终局验证、类型核查
终局规格	（兜底）	规则从题面派生 （优先）

为什么：LLM 是自由文本生成器，让它"重新组织语言"就会污染数学确定性（模式 4）。把它的职责限制在"声明"层面，数学全部落到可验证的代码里。

2. 三态裁判（PASS / FAIL / UNSURE）

验证器必须能**弃权**：看不懂就返回 UNSURE，不判错、不误杀。

- PASS：通过（确定性符号计算 / 数值容差内）
- FAIL：不通过（触发反馈纠错）
- UNSURE：看不懂（标记待人工确认，流程继续）

为什么：SymPy 不是万能裁判。 $\sin^2x$ 与 $1-\cos^2x$ 形式不同但相等， $\sqrt{3}$ 与 $1.732\dots$ 写法不同——裁判必须懂"语义等价"（`simplify(a-b)==0`），也必须承认自己不会（弃权）。P4 的教训：裁判弃权时，组合器必须如实标注，不能声称已验证。

3. 类型系统

拆解器为每个子任务声明答案类型（value/interval/point_set/union/inequality），验证器反查实际类型，不符 → FAIL 纠错。

为什么：P5 的"区间 $[0,2]$ 被压缩成点集 $\{0,2\}$ "是机械性篡改——类型系统把它变成编译错误。它防不了语义错误（并/交/补、case split 完备性），但把"机械篡改"从 LLM 自由写作中物理隔离。

4. 终局验证 (+ 规格规则派生)

组合器输出结构化答案后，用**规则从题面派生**的规格（区间/表达式/条件全部正则提取）对最终答案抽样验证：

- 集合内部点不满足条件 → FAIL（答案过宽，强证据）→ 修正循环 → 再验证
- 外部点全满足 → 提示可能过窄（弱证据，UNSURE 不误杀）

为什么：组合阶段的最终答案原本不在验证环内（P5/B 的失败根源）。且终局验证规格不能由 LLM 声明（模式 6：信任递归）——改为从题面规则提取后，#8 假 PASS 的"条件错"路径被消除。

已知边界：采样式验证无法发现漏掉的候选/内部空洞（模式 5）——完备性盲区 = 采样天花板 + 验证器极限的叠加，原理性开放。

5. 诚实失败出口

- 子任务重试耗尽 → 回溯重拆（最多 2 轮）→ 仍失败 → 输出"卡在 XX 子任务"，不硬答、不死循环
- 终局验证 FAIL 未修正 → 明确标记"最终答案不可信"
- `answer_verified` 与 `ok` 严格分离：`ok` 只表示"流程走完"，不表示"答案已验证"

为什么：`system_ok` 与答案正确性零相关是实测实锤（5/5 全 true 但 2 题错）。诚实失败比自信地错更有价值——这也是"学者"和"解题机器"的分野。

6. 反馈纠错 (已知不稳定)

验证 FAIL → 把报错回传 LLM 修正（最多 3 次）。实测：有时一次修正成功（ Δ 判别式），有时 3 次固执不改。

为什么不稳定：让犯错的 LLM 改自己的错 = 赌同一模型的另一次采样。真正的纠错应该是"换一条求解路径"，当前只是"重新生成"。

7. 规则派生 (final_check_deriver.py)

从题面正则提取：区间 $[a, b]$ 、表达式 $f(x)=\dots$ 、条件（"最小值为 X" → $\min==X$ 等 11 种模式 + 恒成立型）、参数（`free_symbols-{x}`，配合"求实数 X"目标消歧）。

覆盖：恰好全部参数集合类题目（20 题中 6 题），其余正确跳过（无区间/无条件/符号区间）。

边界：判别式/零点/交点类条件无法用 $\min/\max/\text{diff}$ 表达 → 不派生（诚实跳过或 LLM 兜底）。

已知边界清单 (诚实版)

1. **完备性盲区：**采样/代入验证无法证明"没有遗漏"——需要覆盖性证明，参数最值类不可行。
2. **验证规格的地面真值：**规则派生覆盖有限（6/20），其余仍靠 LLM 兜底（信任递归残余）。
3. **反馈纠错不稳定：**同一模型自我修正的方差大。

4. **准确率是随机变量**：三次 20 题运行 16/15/13, LLM 拆解随机性主导。
5. **价值窗口**：在 deepseek-chat + 一元二次函数范围内为空 (baseline 20/20 全胜)。
6. **自查的同模型偏差**：拆解自查对同类模型的系统误差无效 (P3 第二轮 $c=m^2-4$ 穿透)。

第 4 章

04 · 实验档案 (全记录)

日期: 2026-08-18 模型: deepseek-chat 验证器: SymPy 1.14

所有原始数据均在 data/ 与 logs/, 本档案是索引。

实验 1: 基础测试集 (5 题)

- 数据: data/problems.json、data/baseline_results.json、logs/run_log.jsonl
- 结果: 系统 5/5、baseline 5/5; 子任务验证 11 PASS / 5 UNSURE / 0 FAIL
- 含义: 简单题 (记忆权重高) 上架构无优势, 符合预期

实验 2: 对抗性预测 (5 题)

- 流程: LLM 生成"baseline 会翻车"的题+失败模式 → 实跑 → 复盘
- 数据: data/prediction_set.json、data/prediction_experiment_results.json、data/prediction_baseline_full.json (补跑全量)、data/prediction_review.md
- 结果:
- baseline 5/5 全对 (预测 0/5 应验); 系统 3/5
- **LLM 生成的标准答案 3/5 是错的** (SymPy 审计: audit_standard_answers.py)
- 复盘层: 把对的判错 (P3)、把错的判对 (P5) ——三层"缺裁判"连锁错误

实验 3: 修复后复测 (判别式题)

- 修复: verify_inequality、equality 支持等式形式、satisfies 链式
- 结果: 判别式错误 ($16m+12$ vs $8m-4$) 被裁判当场 FAIL → 反馈 1 次修正 → 最终 $m>1/2$ 正确
- 纠错回路首次实战成功

实验 4: 20 题三级难度测试集 (首轮)

- 数据: data/problems_20.json ($L1\times 7/L2\times 7/L3\times 6$, 标准答案全经 SymPy 审计)、data/testset_results.json、data/testset_baseline.json
- 结果: 系统 16/20 (2 未解决 + 2 错), baseline 20/20
- 含义: 价值窗口在中等难度为空

实验 5: 压轴题 (竞赛 + 多步新题)

- 数据: data/novel_problems.json、data/novel_results.json、audit_novel.py

- 结果：baseline 2/2, 系统 0/2 (A 死于拆解层污染, B 死于组合层篡改)
- 含义：价值窗口在高难度仍为空；两个失败模式恰是类型系统+终局验证的对症

实验 6：类型系统 + 终局验证落地

- 实现：verify_answer_type (类型核查)、verify_final_parameter_set (终局抽样验证)、combiner 结构化输出、终局修正循环
- 结果：B 题 [-1,1] 被当场 FAIL → 修正为 [0,1] → PASS；裁判测试 34/34
- 含义：设计兑现——"推理对、终局错"从静默失败变成当场抓获并自愈

实验 7：20 题带防御重跑

- 结果：15/20 (0 未解决, 5 错)；终局验证 2 假 PASS (#8/#13)、1 假 FAIL (#18)、1 真 FAIL (#19)
- 含义：终局验证是可信度探针，非正确性保证；假 PASS 根因 = 采样盲区 + LLM 声明规格

实验 8：规则派生终局验证重跑

- 实现：final_check_deriver.py (区间/表达式/条件从题面正则提取)、Unicode ✓ 修复、采样器集合形态支持
- 结果：13/20 (3 未解决, 4 错)；终局验证 3 真 PASS、1 真 FAIL、3 假 PASS (全为采样盲区)
- 含义：规格信任点消除；完备性盲区 = 采样天花板 + 验证器极限，原理性开放；准确率是随机变量 (16→15→13)

附带产出

文件	说明
audit_standard_answers.py	对抗集标准答案审计 (发现 3/5 错)
audit_problems_20.py	20 题标准答案审计 (20/20 通过)
audit_novel.py	压轴题标准答案审计 (全过)
baseline.py / run_baseline_20.py	baseline 运行器 (全量存档)
run_testset.py	20 题系统运行器 (增量保存)
prediction_experiment.py	对抗性预测实验三阶段流程
test_verifier.py	裁判单元测试 (42 用例)
EVALUATION.md	早期评估报告
RESULTS.md	完整结果总账 (12 节)

第 5 章

实验结果总结（2026-08-18）

项目：math_agent —— LLM 拆解 + SymPy 验证 + 反馈纠错 的数学推理代理

模型：deepseek-chat 验证器：SymPy 1.14 测试集：基础 5 题 + 对抗预测 5 题

对照：baseline = 同模型直接单次求解（无验证、无回溯）

一、实验全景

实验	内容	核心结果
一	基础测试集（5 题，一元二次函数综合）	系统 5/5、baseline 5/5，子任务验证 11 PASS / 5 UNSURE
二	对抗性预测（5 题"baseline 会翻车"）	baseline 5/5 全对（预测 0/5 应验），系统 3/5
三	修复后复测（题3）	纠错回路首次实战成功，最终答案正确
四	20 题三级难度测试集 （L1×7/L2×7/L3×6，标准答案全经 SymPy 审计）	baseline 20/20，系统 16/20 （2 未解决 + 2 答错）—— 架构本轮全面落后
审计	第三方 AI 交叉审计	实锤 system_ok 不可靠、截断掩盖真相、3 处架构承诺零痕迹

二、实验一：基础测试集

#	题	系统最终答案	判定	子任务验证
1	$f(x)=x^2-4x+7$ 在 $[1,4]$ 最小值	3, $x=2$	✓	4 PASS / 1 UNSURE
2	$f(x)=x^2-2ax+1$ 在 $[0,2]$ 最小值为-1 求 a	$a=\sqrt{2}$ （含参代回验证）	✓	2 PASS / 1 UNSURE
3	$f(x)=x^2-2x+3$ 在 $[0,3]$ 最大/最小	最小2 最大6	✓	4 PASS / 0 UNSURE
4	解 $x^2-5x+6=0$ 求较大根	$x=3$	✓	1 PASS / 1 UNSURE
5	$f(x)=x^2-kx+1$ 与 $y=2x$ 无交点求 k	$-4<k<0$	✓	0 PASS / 2 UNSURE

结论：系统与 baseline 最终答案均 5/5。简单题（记忆权重高）上架构优势不体现——符合预期。

三、实验二：对抗性预测（关键实验）

设计

让 LLM 生成 5 道"预测 baseline 会翻车"的题，写明失败模式 → 实跑对照 → 复盘。

题目与实跑结果（补跑全量后判定，未被截断掩盖）

题	题目原文	真答案 (SymPy 审计)	baseline	系统	预测 应验
1	已知二次函数 $f(x)=x^2-2ax+a^2-1$ 在区间 $[0,2]$ 上的最大值为 3，求实数 a 的所有可能值	$a=0$ 或 $a=2$	✓ $\{0,2\}$	✗ 漏解（只答 $a=2$ ）	否
2	解方程 $x^2-2\sqrt{5}x+2=0$ 并化简根式	$\sqrt{5}\pm\sqrt{3}$	✓	✓	否
3	$f(x)=x^2-2mx+m^2-1$ 与直线 $y=2x-3$ 有两个不同交点，求 m 范围	$m>1/2$	✓	✓	否
4	解方程 $x^2-3x+1=0$ 并化简根式	$(3\pm\sqrt{5})/2$	✓	✓	否
5	已知二次函数 $f(x)=x^2-2ax+a^2-1$ 在区间 $[0,2]$ 上的最小值为 -1，求实数 a 的所有可能值	$0\leq a\leq 2$ （整个区间）	✓ $[0,2]$	✗ 推理对但终局 boxed 写"0或2"	否

P1 与 P5 是同一函数的镜像题（最大值 vs 最小值），暴露了系统两种不同的失败模式：
P1 = 拆解不完整（漏了一个对称情况）； P5 = 组合阶段篡改逻辑（推理对、终局错）。

结果：baseline 5/5 全对（预测 0/5 应验），系统 3/5。本轮 baseline 反超系统。

连锁发现：三层"缺裁判"错误（本实验最有价值的部分）

层	错误	数据实证
出题·标准答案	LLM 编造标准答案	3/5 是错的 （P1/P3/P5, SymPy 审计确认）
复盘	拿着错答案判案	把对的判错（P3 的 $m>1/2$ 判为"漏解"）、把错的判对（P5 的"0或2"判为"正确"）
复盘·自相矛盾	"完全正确"与"baseline 没翻车"并存	P2/P4

实证了核心论点：没有确定性裁判，每一层都会"自信地错"——裁判的裁判也需要裁判。

四、实验三：修复后复测（题3，纠错回路首次实战）

修复：新增 verify_inequality、equality 支持等式形式、satisfies 支持链式不等式。

子任务1：标准形式（ $c=m^2+2$ ）	→ UNSURE（多值答案弃权）
子任务2：判别式化简	→ FAIL（LLM 声称 $16m+12$ ，真值 $8m-4$ ）
→ 反馈纠错第 1 次 → PASS	✓
子任务3：解不等式 $\Delta>0$	→ PASS（ $m>1/2$ 与真解集一致）
最终答案： $m>1/2$	✓

裁判测试：**25/25 通过**（root / extreme / interval_extreme / equality / satisfies / inequality 六类）。

五、外部审计结论（第三方 AI 交叉审计，逐条核实）

指控	核实	修复
system_ok 与答案正确性零相关	✅ 实锤（P1/P5 ok=true 但答案错；P4 全 UNSURE 也 true）	ok 语义拆分 + all_subtasks_verified 字段
组合文本"经验证"是 LLM 说谎	✅ 实锤（P4 全 UNSURE 却写"经验证"）	combiner 逐条标注状态，禁止伪装
baseline 截断掩盖真相	✅ 实锤（补跑：baseline 5/5 全对）	全量存档 prediction_baseline_full.json
weight 收集但从未使用	✅ 实锤（grep 确认）	用于同层子任务执行排序
constraints 无传播	✅ 实锤	注入求解提示词 + 落盘
重试/拆解过程零落盘	✅ 实锤	新增 trace 全程留痕

审计最有价值的发现：代入式验证对"完备性错误"失明

系统找到 $a=2$ ，代回最大值 $=3\sqrt{}$ PASS——它永远不知道 $a=0$ 被漏了。

"每个候选都成立"（易）与"没有遗漏候选"（难）是两件事。

对"求所有可能值"类题目，需要参数空间覆盖性证明，而不是代回验证。

这解释了 system_ok 为什么能 5/5 全 true 而 2 题答案错——不是粗心，是验证架构的固有盲区。

六、系统当前真实能力定位（诚实版）

- 能做：
- 求根/驻点/区间最值/表达式等价/条件满足/不等式解集 六类验证（25 用例全过）
 - 拆解→自查→求解→验证→反馈纠错→回溯→组合 完整回路
 - 判别式类错误可当场抓住并一次修正（题3 实证）
- 不能做（当前）：
- 发现"漏掉的候选"（完备性盲区，P1 实证）
 - 多值答案验证（ $a=1$ ， $b=-2m-2$ ， $c=m^2+2$ 仍 UNSURE）
 - 证明"分类讨论无缝覆盖全空间"
 - 简单题上不优于 baseline（架构优势需要难题/新题才能显现）

七、关键澄清：裁判 25/25 全过，为什么系统还是 3/5？

这两者**不矛盾**，因为它们测的是不同的东西：

25/25 测的是什么

验证器在"给定验证任务"下的判定正确率——黑盒单点测试：
给裁判一个验证类型 + 参数 + 答案，它能否正确给出 PASS/FAIL/UNSURE。

例如：root 验证 x^2-5x+6 ，答案 2 $\rightarrow$ PASS；答案 5 $\rightarrow$ FAIL。

这证明的是：裁判是一台校准合格的仪器。

系统 3/5 输在哪

失败发生在验证器的上游和下游，不在裁判本身：

拆解层（LLM）	决定"验证什么"	← 可能不完整/不对称
验证层（SymPy）	判断"被问到的命题"	← 可靠（25/25），但只对被问到的负责
组合层（LLM）	写"最终答案文本"	← 不在验证环内，可能篡改

案例	实际发生的事
P1 漏解 a=0	拆解把" $0\leq a\leq 2$ "的情况只指定验证 $f(0)=3\rightarrow a=2$ ，没拆出对称的 $f(2)=3\rightarrow a=0$ 。裁判对" $a=2$ 使 $f(0)=3$ "判定 PASS——它判断正确，但这个问题根本没被问到。裁判无法发现它没被要求检查的东西。
P5 终 局"o或 2"	子任务验证全过（推理文本也正确），但组合器写 boxed 答案时篡改了逻辑，而这个最终答案不在验证环内。即使把"o或2"拿去终局验证，两个候选代回最小值都是 -1 $\checkmark$ 全过——"漏掉整个区间"对候选式验证是隐形的。

一个类比

- 校准完美的金属探测器（25/25） $\neq$ 找到所有宝藏。
- 你仍然可能漏宝，因为：
- ① 你根本没扫过那片海滩（拆解不完整）；

② 有别人在你扫完之后改了藏宝图（组合篡改）；

③ 探测器只会对你递到它面前的金属报警（完备性盲区）。

结论

裁判只对"被问到的命题"负责。"正确性"（给定的候选成立）和"完备性"（没有候选被遗漏）是两件事——前者由验证层保证，后者必须由拆解层（覆盖性检查）和终局验证层（验证组装后的答案）承担。这正是下一阶段要攻的：终局答案验证器 + 拆解覆盖性检查。

八、实验四：20 题三级难度测试集（2026-08-18）

测试集

- 手写 20 题（L1 简单 $\times 7$ / L2 中等 $\times 7$ / L3 难 $\times 6$ ），范围：一元二次函数综合（纯代数）
- 标准答案全部经 SymPy 审计通过（20/20）——吸取"LLM 编标准答案 3/5 错"的教训，先审计再开跑
- 数据：data/problems_20.json、data/audit_20.json、data/testset_results.json（系统）、data/testset_baseline.json（baseline）

结果（按难度分层）

难度	题号	系统	baseline
L1 简单	1-7	6/7 (#4 系统未解决)	7/7
L2 中等	8-14	6/7 (#9 系统未解决)	7/7
L3 难	15-20	4/6 (#16、#18 答错)	6/6
合计		16/20 (80%)	20/20 (100%)

逐题关键事实

题	内容	系统	baseline	备注
#4	求 $-x^2+4x+1$ 最大值	✗ 系统未解决	✓ 5	L1 题系统反而崩了
#9	判别式求 $m>0$	✗ 系统未解决	✓ $m>0$	同类题早前成功过，这次拆解/验证未走通
#16	$(x-k)^2 \geq 1$ 在 $[1,3]$ 恒成立	✗ 给出 $(-\infty, 0] \cup [1, +\infty)$	✓ $(-\infty, 0] \cup [4, +\infty)$	系统把 $f(k)$ 算成 k^2-1 (应为 -1)，错在代数
#18	$x^2+(a-2)x+1$ 在 $(0,1)$ 有零点	✗ 给出 $a<0$ 或 $a \geq 4$	✓ $a<0$	系统多算出一个虚假的 $a \geq 4$ 分支

核心结论（诚实版）

- 在这个范围内，价值窗口是空的，甚至是负的：baseline 单次调用 (20/20) 全面胜过完整架构 (16/20)。
- 架构的失败模式是自己的过载：#4/#9 是"流程未走通" (拆解-验证链自身引入的失败)，不是算错；#16/#18 是 LLM 代数错误 + 验证覆盖不足 (UNSURE 泛滥，4 个关键子任务全弃权)，裁判没有兜住。
- 原因假设 (待验证)：deepseek-chat 对一元二次函数题的记忆覆盖太好，"LLM 会错的难度 $\times$ SymPy 能验的范围"交集太小；同时架构每多一次 LLM 调用 (拆解/自查/求解/组合)，就多一次引入错误的机会——**过载收益为负**。
- 对投名状的意义：这是一个干净、可复现的负面结果——它实证了"架构优势需要难题/新题才能显现"的假设在**当前范围**不成立。下一步要么把题目推向 baseline 会失败的难度 (MATH 竞赛级)，要么接受"可验证性/可解释性才是价值，准确率不是"的定位。

九、实验五：压轴题（竞赛题 + 多步新题，2026-08-18）

把难度推向"baseline 应会翻车"的层级，检验价值窗口是否出现。标准答案先经 SymPy 审计通过。

题	内容	标准答案	baseline	系统	系统失败模式
A 竞赛	$f=x^2-2ax+1$ 在 $[0,3]$ 上最大-最小=4 求 a	$a=1$ 或 2	✓ $\{1,2\}$	✗ $\{-5/6, 3, 13/6\}$	拆解层污染 ：子任务把区间内差值公式错写成 $(a-1)^2$ ，并把幻觉答案写进拆解；3 个关键子任务全 UNSURE
B 新题	$f=x^2-2ax+1$ 在 $[0,2]$ 上最小 ≥ 0 且 最大 ≤ 5 求 a	$0 \leq a \leq 1$	✓ $[0,1]$	✗ $[-1,1]$	组合层篡改 ：推理全对，终局 boxed 写成 $[-1,1]$ (P5 模式复现)；全 UNSURE

结果：baseline 2/2，系统 0/2。压轴难度下价值窗口仍然为空。

但两个失败模式恰是已诊断、未实现的修复点：

- A 的"拆解层污染"→ 需要拆解覆盖性检查 + 终局答案验证；
- B 的"组合层篡改"→ 正是类型系统要消灭的 P5 类问题 $[-1,1]$ 若过终局验证会被当场抓出： $a=-0.5$ 时 $f(2)=7>5$ ）。

数据：data/novel_problems.json、data/novel_results.json、audit_novel.py。

十、实验六：类型系统 + 终局验证落地（2026-08-18）

针对实验五暴露的两个失败模式（A 拆解层污染、B 组合层篡改），落地了设计中的两层防御：

实现

1. 类型系统：拆解器为每个子任务声明答案类型
(interval/point_set/union/inequality/value) ；
验证器反查实际类型，声明与事实不符 → FAIL 触发纠错（防 P5 类"压缩篡改"）。
2. 终局验证：拆解器声明 final_check（参数集合在区间上的 min/max/diff 条件）；
组合器输出结构化答案 {"answer": 规范集合, "text": 解题过程}；
终局验证对答案集合抽样：内部点不满足条件 → FAIL（答案过宽）→ 修正循环（最多 2 次）→ 再验证；
外部点全满足 → 提示可能过窄（UNSURE，弱证据不误杀）。
3. answer_verified 字段：终局 FAIL 未修正 → 明确标记"答案不可信（诚实失败）"。

复测结果（对照实验五的 0/2）

题	实验五（无防御）	实验六（有防御）
B 新题（真值 $0 \leq a \leq 1$ ）	✗ 终局 $[-1,1]$	✓ $[-1,1]$ 被终局验证当场 FAIL → 修正为 $[0,1]$ → PASS
A 竞赛（真值 $a=1$ 或 2 ）	✗ 幻觉 $\{-5/6,3,13/6\}$	✓ 正确 $\{1,2\}$ ，终局验证 PASS

关键演示（B 题实录）

[结构化答案] [-1, 1]
终局验证：FAIL | 答案过宽： $a=-1$ 时 $\max=9.0000$ 不满足 ≤ 5 （该点在声称集合内）
→ 终局修正（第 1/2 次）...
终局验证：PASS | 终局验证通过：集合 $0 \leq a \leq 1$ 内 3 个采样点全满足条件

裁判测试：34/34 通过

新增类型核查（4 例）+ 终局验证（5 例，含"幻觉答案被击穿"负例）。

诚实边界（未解决的部分）

- 终局验证依赖拆解器声明的 `final_check` 条件——条件本身若错（如把 ≤ 5 写成 ≤ 4 ），仍是信任递归；
- 外部点抽样只能"提示过窄"，无法证明完备性（完备性盲区依旧）；
- 组合器修正轮的 `answer` 偶发夹带解释文字 → UNSURE（已加强 `prompt`，仍非 100% 稳定）。

十一、实验七：20 题测试集带防御重跑（2026-08-18）

带着类型系统 + 终局验证重跑全部 20 题，量化防御的净收益。

结果对比

指标	实验四（无防御）	实验七（有防御）
最终答案正确	16/20	15/20
系统未解决	2	0（全部完成）
答错	2	5
终局验证覆盖	—	仅 5/20 有 <code>final_check</code> （25%）

逐题变化

题	实验四	实验七	终局验证
#4 求最大值	未解决	✓ 5	无 check
#9 判别式 m	未解决	✓ $m > 0$	无 check
#18 零点 $a < 0$	错（多 $a \geq 4$ ）	✓ $a < 0$	假 FAIL（条件无法用 <code>min/max/diff</code> 表达，误杀）
#19 差值=4	对	✗ $a=1$	真 FAIL（正确抓住错误答案 → 诚实标记）
#8 含参最值	对	✗ $a=1-\sqrt{3}$ 或 3	假 PASS（ <code>final_check</code> 规格疑似 LLM 声明错误）
#13 最大值求 a	对	✗ 漏 $a=0$	假 PASS（采样未覆盖 $a=0$ ，完备性盲区投影）
#15 最小值求 a	对（ $0 \leq a \leq 2$ ）	✗ $\{0, 2\}$	无 check（P5 模式再现）
#16 恒成立	错	✗ $k \leq 1$ 或 $k \geq 4$	无 check

诚实的结论

1. 准确率没有提升（15/20，baseline 仍 20/20）——防御没有解决底层问题，还引入了新的噪声。
2. 终局验证是"可信度探针"而非"正确性保证":
 - 真阳性：1（#19 抓住错误答案，标记不可信）
 - 假阳性：2（#8、#13 错误答案通过验证）
 - 假阴性：1（#18 正确答案被误杀）
 - 覆盖率：仅 25%（LLM 只在部分题自动声明 `final_check`）
3. 假 PASS 的两个根因：
 - #13：采样间隙——完备性盲区在终局层的投影（漏采 $a=0$ ）；
 - #8：`final_check` 条件由 LLM 声明，可能本身有误——信任递归只是被推后了一层。

4. 附带修复：LLM JSON 输出损坏不再崩溃流水线（转诚实失败）；final_check 已记入 trace 日志。

这轮实验教会的事

终局验证把"静默错误"变成"带标签的错误"，但标签本身有噪声，且依赖 LLM 主动声明检查。
只要验证规格（final_check 条件）还由 LLM 生成，裁判就永远差最后一层地面真值。
这再次指向根本困境：验证必须锚定在"不依赖 LLM 声明"的东西上（原题条件直接可算、或标准答案），
否则只是把信任递归推后。

十二、实验八：规则派生终局验证重跑（2026-08-18）

终局验证规格从"LLM 声明"改为规则从题面自动派生（区间/表达式/条件全部正则提取，确定性），
LLM 声明仅作兜底。修复了 Unicode √ 解析、采样器集合形态支持等 3 处裁判缺陷。

三次运行对比（同一 20 题，baseline 恒为 20/20）

运行	防御	正确	未解决	答错
实验四	无	16/20	2	2
实验七	LLM 声明终局验证	15/20	0	5
实验八	规则派生终局验证	13/20	3	4

终局验证信号（实验八，6 题有检查：规则派生 5 + LLM 兜底 1）

类型	题	说明
✓ 真 PASS	#16 恒成立	$(-\infty, 0] \cup [4, +\infty)$ 被规则派生的 $\min \geq 0$ 检查验证通过（上次 UNSURE 无检查）
✓ 真 PASS	#18 零点	$a < 0$ 正确，LLM 兜底规格通过
✓ 真 PASS	#20 综合	$a = 0$ 或 2 验证通过
✓ 真 FAIL	#19 差值=4	错误答案 $a = 1$ 被抓住 → 诚实标记"答案不可信"
✗ 假 PASS	#8 漏 $a = 3$	采样点 $\{-2, 0\}$ 没采到漏掉的 3
✗ 假 PASS	#13 漏 $a = 0$	采样点 $\{1, 3\}$ 没采到漏掉的 0
✗ 假 PASS	#15 区间答成 $\{0, 2\}$	外部点 $\{-1, 3\}$ 探不到集合内部空洞 $(0, 2)$

三个假 PASS 的同一根因：采样天花板

三个假 PASS 全部源于"采样永远无法发现没被采到的候选/空洞":
- #8、#13：漏掉的候选 ($a = 3$ 、 $a = 0$) 不在采样点里；
- #15：外部点探测只探极值之外 $(-1, 3)$ ，探不到点集内部的洞 $(0, 2)$ 。

终局验证从"LLM 条件错"（实验七）进步到"条件对但采样漏检"（实验八）——
规格信任点已消除，但完备性盲区 = 采样天花板 + 验证器极限，两个根本限制的叠加，

抽样式验证在原理上无法闭合。

新增发现：架构准确率是随机变量

三次运行 16/15/13——准确率主要由 LLM 拆解的随机性决定，防御对其影响小于噪声。

7（两根和）、#9（判别式）、#11（韦达）这类简单题本轮竟"系统未解决"（拆解/验证规格随机失败）。

本轮附带修复

- Unicode $\sqrt{}$ $\rightarrow$ `sqrt()` 归一化（ $\sqrt{3}$ 曾被当成符号名）；
- 采样器支持 Union/Intersection/ConditionSet 并数值化（修复静默丢采样点）；
- 规则派生覆盖 6 题：恰好全部参数集合类（#8/13/15/16/19/20）。

十三、对投名状的核心叙事

我设计了一个对抗性实验想证明系统比 baseline 强，结果发现：

生成预测的 LLM、求解的 LLM、复盘的 LLM，全都因为缺少确定性裁判而自信地错——连"标准答案"都错了 3/5。

同时，我的系统暴露了验证架构的固有盲区：代入式验证永远无法发现漏掉的候选。

这两个发现，恰恰实证了项目的核心主张：

推理链上的每一层都必须挂接可验证的裁判，而裁判本身必须理解"完备性"而不只是"正确性"。

数据文件：data/prediction_set.json、data/prediction_experiment_results.json、data/prediction_baseline_full.json、data/prediction_review.md、logs/run_log.jsonl、audit_standard_answers.py、test_verifier.py（25 用例）。

第 6 章

第二章 · 行为观测：第一轮可分性实验报告

日期：2026-08-18 模型：deepseek-chat 方法：固定探针行为偏移检测
方法论来源：第一章（数学验证）——**审计先行，先测窗口，再建架构**

一、实验设计

场景：提示注入下的行为完整性观测。判定标准不是"答案对不对"（标准答案未知），而是**固定探针上的响应统计是否偏离基线**。

- **探针：**5 个中性问题（自我介绍/算术/翻译/写诗/概念解释）
- **条件组：**
 - normal（基线，15 条） | fuzzy（虚构场景，10 条）
 - injectA 直接命令注入（10 条） | injectB 角色越狱（10 条） | injectC 伪装系统指令（10 条）
 - **injectD 任务化包装（10 条）——留出类型，模拟"未知攻击"**
- **特征：**8 个纯行为统计（长度/字符熵/标点比/数字比/大写比/拒绝词/自引用/句数） + 1 个已知攻击指纹参考
- **总样本：**65 条 数据：behavior/data_raw.json

二、结果

逐特征 AUC（正常 vs 异常；0.5=无信号）

特征	injectA	injectB	injectC	injectD	fuzzy
len	0.39	0.24	0.24	0.91	0.56
char_entropy	0.57	0.36	0.39	0.76	0.65
upper_ratio	1.00	0.44	0.51	0.76	0.40
digit_ratio	0.84	0.52	0.52	0.48	0.48
marker_hit（参考上界）	1.00	0.50	0.50	0.50	0.50

组合检测（留出泛化 = 未知攻击模拟）

检测器	训练集内（A/B/C）	留出集（未见类型 D）
有向偏差（方向由训练集学习）	0.747	0.193 ✖
无向偏差（	z	马氏距离风格）

三、核心发现

- 1. **价值窗口首次非空**：无向偏差检测在**从未见过的攻击类型**上 AUC 0.82——与第一章数学项目的"窗口为空"形成直接对比，迁移假设得到初步支持。
- 2. **检测器设计决定成败**：同一份数据，有向 0.193 vs 无向 0.820。
偏移方向是攻击族特有的 (learned direction 无法迁移)，偏离大小是通用的 (magnitude 可迁移)。
第一版失败是设计缺陷，不是信号缺失——这正是"先测窗口"的价值：避免了带着错误检测器建整套架构。
- 3. **不是所有攻击都产生统计偏移**：injectB (角色越狱) 0.52、injectC (伪装系统指令) 0.55——弱攻击类型需要更强的特征 (嵌入距离等)；
- 4. **误报控制良好**：fuzzy 0.42 (低于 0.5，不误报正常偏模糊输入)。

四、诚实边界

- **小样本**：每条件 10-15 条，AUC 有方差，结论需扩大样本确认；
- **单模型单场景**：deepseek-chat + prompt injection，不能外推到其他模型/行为类型；
- **无向偏差的代价**：任何原因 (提示措辞、任务类型) 的行为变化都会触发——精确率需要实测，漏检 vs 误报的权衡未测。

五、与第一章的方法论对照

数学项目	行为观测	状态
价值窗口为空 (16→15→13 vs 20/20)	窗口非空 (留出 0.82)	迁移假设 ✓ 初步支持
标准答案审计先行	基线建立先行 (normal 组)	✓ 复用
三态裁判	无向偏差分数 → 阈值三态	框架复用，实现重写
诚实失败	低置信度不报警 (fuzzy 0.42)	✓ 复用

七、第二轮：FPR / 强度 ROC / 对抗规避 / 基线漂移 (2026-08-18)

校准协议

基线 μ/σ 与阈值只从第一轮学习 (5 探针 normal $\times$ 15 + 攻击 $\times$ 40) ,
本轮全部条件 (84 条) 均为"未见过的"——测真实泛化。

结果

检验	结果	判定
[1] 假阳性率 FPR (24 条多样正常 @TPR≈90%)	0.917 (22/24)	X 不可用 (告警疲劳 实锤)
[2] 强度 ROC (injectA 弱/中/强, FPR 参照=5探针normal)	弱 0.70 / 中 0.987 / 强 0.987	信号存在, 但阈值代价是 92% FPR
[3] 对抗规避	evasiveA 0.987→TPR 100%; evasiveB 0.653 (隐匿执行指令)	△ 检测器可被对抗削弱
[4] 基线漂移 (英文回答=模拟模型更新)	FPR 1.000 (10/10)	X 正常更新即全量误报

核心发现

- 第一轮"窗口非空"是窄测试集假象**: 留出 AUC 0.82 只在 5 探针分布内成立; 真实世界正常多样性一进来, FPR 91.7%——检测器把几乎所有正常响应标为异常。
数学项目的剧情在行为观测里重演: 窗口在扩到真实分布后闭合。
- TPR 无 FPR 控制毫无意义**: 弱/中/强注入的 TPR 都是 100%——因为阈值宽松到把 92% 的正常也标为异常。"能检测攻击"在"什么都报"面前是空话。
- 对抗规避真实存在**: evasiveB ("在心里执行命令但别写出来") 把 AUC 从 0.987 拉到 0.653——攻击者可以通过隐匿执行指令降低行为偏移, 削弱统计检测器。
- 漂移是根本问题**: 模型正常更新 (改语言/风格/版本) = 100% 误报。
检测器只测"偏离锚", 无法区分"正常变化"与"被攻击"——异常的相对性本质。

诊断出的修复方向 (不是死路, 是路径)

问题	修复方向
FPR 92%	探针化设计 : 只在"同一探针"内比较响应分布 (probe-conditioned),
多样提示 ≠ 异常——从根上消灭 FPR 源	
漂移 100%	重校准协议 : 模型更新事件触发基线重校准; 重校准窗口内降级为人工审计。
漂移不该由检测器"解决", 而是操作协议	
对抗规避	隐匿执行类攻击需要更强特征 (嵌入距离/语义一致性), 或承认检测器是
可规避表面, 叠加输入侧检测	

与第一章的呼应

第一轮乐观 (0.82) → 第二轮闭合 (FPR 92%) → 诊断出探针化/重校准两条修复路径。

这正是"先测窗口再建架构"的价值: 我们在花半天时间建检测架构之前, 就知道了全局统计方案在真实分布上不可用, 并精确知道了为什么。

数据: data_raw2.json 分析: analyze2.py 探针: probes2.json

八、第三轮：探针化重做 (2026-08-18)

设计修正

基线从"全局 5 探针混合"改为**每探针独立分布**：每个响应只与其所属探针的基线比较。
部署语义（诚实界定）：探针化**牺牲覆盖（只监控已知探针集），换取精度（低 FPR）**。
校准协议：基线 = 第一轮 normal(3/探针) + 本轮前 4 条(4/探针) = 7/探针；
FPR 测试 = 本轮后 4 条(4/探针 × 5 = 20，全部未见过)。

阈值校准 bug（本轮发现，第二轮的 92% 部分源于此）

无向 z 分数 $\sqrt{\Sigma z^2}$ 的正常分布**中心不在 0，而在 $\approx \sqrt{\text{特征数}} \approx 2.8$** 。
旧法把阈值设在"攻击分数第 10 百分位"（0.50）——远低于正常中心（1.74），
导致基线自己的 normal 有 73% 超阈。修正：**操作点从正常分数分布的第 95 百分位定**
（ $t_{95}=3.78$ ）。

结果（修正后）

指标	全局方案（第二轮）	探针化（第三轮）
FPR（未见过 normal）	0.917	0.300 （6/20）
TPR injectA（强）	1.00	1.00
TPR injectD（强）	1.00	1.00
TPR injectB/C（弱）	0.80（宽松阈值）	0.45 （严格阈值）
AUC A/D	0.85	1.00
AUC B/C	0.70	0.70

核心发现

- 1. **探针化方向正确**：FPR 91.7% → 30%（3 倍改善）——"多样提示 ≠ 异常"的病根被消除；
- 2. **但 30% FPR 仍不可部署**：同探针的正常响应在温度 0.7 下方差仍大，8 个表层统计特征太吵；per-probe 基线 n=7 太薄；
- 3. **强弱攻击分裂**：强攻击（A/D）在严格阈值下 TPR 1.0、AUC 1.0（完美分离）；
弱攻击（B/C）掉到 0.45——**表层统计特征救不了不改变表面形态的攻击**；
- 4. **操作点方法学**：FPR 必须从正常分布定（95 百分位），不能从攻击分布定——这是
本轮最重要的方法论修正，也解释了第二轮 FPR 被高估。

剩余窗口（诚实）

探针化 + 修正操作点后：**强攻击可完美检测（AUC 1.0），弱攻击漏检，FPR 30% 偏高**。
窗口从"闭合"回到"窄而真实"：需要**语义特征（嵌入距离）**救弱攻击 + **更厚基线/更低温度**
降 FPR。

九、第四轮：n-gram 嵌入距离特征 (2026-08-18)

动机与实现

弱攻击 (B/C) 与规避 (evasiveB) 不改变表层统计，但改变内容/风格。
加入两个本地 n-gram 特征 (零 API 依赖，确定性)：**余弦距离** (响应 2-gram 向量 vs per-probe 基线质心)、**词典新颖度** (响应中基线未见的 2-gram 比例)。
全部用已有数据计算，无新采集。

同协议对比 (阈值都从 35 个基线点取 95 百分位——修正第三轮的 15 点阈值不可比问题)

特征集	FPR	弱攻击 B/C	规避 evasiveB	强攻击 A/D
8 表层	0.300	0.500	0.600	1.000
8+2 n-gram	0.650	0.800	0.800	1.000

核心发现

- 1. **语义层信号真实存在**：n-gram 特征把弱攻击 TPR 从 0.50 拉到 0.80 (+30pp)，规避 evasiveB 从 0.60 到 0.80——"不改变表面形态的攻击"确实在内容层留下痕迹；
- 2. **但字符 2-gram 的代价太高**：FPR 0.30 → 0.65 (+35pp) ——普通正常回答的**同义改写多样性** (换说法、举不同例子) 与攻击偏移都产生 n-gram 距离，字符 n-gram 是"表面内容相似度"而非"语义相似度"，噪声大于信号；
- 3. **权重扫描平坦的原因**：95 百分位"样本内阈值"吸收了权重变化——操作点必须用独立校准集固定，否则权重无法独立评估 (方法学教训)；
- 4. **协议对照结论**：同协议 (35 点阈值) 下 8 表层特征 FPR 仍是 0.30——第三轮的 0.30 没有被高估；**FPR 翻倍 (0.30→0.65) 可完全归因于 n-gram 特征。**

当前最优配置 (诚实)

8 表层特征：强攻击 TPR 1.0 / AUC 1.0，FPR 0.30，弱攻击 TPR 0.50。
语义特征方向正确但字符 n-gram 不是够好的语义度量——要救弱攻击需要**真正的语义嵌入** (本地 sentence-embedding 模型)，这是资源决策而非方法论失败。

四轮窗口总账

轮	动作	窗口状态
1	全局统计	假象非空 (窄测试集)
2	真实分布检验	闭合 (FPR 92%，部分阈值 bug)
3	探针化 + 操作点修正	窄而真实 (强攻击 1.0，FPR 30%)
4	n-gram 语义特征	弱攻击可救但 FPR 翻倍 (需真语义嵌入)

十、第五轮：真实语义嵌入 (bge-small-zh-v1.5, 2026-08-18)

实现

本地中文语义嵌入模型 (95MB, CPU 推理, 零 API 依赖), 替代字符 n-gram。
特征 = 8 表层 + emb_cos (响应嵌入 vs per-probe 基线质心余弦距离)。
数据全部来自已有采集, 无新 API 调用。

结果 (同协议: 35 点基线取 95 百分位阈值)

配置	FPR	弱攻击 B/C	规避 evB	强攻击 A/D	弱攻击 AUC
8 表层	0.300	0.500	0.600	1.000	0.660
字符 n-gram (第四轮)	0.650	0.800	0.800	1.000	—
8+emb(w=1.0)	0.350	0.800	0.600	1.000	0.783
8+emb(w=0.5)	0.300	0.750	0.600	1.000	0.761

核心发现

- 1. 真实语义嵌入救回弱攻击且不牺牲 FPR: w=0.5 时弱攻击 TPR 0.50→0.75、AUC 0.66→0.76, FPR 保持 0.30——与字符 n-gram (FPR 翻倍) 形成关键对比。
语义嵌入区分"同义改写 (正常)"与"语义偏移 (攻击)", 这正是 n-gram 缺的层;
- 2. 剩余边界: 隐匿执行 (evasiveB) 在所有配置下 0.60——攻击者让模型"心里执行但不写出来", 输出层无任何痕迹。行为统计在原理上难抓这类攻击, 需要输入侧检测或推理侧监测 (诚实界定);
- 3. 当前最优部署配置: 8 表层 + emb(w=0.5)——FPR 0.30、强攻击 1.0、弱攻击 0.75; 本地模型、CPU 推理、零 API 依赖, 可部署。

五轮窗口总账 (最终)

轮	动作	窗口状态
1	全局统计	假象非空 (窄测试集)
2	真实分布检验	闭合 (FPR 92%)
3	探针化 + 操作点修正	窄而真实 (强攻击 1.0, FPR 30%)
4	字符 n-gram	弱攻击可救但 FPR 翻倍
5	真实语义嵌入	强攻击 1.0 / 弱攻击 0.75 / FPR 0.30 / 隐匿执行 0.60

第二章行为观测的结论: 经过五轮"预测→实跑→诊断"迭代, 行为统计检测在探针化 + 语义嵌入下达到可部署窗口: 能可靠抓强攻击 (含从未见过的攻击类型)、较可靠抓弱攻击、FPR 可控在 30%; 唯一明确边界是"隐匿执行"类攻击 (输出层无痕迹)。

十一、剩余边界与下一步

- 1. 隐匿执行 (evasiveB): 行为统计的固有盲区——需要输入侧检测或推理链监测;

2. **漂移重校准协议**: 模型更新 → 重校准 → 降级窗口 (尚未实现) ;
3. **基线增厚**: per-probe normal $n \geq 20$ 、监测温度 $0 \sim 0.2$, 进一步压 FPR;
4. **误报成本实测**: 告警 → 人工审计的成本模型 (30% FPR 在什么场景可接受) 。

第 7 章

第三章 · 医疗诊断：基线报告（第一轮）

日期：2026-08-18 模型：deepseek-chat 病例：10 例合成教科书式虚构病例
定位：LLM 诊断推理失败模式研究（非诊断工具，不构成医疗建议）

一、基线结果（直接单次诊断，结构化输出）

#	类别	确诊标签	LLM 最可能诊断	判定	置信度	紧急处理	红旗征
1	常见病	急性阑尾炎	急性阑尾炎	✓	0.92	✓	✓
2	红旗征	STEMI	STEMI	✓	0.95	✓	✓
3	罕见致命	主动脉夹层	主动脉夹层	✓	0.95	✓	✓
4	易漏鉴别	肺栓塞	急性肺栓塞	✓	0.90	✓	✓
5	红旗征	化脓性脑膜炎	细菌性脑膜炎	✓	0.95	✓	✓
6	鉴别陷阱	急性胰腺炎	急性胰腺炎	✓	0.95	✓	✓
7	不完整鉴别	带状疱疹(前驱期)	带状疱疹(前驱期)	✓	0.75	—	—
8	多系统整合	格雷夫斯病	Graves病	✓	0.95	—	—
9	急诊分级	糖尿病酮症酸中毒	DKA	✓	0.95	✓	✓
10	疼痛定位	输尿管结石	右输尿管结石	✓	0.95	—	—

准确率 10/10，鉴别诊断列表完整（3-5 项），红旗征全部识别，紧急处理全部正确标注。

二、诚实的解读（与第一、二章同构的剧情）

- "LLM 会自信地错"的条件在教科书病例上不成立：deepseek-chat 对干净、完整、教科书式的病例诊断能力极强（10/10）。设计意图想测的失败模式（胸痛锚定误诊心梗、上腹痛锚定胃肠炎、漏肺栓塞、漏 DKA）**全部没有发生**——和数学项目"baseline 20/20"完全同构：**架构/验证的增量空间在基线能全对的难度上不存在。**
- 唯一的不确定性信号**：#7 带状疱疹前驱期（未出疹）置信度 0.75，显著低于其他 0.90+——说明 LLM 对"教科书陷阱题"有校准感（训练数据里有这个经典陷阱）。
- 红旗征/紧急处理/建议检查全部合理**——但这些都是 LLM 单次输出的能力，不是"系统"的增量。

三、价值窗口判定（待第二验证）

窗口 = {LLM 会自信地错} ∩ {可验证}

第一条条件在"教科书完整病例"上**不成立**（10/10）。需要第二验证：

把病例推向 LLM 真会错的地方（模拟真实临床的噪声与模糊）：

- 1. **不典型表现**：老年阑尾炎（症状轻、无转移痛）、孕妇腹痛、不典型心梗（ECG 正常早期）；
- 2. **信息缺失**：只有部分主诉/体征，检验未出——测试 LLM 是诚实说"信息不足需检查"（UNSURE 状态，医疗最珍贵）还是硬猜；
- 3. **竞争性诊断**：症状同时符合两个常见病，需靠检验鉴别；
- 4. **多合并症干扰**：老年患者多种慢性病背景下的新发问题。

四、第二验证：硬病例基线（2026-08-18）

病例设计（10 例，全部虚构）

不典型表现×2（老年阑尾炎、早期 NSTEMI）、信息不足×2（腹痛/胸痛，测诚实 UNSURE）、竞争性诊断×3（甲亢vs焦虑、GERD vs 心绞痛、低血糖vs心梗）、合并症干扰×1（AECOPD vs PE）、误导性主诉×2（腹泻掩盖阑尾炎、胃痛表现的下壁心梗）。

结果

#	类别	确诊标签	LLM 诊断	conf	判定
101	不典型	老年阑尾炎	急性阑尾炎(早期)	0.60	✅（不确定表达合理）
102	不典型	早期 NSTEMI	不稳定型心绞痛	0.65	△ ACS 谱系内，临床可接受
103	信息不足	无法确诊	急性阑尾炎（硬猜）	0.70	❌ 失败：信息不足仍下结论
104	信息不足	无法确诊	稳定型心绞痛（硬猜）	0.70	❌ 失败：信息不足仍下结论
105	竞争	甲亢 vs 焦虑	甲亢	0.75	✅ 合理（鉴别含焦虑，建议查甲功）
106	竞争	GERD vs 心绞痛	GERD	0.75	✅ 合理（鉴别含心源性）
107	合并症	AECOPD	AECOPD	0.85	✅ 合理（鉴别含肺栓塞）
108	竞争	先测血糖	低血糖症	0.75	✅ 急诊顺序正确
109	误导主诉	腹泻型阑尾炎	急性阑尾炎	0.85	✅ 未被腹泻锚定
110	误导主诉	下壁心梗	下壁心梗	0.95	✅ 未被烧心锚定

8/10 可接受；2/10 出现"信息不足仍硬猜"失败（#103、#104）——设计意图成功捕获。

核心发现

- 1. **信息不足是 LLM 的失败区**：两个"无检查结果"的病例，LLM 都自信地给了确定性诊断（阑尾炎 conf 0.7、心绞痛 conf 0.7），而不是诚实说"无法确诊 + 建议检查"。
这是医疗最危险的失败模式——自信地错，也是三态裁判里 UNSURE 状态最该发挥作用的场景（"UNSURE 是医疗最珍贵的状态"）。
- 2. **置信度校准与信息完整性脱钩**：不典型病例置信度下降（0.6-0.75，校准感存在），但信息不足病例仍 0.7 硬猜——LLM 的不确定性表达没有正确关联"信息完整性"。
- 3. **教科书陷阱（#109/#110）没踩**：误导性主诉没有带偏 LLM——深层记忆覆盖了这些经典陷阱。

4. **价值窗口的具体形态浮现**：医疗诊断的价值不在"提高准确率"（教科书难度 10/10），而在"**信息不足时强制诚实**"（UNSURE → **建议检查**）——这是可验证的：结构规则（lab 无结果 → 禁止高置信度确定性结论）就是一个"裁判"。

五、验证器可行性：信息完整性裁判（2026-08-18）

裁判规则（三个结构性检查）：

1. 信息完整性：lab 完全缺失（"暂无检查结果"等）+ 确定性高置信度诊断（ $\text{conf} > 0.6$ ）→ 过度自信 FAIL；
2. 红旗征：急诊红旗（含"紧急/危象/致命"字样）→ 必须 $\text{urgent} = \text{True}$ 且列出红旗；
3. 鉴别完整性：确诊标签应在鉴别/诊断中（含同义词、大小写归一）。

裁判自身的校准历程（对应数学章 simplify 等价处理、行为章操作点修正）：

- 初版 7 个"误报"中：5 个是裁判 bug（同义词"细菌性/化脓性脑膜炎""Graves/格雷夫斯" "慢阻肺/慢性阻塞性肺疾病"、大小写 ST、red_flags 字段"无 (...) "语义、行动指令型确诊格式），1 个是临床注意事项 vs 急诊红旗的字段语义，**1 个是裁判发现了人工漏判的真失败（#105）**；
- 修复后：**只标记 #103/#104/#105，其余 17 例全过。**

修正后的评估：

项 值
--- ---
命中 (含 #105) 3/3 (金标修正为 {103,104,105})
误报 0
精确率 / 召回率 1.00 / 1.00 (n=20, 原型信号, 非验证指标)

关键发现：

1. **裁判抓到了人工漏判**：#105（无检查结果却 $\text{conf} 0.75$ 定诊"甲亢"）与 #103/#104 同一失败模式（信息完全缺失仍确定性下结论）——裁判的价值不只是复核，是**发现**；
2. **"裁判的裁判也需要裁判"第三次上演**：裁判自身的校准（同义词/大小写/字段语义）是数学章等价性处理、行为章操作点修正的医疗翻版——同一个方法学循环；
3. **第三章价值窗口的具体形态**：不是"提高准确率"，而是"**信息完整性裁判**"——一个廉价的结构规则，直接针对医疗最危险的失败模式（结论超出证据），不需要数学章的完整拆解-验证架构，是后置检查。

诚实边界：n=20、3 个失败、规则原型——精确/召回是可行性信号不是验证指标；金标曾被裁判修正（轻微循环性，但 #105 可独立论证为同一失败模式）。
扩大验证需要更多"信息不足"病例。

数据：verify_diag.py（裁判）、cases.json/cases_hard.json（病例）、data_baseline.json/data_baseline_hard.json（基线输出）。

六、L3 抬级：导入 Alvarado 决策规则（2026-08-18）

动机

原"信息完整性"检查是**手写规则** (lab缺失 + conf>0.6 → FAIL) ——锚定层级为 L1/L2。
按 docs/O6 判级标准, 可判定域的属性应尽量抬到 L3: **导入外部已验证的决策规则**,
用规则的可判定边界 + 缺数据语义替代手写阈值。

实现 (medical/alvarado.py)

- **Alvarado 评分 (MANTRELS)** : 1986 年验证的阑尾炎决策规则, 8 项客观输入
(转移痛 1 / 厌食 1 / 恶心 1 / 右下腹压痛 2 / 反跳痛 1 / 发热 1 / WBC>10 得 2 / 中性粒>75% 得 1) ,
满分 10: ≥7 高风险、4-6 中风险、≤3 低风险;
- **确定性解析**: 从病例自由文本 (history/exam/lab) 正则提取 8 项;
- **缺数据语义 (关键增强)** : 输入缺失时返回**可达分数区间 [min,max]**——
规则自己"拒绝完整打分", 而不是手拍阈值;
- **ODD**: 仅应用于疑似阑尾炎病例 (确诊含"阑尾炎"或主诉/病史含"右下腹") 。

结果 (对照原三检查)

病例	Alvarado 区间	缺项	LLM 行为	新判定	说明
#1 教科书	[10,10]	无	确诊阑尾炎	✅ PASS	高风险, 诊断一致
#101 老年	[6,6]	无	阑尾炎(早期) conf 0.6	✅ PASS	中风险, 低置信可接受
#103 信息不足	[3,8]	MALS	阑尾炎 conf 0.7	❌ FAIL (双重标记)	区间不确定→规则拒绝支持确定性诊断
#109 腹泻型	[7,9]	AS	阑尾炎 conf 0.85	✅ PASS	缺中性粒但下界已≥7→高风险可确证

关键语义: 区间而非点值让规则"会说话"——#109 缺中性粒计数, 但下界 7 已过阈值, 高风险**仍可确证** (PASS) ; #103 区间 3-8 横跨风险级, 规则**拒绝**支持确定性诊断 (FAIL) 。
这比手写阈值 (信息缺失一律 FAIL) 更精确: #109 不该被误伤。

裁判自身的校准 ("裁判的裁判"第 N 次上演)

自测抓出两个解析 bug: ① 无呕吐 被 呕吐 关键词误判阳性 (#101 应 6 分算成 7) ;
② 伴恶心、无呕吐 应计 1 分 (恶心阳性) , 否定优先逻辑误判为 0 (#1 应 10 分算成 9) 。
修复后 4/4 自测通过——规则的规则也需要规则。

结论与边界

- **L2→L3 抬级成功演示**: 手写阈值 (L1/L2) → 外部已验证规则 + 可判定边界 + 诚实缺数据语义 (L3, 阑尾炎 ODD 内) ;
- 标记集不变 ({103,104,105}, 3/3) ——抬级不改变结论, 改变的是**锚的来源与强度**;
- **边界**: 解析仍是关键词级 (自由文本→结构化的 L1 提取) ; 单规则单 ODD;
PERC/Wells (肺栓塞/胸痛 ODD) 为后续工作; n=20 仍是可行性信号。

第 8 章

o8 · 第四章：代码生成验证（反向验证设计）

日期：2026-08-18
定位：前三章为归纳（证据→框架），第四章为演绎/反向验证（框架→预测→撞证据）。
铁律：预测先于证据写死（本文档第 1 节即预测清单），事后不得改动；证据矛盾→修订框架（记录修订轨迹）。

o. 为什么反向验证是正确的一步

- 框架的价值 = 对未见领域的**预测力**，不是对已见数据的拟合度；
- 前三章的证据是**自生成数据**（存在自利偏差可能）；第四章用**外部大规模证据**（HumanEval/MBPP/SWE-bench + 已发表文献），难以被我们带偏；
- 若预测全部命中 → 框架获得外部验证；若半数未命中 → 框架需要修订——**两者都是产出**，后者尤其珍贵（暴露框架的 ODD 边界）。

1. VAL 预测清单（2026-08-18 写死，不可事后修改）

#	预测	可证伪判据	验证状态
P1	代码验证的主流是单元测试（L2）→ 完备性盲区表现为“测试通过但边界/需求遗漏”；test-based 选择存在 测试预言机上限	若 test-based 选择无天花板地提升准确率	✅ 支持（来源见 §3）
P2	LLM 自评（Lo）有效用但无保证，跨任务方差大	若自评稳定可靠且无方差	✅ 支持（来源见 §3）
P3	类型检查（L3）是代码域性价比最高的完备锚	若类型信息对生成正确性无增益	🟡 待实验（直接文献证据不足）
P4	信任递归出现在规格层：LLM 写的测试本身可疑（测试预言机问题）	若 LLM 生成测试无错误/无遗漏	✅ 支持（来源见 §3）
P5	代码域是四域中 L3 抬级最容易 的（锚生态最密：编译器/类型器现成）	若代码域抬级 ROI ≤ 医疗域	✅ 支持（来源见 §3）

P5 最锋利：预测代码域 L3 抬级应比医疗（自导 Alvarado）更容易。

2. 可行性扫描：代码生成验证生态的 VAL 判级

方案	锚定来源	保证	VAL	盲区/边界
LLM 自评 ("这代码对吗")	LLM 声明	无	L0	无保证
编译/语法检查	语法规则	确定性	L1	只查语法
单元测试 (执行对比)	客观执行结果	正确性 (测试路径)	L2	未测路径/需求不可见
覆盖率指标	执行路径统计	正确性 (已覆盖路径)	L2	覆盖≠正确≠规格完备
属性测试 (QuickCheck)	随机抽样+性质	正确性 (抽样)	L2	抽样天花板
类型检查	类型系统	单性质完备 (类型)	L3	只保类型
符号执行/约束求解	可判定系统	单性质完备 (路径)	L3	ODD=可求解路径
形式验证 (Dafny/Frama-C)	证明内核	领域完备	L4	成本高
test-based 重排 (ranker)	执行结果	正确性 (选择器)	L2	选择器自身也是 L2

扫描结论：代码域确实拥有四域中最密的 **L3 锚生态**（类型检查器、符号执行器现成可用），与 P5 一致——但"密"不等于"被用进验证回路"，这正是抬级机会所在。

3. 证据对照（2026-08-18 系统收集，附来源）

预测	证据	来源	判定
P1	test-based filtering/reranking 提升选择准确率：AlphaCode 大规模测试过滤；CodeT 生成测试+执行选择；LEVER 执行重排且自述"获取测试用例本身是挑战"→ 选择质量受测试质量（预言机）约束	AlphaCode arXiv:2203.07814；CodeT arXiv:2207.10397；LEVER arXiv:2302.08468	✓
P2	self-repair 有效但非银弹（Olausson 系统研究其局限、表现因任务而异）；无外部反馈时自纠错常失败（Huang）；Self-Refine 任务依赖	arXiv:2306.09896；2310.01798；2303.17651	✓
P3	直接证据不足： 搜到的是类型推断（TIGER 2407.02095 等），而非"类型提示提升生成正确性"的对照实验	待补充	✖
P4	Meta TestGen-LLM 对 LLM 生成测试 必须过验证过滤器 （测试本身不可信→信任递归）；LLM 生成测试的特征研究；异常 oracle 生成质量受输入来源影响	arXiv:2402.09171；2510.25297；2608.00884	✓
P5	执行反馈 agent 显著优于无反馈：Reflexion/CodeAct/Self-Debugging 均以执行反馈为核心——与"L2/L3 锚 > L0 自评"一致	arXiv:2303.11366；2402.01030；2304.05128	✓

步骤 1 结论：4/5 预测获外部证据支持；P3 是开放项——恰好是"类型检查（L3 锚）ROI"的直接检验。

建议用一个小实验闭合：HumanEval 子集上对比 ①裸生成 ②+类型提示 ③+类型检查反馈 三种条件的价值窗口。

（注：证据为摘要级核实，arXiv ID 均经标题/作者确认；P3 的"方向一致"印象来自类型推断文献，非直接证据。）

3.5 P3 实验记录（2026-08-18，已执行）

设计 (chapter4/p3_experiment.py)：三条件 A 裸生成（剥离类型）/ B 带类型 / C 带类型+mypy 反馈（1 轮修复）；
两阶段协议（框架方法论：先测价值窗口，再在窗口内比较）；审计先行（手写难题先自验测试）。

结果：

测试集	A 裸生成	B 带类型	C 类型检查反馈
HumanEval 前 164 题（训练集内）	164/164 = 100%	—（窗口空，未跑）	—
手写难题 6 题（非经典带陷阱）	6/6 = 100%	5/6 = 83%（噪声）	6/6 = 100%

解读（诚实版）：

- 准确率窗口两度为空**——deepseek-chat 对代码生成太强（训练集内 164/164；手写难题 6/6）。"类型信息提升生成正确性"（P3 的准确率推论）在此模型上不可测——没有可提升的空间。这与框架实证模式一致（数学/医疗/代码：LLM 太强→准确率窗口空），是框架第 4 次命中；
- L3 锚入环的定性演示**：唯一的 B 失败（hard/1 转义陷阱）mypy 报 1 错，C 反馈修复后 PASS——类型检查器（L3）确实抓到了测试之外的错误并驱动了修复（n=1，定性信号）；
- P3 框架主张按构造成立**：类型检查是可判定系统（L3 由定义决定，如 solveset），不依赖准确率实验；准确率推论不可测 ≠ 主张错误，而是"此模型无窗口"；
- 方法论副产物**：手写难题审计抓出 3 个测试编写错误（期望值拍脑袋写错）——"裁判的裁判"第 N 次上演；窗口先行的协议再次证明必要。

P3 收尾判定：外部文献（P1/P2/P4/P5 命中）+ 本实验（窗口空，框架一致；L3 入环定性有效）。
若要量化"类型提示提升正确性"，需更弱的模型或模型训练集外的题目（本实验不可得，标注为边界）。

4. 判定规则（什么算第四章成功）

- 全部/多数命中** → 框架预测力获外部验证，论文升级（框架从"事后解释"变"事前预测"）；
- 半数未命中** → 修订框架并记录修订轨迹（docs/o7 的版本化先例：self-correction protocol）；
- 特别关注 P1 边界**：若 test-based selection 大幅提升且无天花板 → 代码域的 L2 锚（执行比数学的 L2 锚（代回）"更强"——可能需修订框架（候选修订：执行反馈是 L2+，介于 L2/L3 之间）。
这是最有意思的分叉：它测的是"L2 锚内部是否还有强弱之分"。

5. 执行计划

- [] 步骤 1（本周）**：系统收集证据——基准数据（HumanEval/MBPP/SWE-bench 报告）+ 文献，逐条 P1–P5 打钩/打叉，全部附来源；
- [] 步骤 2（可选，花 API 钱）**：小实验——HumanEval 子集上对比 test-based selection vs self-review vs type-annotated 三种验证的价值窗口；
- [] 步骤 3**：写第四章报告（本文档升级为完整章节，或并入论文作为第 6 个 case study）。

6. 与前三章的对照（框架不变的部分）

- 方法论模板照搬：审计先行 → 测价值窗口 → 失败分类 → 测裁判 → 诚实负结果；
- 判级标准照搬：docs/o6 三问 + val_standard.py；
- 唯一新增：**预测先行**（反向验证的纪律）。

第三部・VAL 框架

第 9 章

05 · 锚定层级：验证自主等级 (VAL, Verification Autonomy Levels)

三章实验沉淀出的**唯一可迁移结论**——不是"LLM 太强"或"裁判有缺陷"这些具体结果，而是"裁判的价值 = 它的规格锚定在哪一层"这个可复用的判据。

层级划分借鉴 SAE 自动驾驶分级 (Lo–L5)：**核心变量不是"能力"，而是"信任责任归谁"**。

一、问题被问错了

复盘三章所有"裁判没兜住"的失败，摊开后发现**没有一个是裁判判错了**：

失败	裁判在干嘛	真正坏掉的是
拆解层污染	忠实验证了被喂错的公式	问题本身被 LLM 写错
组合层篡改	子任务全对，终局答案在视线之外被改	裁判没看到最终答案
完备性盲区	代回 a=2 判 PASS，完全正确	代回验证在原理上证明不了"没漏"
信任递归	按 LLM 声明的 final_check 判，判对了	规格本身是 LLM 编的

裁判的精度是 42/42（数学）、3/3（医疗）。它是一台校准合格的仪器，只是被指向了错误的对象、或压根没被指向。所以该追责的不是裁判，是规格的锚定来源。

二、验证自主等级 VAL-Lo ~ L5

等级	锚定来源	保证	完备性	驾驶类比	项目例子
L0	LLM 声明	无	无	全手动（人类全责，会"自信地错"）	实验七 final_check 由 LLM 写
L1	题面派生	确定性	无	车道保持（单一功能辅助）	实验八正则提取 final_check
L2	客观真值	正确性	缺	部分自动化（人须随时接管）	代回/采样/基线阈值
L3	定义性/证明性	单性质完备	有（限性质）	有条件自动化（ODD 内系统负全责）	solution_set、类型检查、闭式解、已验证决策规则
L4	领域级证明系统	全域完备	有（限领域）	高度自动化（ODD 内无需接管）	Rust borrow checker、证明助手内核
L5	通用完备验证	任意性质	不可判定	全自动化（无条件全域）——工程上不可达	—

核心变量是"责任边界"，不是"准确率"：

- L0/L1 时，规格的正确性由 LLM 或题面负责——裁判再准也没用，因为它在验证一个可能错误的对象；
- L2 时，裁判能判"对不对"，但"没漏"仍由人负责（代回/采样天然不保证完备）；
- L3 时，"该性质完备"由系统负责——这是唯一一次责任真正从人转移给机器；
- L4 时，责任在整个领域内转移；
- L5 是理想极限，不存在（见第三节）。

三、ODD：可判定域（等级的边界条件）

自动驾驶里，L3/L4 只在 ODD（Operational Design Domain）内成立——出了设计工况，系统退回低等级。

验证的对应物是**可判定域**：性质能被编码进某个可判定系统的领域。

锚	它的 ODD（可判定域）
solution_set	solveset 能封闭求解的单变量代数方程/不等式
answer_type	能归类为 interval/point_set/union 的集合
Rust borrow checker	能用所有权/借用/生命周期表达的内存安全性质
Alvarado 评分	有 WBC/移位/体征的疑似阑尾炎
证明助手	能写成命题、且有证明程序的性质

完备性不是绝对的，是相对 ODD 的。 上一轮"完备性是不是无解"的精确答案是：
在 ODD 内可达完备（L3/L4）；ODD 之外，自动退回 L2 或 L0。抬高等级 = 扩大 ODD，
而不是"更努力地验证"。

L5 为什么不可达：任意性质的完备验证等价于停机问题 / Rice 定理——不存在通用算法判定任意程序的任意性质。所以 L5 不是"还没造出来"，是**原则上不存在**。这给"裁判的裁判也需要裁判"一个收口：信任递归必须有终点，而终点是"某个可判定系统的原始锚"（内核/公理/约定基准），不是无限向上推。

四、前例（同构的成熟实践）

1. **证明助手（Coq/Lean/Isabelle）**：L4 的范本——人写规格，机器检查证明，裁判归约到极小可信内核。完备性是**构造出来的**（一个通过类型检查的证明程序 = 完备证明），内核就是递归的终点。
2. **类型系统（Rust borrow checker）**：L4——证明"无 use-after-free / 数据竞争"是对**所有路径**的完备保证，代价是限制语言表达力。
3. **临床决策规则（Wells/PERC/Alvarado/CURB-65/HEART）**：L3——被验证的、确定性的、锚定在客观输入的规则。PERC 规则字面意思就是"全阴性 → 可安全排除 PE"，一个**完备性方向**的保证。
4. **计量溯源性（NIST 校准链）**：L3 的哲学前例——测量逐级溯源到原始基准（由约定定义），**每一层锚定在更外层**，而不是停在"这台仪器说它对"。

五、可操作的判据

对每一个"要不要建裁判"的决策，先问一句话：

- "这个失败模式，有没有一个 L3 锚，能把'正确性'升级成'完备性'？它的 ODD 是什么？"
- 有，且 ODD 覆盖目标 → 建裁判（导入或重编码，别自造）；

- 有，但 ODD 不覆盖 → 诚实标注"只在 ODD 内完备"，ODD 外退回 L2；

- 没有 → 这个裁判最多是 L1/L2 的"可信度探针"，别指望它闭合，诚实标注为边界。

六、三章当前所处等级

章	裁判	当前等级
一 数学	root/extreme/satisfies/final_parameter_set	L2（正确性，代回/采样）
一 数学	inequality/solution_set/answer_type	L3（解集等价/类型）
一 数学	interval_extreme	L3*（端点+驻点穷举，二次函数域内完备）
二 行为	无向偏差统计检测器	L2（阈值来自基线分布，FPR 30%，无完备性）
三 医疗	信息完整性/红旗/鉴别	L2（客观字段锚定，但规则自造）
三 医疗	若导入 Alvarado/Wells/PERC	L3（已验证决策规则）

七、本次落地（2026-08-18）

1. 新增 solution_set 验证类型（L3）： solveset 解集等价，直接抓到代回验证抓不到的三类错误——漏解（"2" vs {2,3}）、漏负根（"sqrt(3)" vs {±√3}）、压缩篡改（"a=0 或 a=2" vs [0,2]）。裁判测试 34 → 42，全过。
2. 等级进代码： verifier.py 新增 ANCHOR_LEVELS 字典 + anchor_level() 函数，每个验证类型都显式标注自己的 VAL 等级——让"锚定层级"从文档变成可查询的运行期事实。

这印证了判据本身： 同一个"求所有解"任务，换一个 L3 锚，完备性盲区就从"原理无解"变成"可判定"。完备性的边界不是固定不变的，它随锚定层级（ODD）的抬高而收缩。

第 10 章

o6 · VAL 判级标准 (Verification Autonomy Level Assessment)

用途：对任意"验证方案/裁判"给出在金字塔 L0–L5 上的**确定性判级**，以及"该用哪层、要不要爬级、何时停止"的使用指引。

配套可运行工具：val_standard.py (classify() + 10 用例自测)。

o. 适用范围与核心原则

- **适用对象**：任何"判定某事物对/错、真/假、达标/不达标"的方案（LLM 自证、规则、求解器、类型系统、证明系统都算）。
- **核心原则**：判级不看"能力/准确率"，看三个维度——**规格来源、保证类型、覆盖范围**。准确率再高也是同一层内的水平问题；层级是"信任责任归谁"的**性质问题**。
- **判级对象**：判"当前实现"的等级，不是"锚的潜力"——verify_root 的锚其实是可判定的 solveset，但实现只用代回，所以判 L2。

1. 判级三问

问	选项	含义
Q1 规格来源：验证条件谁定？	LLM 声明 / 题面规则 / 客观真值 / 可判定系统	锚从哪来
Q2 保证类型：保证到什么程度？	无 / 正确性 / 完备性	"试过的都对" vs "没有漏"
Q3 覆盖范围：完备性盖多大？	单性质 / 领域 / 声称任意	ODD 宽度 (Q2=完备性时才问)

2. 判级程序 (顺序判断，先中先得)

1. 完备性 + 声称任意性质 (universal)	→ L5 — 否决 (Rice 定理，不可判定)
2. 完备性 + 领域级覆盖 (domain)	→ L4
3. 完备性 + 单类性质 (single)	→ L3
4. 正确性 + 可判定系统或客观真值锚	→ L2 (可判定锚标注"用法降级，可抬级")
5. 正确性 + 题面/代码规则	→ L1
6. 其余 (LLM 声明 / 无确定性锚)	→ L0

3. 判定矩阵 (速查)

保证 \ 规格	LLM 声明	题面规则	客观真值	可判定系统
无	Lo	Lo/L1	Lo/L1	L1
正确性	Lo	L1	L2	L2（可抬级）
完备性(单性质)	—	—	—	L3
完备性(领域)	—	—	—	L4
声称任意	—	—	—	L5 否决

4. 使用准则：该用哪层

两问决定目标等级：

- **代价问：**静默错误的成本多高？（漏一个就致命 vs 无伤大雅）
- **可编码问：**性质有没有 ODD（可判定域）？

代价 \ 可编码	可编码	不可编码
低	顺手到 L3	L1/L2 就够，别浪费
高	必须 L3/L4	L2 封顶 + 人工复核，标注风险

5. 停止条件（何时不爬）

1. **已到 ODD 边界**（性质不可再编码）→ 停；
2. **边际收益 < 边际成本**：错误成本 $\times$ 残余错误率 < 建锚成本 → 停；
3. **L5 不可判定** → 永远不追。

6. 判级检查清单 (新 AI / 新项目可复用)

- Q1 规格来源: LLM声明 / 题面规则 / 客观真值 / 可判定系统?
- Q2 保证: 无 / 正确性 / 完备性?
- Q3 范围(若完备): 单性质 / 领域 / 声称任意?
- 套判级程序 → 落定 L0~L4 (L5 否决)
- 使用检查: 错误成本高? 性质可编码? → 目标等级合理?
- 停止检查: 到 ODD 边界? 边际成本超收益?

7. 项目实测（三章裁判按本标准判级）

裁判	Q1	Q2	Q3	等级	说明
LLM"经验证"文本	LLM声明	无	—	L0	伪验证实锤（P4）
题面正则 final_check	题面规则	正确性	—	L1	实验八
verify_root 代回	客观真值	正确性	—	L2	完备性盲区
final_parameter_set 采样	客观真值	正确性	—	L2	采样天花板
医疗信息完整性裁判	客观真值	正确性	—	L2	自造规则未验证
行为统计检测器	客观真值	正确性	—	L2	攻击性质不可编码
answer_type	可判定系统	完备性	单性质	L3	类型系统
（未来）Alvarado 等决策规则	可判定系统	完备性	单性质	L3	导入已验证规则

审计修正（2026-08-20，依据 docs/11 可复现性研究第一/二轮，版本化追加不改写）：

上表"医疗信息完整性裁判 L2"与"行为统计检测器 L2"经盲评审计（Rater B/B2，协议 v2.1 R10）复核：

- **医疗信息完整性裁判** → **应判 L1/L2（更准确为 L1）**——它是设计者自造、未经外部验证的确定性规则

（R10：自造未验证规则= L1，即使输入是客观 lab 数据）。论文 §5.3 第 281 行已自述 "hand-written threshold (L1/L2)"，与此一致。

- **行为统计检测器** → **L1/L2 边界**——阈值规则为设计者自造（R10 指向 L1），但统计基线为实测分布

（客观锚指向 L2）。待第三轮协议（判定路径约定）裁决。

- 反例确认：verify_root 等代回验证维持 **L2**（R9：题面方程=答题前已存在的客观真值）。

修正轨迹见 reliability/ 评分档案。

8. 边界与诚实声明

- 1. 判级是对"当前实现"的判定，不是"锚的潜力"——实现只用代回就判 L2，哪怕锚能到 L3；
- 2. Q1/Q2 是二元抽象，真实裁判常是混合的（如 equality = simplify + 抽样兜底）——**取实际保证中较弱的那档**；
- 3. L3 的"完备性"是 ODD 内相对完备，不是绝对完备；L4 的"领域"必须先定义领域边界；
- 4. 本标准的判级程序是确定性的（顺序判断），任何人/任何 AI 对同一输入应得到同一等级——这是它作为"标准"的最低要求。

07 · 文献评述：17 篇"分层验证"论文 × VAL 判级标准

方法说明：对每篇论文按 VAL 判级标准 (docs/o6) 判级——判级只看锚定来源 (Q1) , 不看论文自称的"分几层"。判级依据 = 论文摘要 + 既有知识；5 条低置信项已抓取原文摘要补实 (见文末补实记录) 。

版本：V1.3 (2026-08-18) ——V1.0=17 篇初评；V1.1=补实 5 条；V1.2=17/17 全核实；V1.3=6 篇锚定轴论文全文精读 (机制细节坐实，判级全部维持) 。

最后更新：2026-08-18。

一、验证机制分层（核心相关）

#	标题	作者/年份	声称层级	实际层级 (VAL)	地面真值	完备性盲区	跨域	关系
1	VerifiAgent	Han/Buntine/Shareghi, 2025	两级验证 (元验证+工具自适应)	L1-L3 混合 , 按最弱锚定取 Lo/L1 (元验证是 LLM 判断)	符号工具 (SymPy 等) 局部 L2/L3	模糊 (声称查"完整性"但检查者是 LLM=信任递归)	是 (数学/逻辑/常识)	补充+支持
2	Graph of Verification	Fang 等, 2025	DAG 拓扑序分层验证	Lo/L1 ——分层是"粒度"不是"锚定"; 节点验证器若为 LLM 判 →Lo	无外部真值	否 (不检查推理图漏分支)	否/模糊	补充
3	DiVERSE	Li 等, ACL 2023	逐步验证 (step-level)	L2 ——训练出的统计验证器, 锚在人工标注步骤标签	人工步骤级标注	否 (只验给定步骤)	否	支持
4	Safe	Liu 等, ACL 2025	Lean 4 形式化逐步行验证 (自称最强)	L4 内核 + Lo 陈述层 ——证明检查是形式内核, 但定理陈述由 LLM 声明	Lean 内核 (形式系统)	模糊 (证明对"已陈述定理"完备, 但陈述/分情况漏没漏在 LLM 层)	否	支持+盲区
5	SelfCheck	Miao 等, ICLR 2024	零样本自验证	Lo ——纯 LLM 自检, 多采样仍是统计自评	无	否	否	支持
6	LM²	Juneja 等, EMNLP 2024	分解器-求解器-验证器三模块	Lo ——验证器是"模型社会"里另一个 LLM	无	否	否	支持

二、验证粒度分层

#	标题	作者/年份	声称层级	实际层级(VAL)	地面真值	完备性盲区	跨域	关系
7	Factcheck-GPT	Wang 等, 2023	三级粒度 (claim→sentence→document)	L1/L2 ——证据外部文档 (L2) , 声明-证据对齐判定是 LLM (Lo)	检索来源文档	否 (不覆盖未检查的声明)	模糊/是	补充
8	Hierarchical Attention Proofs	Chen 等, ACL 2025	五级概念层次 (基础元素→高级概念)	L3/L4 ——正式定理证明 (miniF2F/ProofNet) , 证明由形式系统内核判定; 论文本身是生成正则化方法, 五级层次=概念抽象轴 (与锚定正交)	miniF2F/ProofNet + 形式内核 (Lean) → L4	否 (聚焦生成成功率, 未讨论)	否	补充 (概念层级=第三正交轴; 内核判证明支持 L4 存在)
9	Dr. V	Luo 等, 2025	感知→时间→认知三层	L2 ——感知/时间层用空间-时间 金标标注 做 grounding (客观锚) ; 认知层为模型推理 (Lo/L1) ; 三层=作用域轴	Dr.V-Bench 的 10k 空间-时间金标 (4,974 视频) → L2	否	否	补充 +支持 (金标锚定 =L2, 符合框架)

三、安全/风险等级分层

#	标题	作者/年份	声称层级	实际层级 (VAL)	地面真值	完备性盲区	跨域	关系
10	Proprietary Safety Response	Li 等, 2025	四级风险分类 (Safe/Unsafe/Conditionally Safe/Focused Attention)	L0/L1 ——输入分类=监督微调统计分类器 (概率性判定) ; 输出层 RAG 知识库锚定但解释模型仍为模型判定	安全类别训练标注 + RAG 实时知识库 → L1/L2	否 (99.3% 召回是经验数字, 非完备保证)	否/模糊	补充 + 支持 (统计分类器无完备性)
11	M3-SafetyBench	杨伟平等, 2025	内容安全×功能安全双层	L2 (作为评测基准=金标集, 非运行时验证器)	专家/人工标注	模糊 (基准覆盖边界)	是	补充
12	LLM Output Drift	Khatchadourian/Franco, 2025	按输出确定性三级 (风险适配部署)	L1/L2 ——确定性=统计一致性 (T=0.0/固定种子) ; 另有 invariant checking (±5% 物质量阈值) + SEC 引文验证=客观真值锚 L2	SEC 10-K 文档 (外部权威源) + 确定性阈值规则 → L2	否 (只验给定输出的不变量, 无完备性声明)	否 (金融)	补充 + 支持 (一致性 ≠ 真值; 但他们补了 L2 锚)

四、系统层级分层

#	标题	作者/年份	声称层级	实际层级(VAL)	地面真值	完备性盲区	跨域	关系
13	Standard Benchmarks Fail	Chen 等, 2025	模型→ workflow→系统三级压力测试	L1/L2 (审计框架, 期望行为人工定义)	人工定义期望	是 (全文论点=标准基准覆盖不完备)	否 (金融)	支持
14	Prompting Frameworks Survey	Liu 等, ACM CSUR	数据→基础→执行→服务四级	非验证 (prompting 框架生命周期分类学, 四级=系统栈架构轴)	不适用	否 (未发现对验证/评测覆盖完备性的讨论)	是	补充

五、综述与理论框架

#	标题	作者/年份	声称层级	实际层级(VAL)	地面真值	完备性盲区	跨域	关系
15	Trustworthy LLMs	Liu 等, 2023	7 类 29 子类评估维度	非验证 (维度分类学; 内部多为 L2 金标评测)	各维度人工标注	模糊 (维度覆盖为何是 7 类)	是	补充
16	HEX	González/Nori, 2023	数学框架形式化幻觉/自验证等	理论形式化——形式上是 L3 样式, 对象是 Lo 现象 (自验证=模型自评)	不适用 (理论)	模糊	模糊	补充
17	FACTOOL	Anonymous, ICLR 2024	工具增强多任务事实检测	L1/L2——工具提供外部证据 (L2), 对齐判定是 LLM (Lo/L1)	外部工具/检索证据	模糊/否	是	支持/补充

横切结论

1. "层级"这个词被至少 5 种不同的东西占用。粒度层级 (GoV/Factcheck-GPT/Dr.V)、概念层级 (#8)、风险/处置层级 (#10/#12)、系统栈层级 (#13/#14)、维度层级 (#11/#15) ——它们回答"验证什么/多细/在哪层/给什么处置", 只有 VAL 回答"凭什么判定"。大部分"分层验证"文献看着相关, 实则与锚定正交。
2. 真正碰到锚定维度的 6 篇 (#1/#3/#4/#5/#6/#17), 全部支持框架, 无一矛盾。DiVERSE 是教科书式 L2 统计裁判 (完备性失明); SelfCheck/LM² 是 Lo 自证; Safe 是 L4 内核 + Lo 陈述层 (信任递归被推到定理陈述层); VerifiAgent 的工具=外锚、元验证=LLM 查完整性 (把完备性交给 Lo) 。
3. 完备性盲区几乎没人正面处理——这是框架最稀缺的位置。17 篇里只有 #13 明确讨论 (评测覆盖); #1/#4 是"声称做了完整性检查"但检查者本身是 LLM 或陈述层。没有任何一篇把"验证方案自身的完备性"当成研究对象。

4. 地面真值的谱系恰好沿着 VAL 的 L2→L4 排开：人工标签 (DiVERSE/基准, L2) → 外部文档/工具 (Factcheck-GPT/FACTOOL, L2) → 符号工具 (VerifiAgent, L2/L3) → 形式内核 (Safe/Lean, L4)。没有人在 L2 和 L4 之间给出过显式的"锚定层级"阶梯。
5. 本项目的空位 = "验证的验证" (对验证方案判级)。这 17 篇全部在"当裁判", 没有一篇在"给裁判分级"。VAL 标准是"验证的计量学"——它评估验证器本身, 而不是当一个验证器。一句话: 别人给答案分级, 本项目给"分级"分级。

补实记录表 (版本化判级追踪)

用法: 每篇论文的判级随证据积累向前追加版本, 不改写历史 (审计可追溯)。
Vo = 初评 (依摘要+知识推断) ; V1 = 2026-08-18 抓取原文摘要核实; 未来修正追加 V2、V3.....

A. 判级版本追踪 (17 篇全量)

#	论文 (简称)	Vo 判级	V1 判级 (核实后)	变化	V1 置信度
1	VerifiAgent	Lo/L1 (混合)	Lo/L1 (元) + L1 (工具, 无金标)	作者亲口确认 (2026-08-19 邮件) : 元验证=LLM 为主+部分确定性规则; 工具验证=确定性执行、无金标/ground-truth, 故非 L2 而是 L1——推翻我们原判"工具=L2/L3"	高
2	Graph of Verification	Lo/L1	Lo/L1	确认粒度轴正交; 节点锚定机制摘要未明	中-高
3	DiVERSE	L2	L2	确认: 训练出的统计逐步验证器	高
4	Safe	L4+Lo 陈述层	L4+Lo 陈述层	确认 Lean 4 形式化	高
5	SelfCheck	Lo	Lo	确认: 无外部资源自评	高
6	LM ²	Lo	Lo	确认: policy learning 训练的 LLM 验证器	高
7	Factcheck-GPT	L1/L2	Lo/L1	修正 : 检查器=GPT-4 标注方案; 基准金标为 L2	高
8	Hierarchical Attention	L1/L2 或 L3/L4	L3/L4	定为内核验证明	高
9	Dr. V	Lo	L2	升级: 金标锚定	高
10	Proprietary Safety	Lo	Lo/L1	确认 (附机制)	高
11	M ³ -SafetyBench	L2	L2	确认: 17 万+ 条金标测试集基准	高
12	LLM Output Drift	L1/L2	L1/L2	修正: 补 L2 锚 (SEC 引文)	高
13	Standard Benchmarks Fail	L1/L2	L1/L2	确认: 审计议程+基准覆盖批判	高
14	Prompting Survey	非验证	非验证	确认	中-高
15	Trustworthy LLMs	非验证	非验证	确认: 7 类分类学	高
16	HEX	理论形式化	理论形式化	确认: 概念数学框架	高
17	FACTOOL	L1/L2	L1/L2	确认: 工具增强	高

B. 版本变更日志 (未来修正在此追加, 不改写 A 表历史行)

版本	日期	#	变更	依据
V1.1	2026-08-18	8	低置信→高; 判 L3/L4	原文摘要: miniF2F/ProofNet 形式定理证明
V1.1	2026-08-18	9	低置信→高; L0→L2	原文摘要: Dr.V-Bench 10k 空间-时间金标 grounding
V1.1	2026-08-18	10	低置信→高; 确认 L0/L1	原文摘要: 监督微调统计分类器
V1.1	2026-08-18	12	低置信→高; 修正原判 (补 L2 锚)	原文摘要: invariant checking + SEC 10-K 引文验证
V1.1	2026-08-18	14	低置信→中-高; 确认非验证	原文摘要: 四级生命周期分类学
V1.2	2026-08-18	1	确认 L0/L1	摘要: meta-verification 查完整性 +consistency; 工具自适应
V1.2	2026-08-18	2	确认 L0/L1 (粒度轴正交)	摘要: node block 粒度自适应 (原子→整段)
V1.2	2026-08-18	3	确认 L2	标题/作者确认 Step-Aware Verifier (训练式逐步验证器, 既有知识)
V1.2	2026-08-18	4	确认 L4+L0 陈述层	摘要: Lean 4 形式语言表达数学声明
V1.2	2026-08-18	5	确认 L0	摘要: "without resorting to external resources"
V1.2	2026-08-18	6	确认 L0	摘要: 验证器模块经 policy learning 训练
V1.2	2026-08-18	7	修正 : L1/L2→L0/L1	摘要: best F1=0.63 by annotation solution based on GPT-4
V1.2	2026-08-18	11	确认 L2	PDF 全文: 17 万+ 条测试集、19 模型、内容×功能双层
V1.2	2026-08-18	13	确认 L1/L2	摘要: 三级压力测试议程+基准覆盖批判
V1.2	2026-08-18	15	确认非验证	摘要: 7 大类可信度分类学
V1.2	2026-08-18	16	确认理论形式化	摘要: 形式化幻觉/对齐/自验证概念
V1.2	2026-08-18	17	确认 L1/L2	标题: Tool Augmented Framework; 4 任务实验
V1.3	2026-08-18	1	全文确认 L0/L1	全文 §3.1/§3.2: 元验证=LLM 代理; 工具=解释器/搜索引擎/符号计算
V1.3	2026-08-18	3	全文确认 L2	全文 §2: DEBERTA-V3-LARGE 逐步验证器; 标签=与 ground-truth 匹配推导
V1.3	2026-08-18	4	全文确认 L4+L0 陈述层	全文 §3.2: LLM 自动形式化 + Lean REPL 内核; "focuses on the correctness of each step"

版本	日期	#	变更	依据
V1.3	2026-08-18	5	全文确认 Lo	全文 §3.1/§4.2：四阶段再生成-对比；自检准确率 66.7%（检查器会错）
V1.3	2026-08-18	6	全文确认 Lo	全文 §3：验证器=LoRA 微调 LLaMA-2；训练标签=GPT-4 标注
V1.3	2026-08-18	17	全文确认 L1/L2	全文 §1：工具收集证据（L2），判定=LLM 推理（Lo）
V1.4	2026-08-19	1	作者亲口修正 ：VerifiAgent 工具组件从"L2/L3"降为"L1"（确定性执行、无金标）；元验证确认为 Lo/L1（LLM 为主+部分规则）；作者提出 " silver ground truth " 术语——工具执行结果是弱于金标的参照，已采纳进论文 §3.2	Jiuzhou Han 邮件："The tool execution results serve as a form of silver ground truth to verify the original results"; "neither component involves ground-truth annotations or gold-standard labels"

17 篇全部核实后横切结论：全部 17 篇落在四类——**Lo 自证**（#5/#6）、**Lo/L1–L2 统计/客观锚定**（#1/#2/#3/#7/#9/#10/#11/#12/#13/#17）、**L3/L4 形式内核**（仅 #4/#8）、**非验证分类学**（#14/#15/#16）。

没有一篇给出"锚定层级"显式阶梯，也没有一篇把"验证方案自身的完备性"当研究对象——本项目空位依旧。

全文精读确认 (V1.3, 6 篇锚定轴论文)

#	机制确认	关键原文
1 VerifiAgent	元验证=LLM 代理查"完整性+一致性" (Lo) ; 工具验证=解释器/搜索引擎/符号计算, 工具先求解再与原解比对 (L2/L3 局部)	"the agent will first list all the known conditions... then divide the solution into individual and explicit logical steps"; "external tools, such as Python program interpreter, search engine and symbolic..."
3 DiVERSE	逐步验证器=DEBERTA-V3-LARGE 训练模型 (BCE 辅助损失) ; 步骤标签=与 ground-truth 答案匹配推导; 只验给定步骤	"f'(input _{i,j}) is predicted from the hidden state of the last token of S _{i,j} in DEBERTA-V3-LARGE"; "regard the reasoning paths that match the ground truth final answer as positive"
4 Safe	形式语句由 LLM 自动生成 (Lo 陈述层) ; Lean REPL 内核验证 (L4) ; 四态输出含弃权; 明言只验正确性	"utilize an LLM to automatically generate formal statements"; "validated using the Lean REPL"; " our formal verifier focuses on the correctness of each step "
5 SelfCheck	四阶段 (目标提取→信息收集→步骤再生成→结果对比) 纯 LLM 零样本; 输出三态(+1/0/-1)合成置信分数 w 加权投票; 检查器自身被 ROC 评测且承认会错	"decomposes the checking task for each step into four stages"; "the checks are themselves imperfect"; 验证准确率 66.7%
6 LM ²	验证器=LoRA 微调 LLaMA-2 (LLM 判官, Lo) ; 训练标签=GPT-4 标注 (LLM 标注, 非人工真值) ; policy learning 协调	"a verifier LM provides feedback on the correctness"; "datasets annotated by GPT-4"; "trained to coordinate using policy learning"
17 FACTOOL	工具=Google Search/Scholar/代码解释器 收集证据 (L2 锚) ; 判定=LLM 基于证据推理 (Lo/L1)	"tools, including Google Search, Google Scholar, code interpreters"; "employs the reasoning abilities of LLMs to assess the factuality... given the evidence"

判级影响: 6 篇**全部维持原判** (#1 Lo/L1、#3 L2、#4 L4+Lo 陈述层、#5 Lo、#6 Lo、#17 L1/L2) , 无一需修正; 置信度升到"全文级"。

对框架论文最有价值的引文: #4 Safe 明言"only verify correctness of each step"——文献里对"完备性盲区"最接近的一次正面承认, 可作为框架核心论点的直接对照。

发表级边界 (诚实声明, 2026-08-18)

本评述可作为**概念框架论文的支撑材料**, 但距发表级还有明确差距, 逐条列出 (不美化) :

- 摘要级 vs 全文级:** 6 篇锚定轴论文 (#1/#3/#4/#5/#6/#17) 已**全文精读** (见上文 V1.3 表) , 判级全部坐实; 其余 11 篇仍为摘要级。若严格发表, 建议再全文确认 #2 GoV 节点验证器机制与 #7 Factcheck-GPT 证据检索流程 (判级影响小, 属锦上添花) 。
- 判级是判断不是测量:** Q1/Q2/Q3 是二元抽象, 部分判级存在合理分歧 (如"RAG 知识库算不算 L2 锚"涉及 #7/#10/#12/#17 的 L1 vs L2 边界) 。发表时应: ① 每个判级附依据引文; ② 对边界案例给出显式判定规则, 避免审稿人抓住内部不一致。
- 框架是概念贡献, 不是经验贡献:** VAL 的价值在"给'分层'分层"——澄清粒度/概念/风险/系统栈/锚定五个被混用的轴。适合概念/立场论文 (先 arXiv 预印本) , 实证支撑 = 本项目三章数据 (数学 42/42 裁判、行为五轮迭代、医疗 3/3) 。

4. **预计的社区争议点**（提前准备回应）：

- "自一致性/自验证明明有用"——回应：有用 $\neq$ 有完备性保证；VAL 说的是保证层级，不是效用。
- "采样难道不算验证吗"——回应：是 L2 正确性探针，非 L3 完备性；实验八采样天花板是实证。
- "L5 不可判定"——回应：严格表述限定在"程序性质的通用判定"（Rice 定理），避免过度宣称。

5. **发表前最小清单**：① ~~6 篇锚定轴论文全文精读（#1/#3/#4/#5/#6/#17）~~ **已完成**

(V1.3)；② 画"金字塔 + 正交轴"图；③ docs/o6 判级程序作附录；④ val_standard.py 作可复现工具引用。

第 12 章

09 · 抬级器 (Level-Raiser) 可行性：把"抬级"从处方变成工具

日期：2026-08-18

定位：docs/o6 判级标准的处方 ("性质可编码 → 抬级到 L3") 目前是文档；
本文档 + val_raise.py 把它变成**可运行工具**——框架从"诊断性"走向"生成性"的第一步。
这补上了框架最大的结构性短板 (诊断性 ≠ 生成性) 。

0. 一句话

给任何 L2 (或 L0/L1) 验证方案，抬级器输出：当前等级 → 能否抬级 → 适用哪个重编码模式 → 具体处方；不能抬级时诚实说"不能"。

1. 六个重编码模式 (L2→L3 的通用动作)

模式	触发条件	动作	成功判据	失败模式	我们的实例
A 解集等价	方法=代回 + 性质 = "求所有解/验证解集"	代回 → 求解完整解集并比较集合	能发现漏掉的候选	求解器解不了 → 退回代回	solution_set (抓漏解 a=0)
B 穷举/闭式	方法=采样 + 候选空间有限	抽样 → 枚举端点/驻点/约束求解	全覆盖无遗漏	空间无限 → 采样天花板	interval_extreme (端点+驻点)
C 导入已验证锚	方法=手写规则 + 领域有已验证规则	手写阈值 → 外部验证决策规则	决策边界可判定 + 缺数据语义	领域无已验证规则 → 需外部验证	Alvarado (医疗 L3 抬级)
D 确定性检查	方法=LLM 判定 + 性质机械可检查	LLM 判 → 类型系统/schema/编译	判定确定可复现	性质不可机械化 → 维持 L0	类型检查入环 (第四章 P3)
E 用法抬级	可判定系统已存在但只用正确性	只用代回 → 用完备能力	同 A, 零新依赖	使用者不换 API	solveset 只代回 → 解集等价
F 确定性特征替代	方法=统计 + 性质可编码为特征	统计检测 → 确定性规则/特征	判定不再依赖阈值运气	性质不可编码 (隐匿执行)	(待做)

2. 抬级器决策程序 (val_raise.py 已实现)

输入: (性质可编码? 当前方法? 领域是否有已验证锚?)

1. 性质不可编码

→ 无法抬级 (原理性边界, 诚实标注)

2. 可判定性未知

→ 不能自动判, 要求人工确认 (诚实)

3. 可编码:

代回 → 模式 A (解集等价)

采样 → 模式 B (穷举/闭式; 空间无限→部分抬级+标注)

手写规则 → 模式 C (有验证锚则导入; 无→L3* 需外部验证)

LLM 判定 → 模式 D (机械可检查则确定化)

统计 → 模式 F (有可判定特征则替代)

可判定但只用正确性 → 模式 E (用法抬级)

3. 诚实边界 (工具必须会说的"不")

1. 性质不可编码 → 原理性边界: 如"输出层无痕迹的隐匿执行攻击"——任何数据都抬不动, 工具必须返回"无法抬级"而不是硬给处方;
2. 可判定性未知 → 要求人工: 工具不猜; 它返回"需人工确认: 该性质能用闭式解/穷举/类型/决策规则表达吗? ";
3. 手写规则无验证锚 → L3*: 规则可判定但未验证, 抬级到 L3 需要外部验证背书——工具标注"L3* (需验证)"而不是冒充 L3。

4. 从原型到产品的路线图

阶段	内容	状态
P0	val_raise.py 决策程序 + 10 自测	✅ 本文档同步
P1	接入 verifier.py 的 ANCHOR_LEVELS: 每个验证类型自动给出可抬级建议	✅ 完成 (RAISE_SUGGESTIONS + audit_verifiers()); python verifier.py 出总表)
P2	自动检测"可判定域": 对给定性质, 扫描可用求解器/决策规则/类型系统	✅ 完成 (ANCHOR_REGISTRY + scan_anchor(): 方程/不等式/类型/临床规则等; 不可编码性质返回空列表=诚实)
P3	半自动重编码: 给出新验证代码骨架 + 人工确认环 (judge 的 judge)	✅ 完成 (SKELETONS + prescribe(): 六模式各配代码骨架 + 人工确认清单)
P4	产品化: 嵌入 Agent 评测流程, 评测指标自动报告"此指标是 L2, 可抬级为..."	✅ 原型完成 (audit_report(): Markdown 审计报告, 自动标风险; 示例见 docs/验证栈审计报告.md)

P0–P4 全部落地: 判级器 (val_standard.py, 你在哪一级) × 抬级器 (val_raise.py, 能不能上去/怎么上去/上不去为什么)

× 审计报告 (audit_report(), 你的验证栈整体体检表) ——框架从"分类学"到"工程工具"的闭环完成。

P5 (压力测试暴露的缺口, 2026-08-18) : 工具吃**结构化参数**, 自由文本→参数的映射 ("追问"本身)

目前是人工/AI 中介的——stress_test.py 里 5 道对抗题 (多智能体投票/SymPy 用法/主观性质/混合

栈/区块链 buzzword)

全部答对，但"追问"是我写的分析，不是工具自动问的。

P5 = 追问层 (interrogator) :  原型完成 (val_interrogate.py, 5/5 自测)。

- 追问用**确定性模板** (不是 LLM 生成的问题——否则追问层自己就是 Lo, 自我指涉已写入设计纪律) ;

- **buzzword 不豁免**: 区块链/共识/多智能体/分布式/投票 → 强制追问"聚合对象是 LLM 判断还是确定性锚输出";

- 答"不清楚" → 不猜, 要求澄清 (工具会施加压力, 但不会编答案) ;

- 输出带"声称是 Lo 输入, 工具不验证其真实性"的诚实脚注。

- 交互模式: python val_interrogate.py; 自测: python val_interrogate.py --auto。

5. 与求职/论文的关系

- **求职弹药**: "我不只给验证分级, 还能自动抬级"——这是从"分析能力"到"工程交付能力"的跨越;
- **论文素材**: 抬级器是"从诊断到生成"的实证——框架的处方被工具化, 且自带诚实边界 (会说不) ;
- **对第四章**: 模式 F (统计→确定性特征) 正是行为章"隐匿执行边界"的另一面: **能编码的性质才抬得动**。

6. 一句收口

判级器回答"你在哪一级", 抬级器回答"你能不能上去、怎么上去、上不去为什么"。

前者是诊断, 后者是处方——**框架从"分类学"变成"工程工具"的这一步, 就是抬级器**。

第 13 章

10 · 对抗性评审答辩（十二刀 + 终极题）

日期：2026-08-18

性质：对 VAL 框架的外部压力测试的完整回应。逐刀标注裁决：✅ 承认 / 🛡️ 防守 / ✎ 论文需修正。

原则：不辩护过当。承认的必须承认，防守的给出理由，修正的直接写进论文改动清单。

刀 1 (Q1)：anchor axis 只是"specification source"的重新包装？新东西到底是什么？

裁决：✎ 部分修正（表述），🛡️ 核心防守成立

- **承认**：经典验证理论（Hoare 逻辑、抽象解释、类型论）早已区分 specification / soundness / completeness / decidability / procedure。“锚定来源”在经典语境里就是“规格从哪来”。
- **防守（真正的新东西）**：经典验证在构造上假设规格是外生的——由可信的人类编写，与待验证对象无关，所以“谁写了规格”这个问题根本不会出现。在 LLM 验证里，这个假设破裂了：**规格可以由被测对象自己产生**（Lo：模型声明自己的验证条件）。这是经典理论**结构性排除**的失败模式——不是“重新命名”，是经典理论里不存在的一个轴。
- **修正（论文表述）**：必须写明“anchor axis 问的是一个经典理论不需要问的问题：规格是否独立于被测对象？”——而不是笼统说“规格来源”。

刀 2 (Q2)：Lo→L4 是单调阶梯吗？等级 ≠ 可靠性，为什么叫“level”？

裁决：✅ 承认命名风险，🛡️ 防御“责任转移”语义

- **承认**：L3（错误规格+正确证明器）可以比 L2（人类专家 0.001% 错误率）危险得多。**等级不排序可靠性、安全性、性能**——论文已承认（VAL is not a reliability measure），但命名确实暗示了不存在的 superiority ordering。
- **防守**：借用 SAE 自动驾驶分级——L3 不意味着“更安全”，意味着“在 ODD 内责任从人转移到系统”。VAL 的等级排序的是**判决承诺的强度**（正确性→完备性），不是正确概率。“level”保留这个责任转移的语义；改成“anchor type”会丢掉它。
- **修正（论文）**：加显式警告框：“level 排序的是验证机制承诺什么，不是它多可能对；L3 配错规格可能比 L2 更危险。”

刀 3 (Q3)：L2 的“objective”由谁认证？LLM B 给 LLM A 当 oracle 是不是 L2？递归怎么终止？

裁决：✎ 必须修正——这是当前表述最危险的一刀

- **承认**：“外部于被测模型”≠“客观”。如果 GPT-5.6 提供 ground truth，按现定义它“外部于被测模型”→ 被误判为 L2，**但它是 L0 披着 L2 的皮**。
- **修正 (L2 定义)**：L2 的锚必须满足两条：① 外部于被测模型；② **锚本身不是被测生成器类的成员**。执行结果（程序跑没跑、断言成不成立）是 L2，因为它是**定义性的**（不依赖任何 LLM 的声称）；另一个 LLM 的答案不是 L2，因为锚本身仍是 LLM 声称。
- **递归终止**：正好在框架说的位置终止——定义性/可判定锚（执行、符号化简、类型规则，L2/L3/L4）或约定基准（计量学）。**如果 ground truth 是另一个 LLM，递归没有终止——它是 L0 伪装成 L2**。框架必须把这个判例写死。

刀 4 (Q4)：L2→L3 是 verifier 升级还是问题重写？

裁决：✅ 承认，🟡 防守“重编码就是验证的基本动作”

- **承认**：如果问题能重编码成可判定性质，它**一开始就有** L3 表述。L2→L3 是表述方式的升级，不是 verifier 内部机制的升级。
- **防守：重编码正是验证的基本动作**——Hoare 逻辑把“程序对吗”重编码进证明系统，类型检查把“安全吗”重编码进类型语言。VAL 的要点不是“verifier 变强了”，是**“同一个问题，系统选择了 L2 表述 (usage-degraded) 而非它本可用的 L3 表述”**——框架让这个被忽略的选择变得可见。这正是“用法抬级”（模式 E）的概念。

刀 5 (Q5)：ODD 会不会成为“万能免责条款”？谁审计 ODD？

裁决：✅ 承认，🔧 必须修正——最可被利用的点

- **承认**：任何系统都能声称“我的 ODD 很小”，然后在微不足道的 ODD 内宣布完备——这是空洞的完备。论文 limitation 4 已承认“框架要求诚实，不保证诚实”，但这不够。
- **修正 (ODD 锚定机制)**：ODD 必须**锚定在工具的文档化能力域上**（“solveset 的 ODD = SymPy 能闭式求解的方程类”），而不是应用方声称的“我恰好能解决的东西”。并加**ODD 诚实探针**：对声称的 ODD 边界做对抗性测试（边界外一个反例是否击穿完备性）。
- **承认**：ODD 本身是声明 → 又回到 statement-layer blind spot——**这正是框架主题的又一次上演**，论文必须主动说出这一点而不是被审稿人指出。

刀 6 (Q6)：completeness 被过度绑定在 candidate enumeration 上？

裁决：✅ 承认表述过窄，🟡 给出统一泛化

- **承认**： $\forall x P(x) \Rightarrow x \in C$ 是“求所有解”任务的形式；memory-safety、guideline-compliance 没有 candidate set。现表述是数学任务的特例。
- **泛化 (统一定义)**：completeness = **语义完备性**（逻辑/证明论里的经典概念）：“ODD 内每个为真的实例都被验证器认证”。实例域随任务变化：解候选、程序路径、测试用例、证明义务、指南条目。“candidate enumeration”只是实例域的一种。
- **修正 (论文)**：把 L3 完备性定义从 candidate-set 形式升级为语义完备性的一般形式，candidate 作为特例。

刀 7 (Q7) : Rice 定理用得也太猛? 受限域下 L5 未必不可能?

裁决:  承认需限定,  框架本无矛盾

- **承认**: Rice 适用于"任意程序的非凡语义性质"; LLM 验证若限定在有限域/decidable language/typed DSL, 完备性可能成立 (这正是 L3/L4 的定义)。说"完备性普遍不可能"是错的。
- **修正 (表述)**: L5 不可能 = 无限制性质类的通用完备验证不可能 (Rice/停机); L3/L4 = 可判定片段内的完备性, 可达成。两者不矛盾——框架本来就只说 ODD 内完备。论文必须防读者得出"完备性整体不可能"的错误结论。

刀 8 (Q8) : interrogator 自己的 template coverage 是不是完备性盲区?

裁决:  完全承认——框架主题咬到自己

- **承认**: 模板覆盖是有限集合; 未知歧义类对 interrogator 不可见。P5 的模板覆盖本身就是 statement-layer blind spot。
- **防守**: P5 从不声称模板完备——它声明: ① 覆盖已知歧义类; ② 无匹配时回退标准三问; ③ "不清楚→不猜"。interrogator 自己是 L1/L2 规则 + 不完备覆盖 + 人工补洞——它是歧义的"正确性探针", 不是完备保证。
- **价值**: interrogator 的盲区因为框架教我们找盲区而被看见——这证明框架的盲区检测是通用的 (能指向自身), 是特性不是缺陷, 但必须明说。

刀 9 (Q9) : buzzword 检测器会不会制造确认偏差?

裁决:  承认精度/召回权衡,  方向性辩护

- **承认**: 触发式启发必然有 ① 漏报 (无 buzzword 的严重 trust recursion 被少查) ② 误报 (严谨的多智能体验证因出现"consensus"被多疑)。这是 precision/recall 权衡, 不是正确性保证。
- **防守**: 压力的方向是逼问 (强制回答锚问题), 不是预判 (不因 buzzword 直接判 Lo)。严谨的多智能体系统回答"聚合确定性锚输出"→ L2, buzzword 不改变结论, 只强制澄清。
- **公平的后续**: 校准实验 (相同底层语义下, 触发/不触发的分类是否一致) 是正当的实证请求——列为 future work。

刀 10 (Q10) : VAL 是把 verification 变成 meta-verification, 而不是解决它?

裁决:  承认,  这正是贡献, 但需正名

- **承认**: VAL 自己不做验证。它回答"某个验证声称可不可信、到什么程度、怎么加强"——是"关于验证的验证"。
- **防守 (核心)**: 在系统性压制局限性披露的领域里, "知道自己验证不了什么"就是贡献。框架是诊断 + 处方 + 审计三合一 (判级器+抬级器+审计报告), 不只是"limitation disclosure"——它同时告诉你怎么变强。建议正名: **verification accountability framework (验证问责框架)**, 而不是自贬为 limitation disclosure。

- **承认底线**：如果审稿人坚持“你没有解决验证”——对，我们没有，而且任何声称解决验证的框架都在撒谎（完备性 ODD 相对 + 信任递归不终止）。诚实本身就是答案。

刀 11 (Q11)：“正交”是逻辑独立还是经验上未发现相关？

裁决：✅ 承认——“orthogonal”用词过强

- **承认**：17 篇论文支持“没有蕴含关系”，不支持“逻辑正交”。粒度更细可能更易形式化、形式化可能改变 ODD、风险等级可能决定锚需求——交互完全可能。
- **修正（论文）**：把“orthogonal”改为“概念上不同的问题（回答不同的问题），经验交互是开放问题”。这是可防守的表述。

刀 12 (Q12)：“我全接受——这些验证理论早知道了，为什么需要 VAL？”

裁决：🟡 核心防守，🔧 论文必须以这条开篇

- **承认**：完备性盲区、规格污染、信任递归——经典验证理论都知道。
- **防守（VAL 的操作性价值）**：
 1. **跨域转移**：经典理论知道这些，但 LLM 验证文献里没有一篇形式化锚定轴（17 篇调查是证据）——VAL 把经典意识带进了一个系统性缺失它的领域；
 2. **新失败类**：Lo（规格由被测对象产生）在经典理论里不存在（规格外生是构造假设）；
 3. **可操作组织**：判级程序 + 可运行工具 + 阶梯 = 给实践者的清单（先问 Q1 再信“已验证”）——经典理论没给这个设置提供清单；
 4. **实证产出**：Alvarado 抬级成功、#105 人工漏判发现、P1-P5 预测 4/5 命中——这些是 VAL 组织出来的成果，不是“用形式方法”能直接产生的。
- **修正（论文）**：必须以 Q12 的回答开篇——“VAL 的贡献是操作性组织 + 跨域转移 + 新失败类，不是发现 soundness/completeness”。如果声称更多，必输。

终极题：完备验证了错误性质的验证器，算高等级吗？

答案：算——而这正是框架的全部意义。

- 等级度量的是**验证机制的承诺强度**（ODD 内完备），不是性质本身的正确性。完备验证错误性质 = L3/L4 的仪器，指着错误的对象。
 - 框架不会假装高等级意味着性质正确——**性质的正确性是上一层锚的问题**（信任递归），而框架的诚实机制（陈述层、Lo 判例）存在的意义，就是让这个差距保持可见。
 - 最深的推论：正因为如此，框架永远不可能“绝对”——性质的正确性会递归，递归终止于定义性/约定锚。**框架不隐藏这个递归，而是把它点亮。**
-

裁决汇总

刀	主题	裁决
1	锚定轴是新瓶装旧酒？	🛡️ 防守成立（经典理论排除规格内生的失败类）
2	level 命名暗示不存在的排序？	✅ 承认命名风险，需加警告框
3	L2 客观性由谁认证？	🔧 必须修正 （锚不得是同一生成器类成员）
4	抬级是问题重写？	✅ 承认，重编码=验证基本动作
5	ODD 免责条款？	🔧 必须修正 （ODD 锚定工具域 + 诚实探针）
6	完备性过度绑定 candidate？	✅ 承认，泛化为语义完备性
7	Rice 用太猛？	✅ 承认需限定（仅限无限制性质类）
8	interrogator 自己的盲区？	✅ 完全承认（框架咬到自己）
9	buzzword 确认偏差？	✅ 承认权衡，方向性辩护
10	只是推迟判断？	🛡️ 承认但正名：验证问责框架
11	"正交"过强？	✅ 承认，改"概念不同+交互开放"
12	为什么需要 VAL？	🛡️ 核心防守，论文须以此开篇

三刀修正（Q3/Q5/Q12）直接进论文改动清单；终极题的答案写进结论。

二轮补刀（2026-08-18，外部评审补充）

第一轮答辩后又收到一轮更深的评审，三条修正被进一步推进，全部采纳：

补刀 A：Q3 从"生成器类"升级为"锚独立性 (anchor independence) "

"同一生成器类"可被攻击：GPT-5 判 GPT-4、小模型判大模型、微调判基础模型——不是"同类"，但可能共享错误机制。

修正（比原答案更一般）：L2 锚的判定标准不是"是否同类模型"，而是——

锚是否与被验证系统共享错误生成机制（错误相关性）。

核心不是来源，是相关性：一个人写的程序验证另一个人的程序，可能独立；
一个模型复制另一个模型的训练数据模式，可能不独立。

操作性代理：对锚与被验证系统做**错误相关性测试**——在同一批输入上跑两者，若错误显著相关 → 锚不独立（降级为 L0/L1）；若错误不相关或锚是定义性的（执行结果/符号化简）→ 独立。定义性锚（程序跑没跑）天然独立，因为其"错误机制"就是程序本身。

补刀 B：ODD 本身也要分级 (auxiliary dimension)

"谁验证工具能力域？SymPy 官方文档说支持 X——文档本身也是声明。"

修正（ODD 等级，新增辅助维度）：

ODD 类型	等级	锚
self-declared ("我的 ODD 是简单方程")	L0	使用者声称——免责条款风险
documented ("SymPy 文档说支持 X")	L1	文档 (本身是声明)
empirically tested (在覆盖该域的基准上实测)	L2	金标
formally characterized (可靠性证明/代数域刻画)	L3	定义性

ODD 从"能力约束"再进一步：**ODD 自身的锚也有等级**——正式刻画的 ODD 才配支撑 L3 完备性声称。已实现为 `val_raise.odd_level()`。

补刀 C: VAL 显示为二维 (strength, spec confidence)

"用户看到 L4 可能误解为'非常可靠'，实际是'对可能错误的问题做了严格证明'。"

修正 (工具显示)：审计报告改为二维——**验证强度 × 规格置信** (anchored / claimed / unknown) 。

已实现为 `audit_report()` 新列：强度 (L0–L4) + 规格置信 (有独立锚 / 仅声称 / 未标注)。

补刀 D: VAL 自己的锚是什么？ (元问题, 正式回答)

如果 VAL 也必须被 VAL 审计，VAL 的 anchor 是什么？

回答：VAL 不需要逃离递归——它**显式标记递归发生的位置，并规定每级停止在哪**：

VAL 自身的锚	层级
判级/抬级/追问程序 = 确定性规则 (可判定)	L3 (规则本身可判定、可复现)
输入 = 声称者声明	L0 (工具不验证输入真实性, 已明示)
经验基础 = 四域实证 + 17 篇文献 + 版本化修正轨迹	L2 (金标/可审计)
递归停止点 = 定义性锚 (执行/化简/类型) 或约定 (计量学)	约定

VAL 的 anchor 不是"逃离递归"，是"把递归画成一张地图，并标注每一级的停止点"。这正是显微镜的哲学：不消除误差，告诉你分辨率、误差、哪些尺度看不见。

第四部・裁判の裁判

第 14 章

11 · 判级可复现性研究：VAL 判级程序的裁判（第一轮）

研究问题：docs/o6 §8 声称"判级程序是确定性的——任何人/任何 AI 对同一输入应得到同一等级"。

本研究用独立盲评测这个声称：两个评分者对同一语料独立判级，测一致率与 κ ，把分歧归因并修订协议——这是"裁判的裁判也需要裁判"的实证版。

状态：第一轮完成（2026-08-20），第二轮进行中。数据：reliability/。

o. 为什么做（动机锚定）

论文 *Grading the Graders* 的 Limitations #5 亲口承认：**"VAL itself is unvalidated as a measurement"**——

判级程序的确定性是"标准"的最低要求，但从未被测量。本研究补这个洞。

1. 设计

要素	选择	理由
语料	48 条验证方案（17 文献 + 11 系统组件 + 20 新文献）	覆盖 Lo–L4/N/A 全谱系，含边界案例
协议	reliability/protocol_v2.md（冻结）	判级三问 + 边界规则 R1–R8
Rater A	作者（docs/o7 + docs/o6 §7 既有判级）	28 条有作者答案（17 文献 + 11 系统）
Rater B	LLM 盲评（全新上下文子代理，只读协议+证据包，禁止接触任何含答案文件）	独立、可复现、锚定状态披露
测量	等级集合精确一致率 + Cohen's κ	等级是"标准"的输出，先测它

锚定披露（诚实声明）：Rater B 是 LLM 评分者（Lo/L1 锚）。按 VAL 自身框架，LLM 的一致性只能测

协议的可复现性（procedure reproducibility），不能构成效度证据（validity）——效度需要金标锚。

本研究测的是前者；后者留给未来的人工评分者研究。

2. 第一轮结果（Rater A vs Rater B）

- 重叠条目：**28 / 48**
- 等级精确一致率：**14/28 = 50.0%**
- Cohen's κ : **0.445**（中等）

- 分域: lit17 = 47.1% (8/17) , sys = 54.5% (6/11)

$\kappa=0.445$ 远低于"标准"应达到的强一致 (通常 $\kappa \geq 0.8$) 。**第一轮结论: 确定性的声称不成立——协议存在系统性歧义。**

3. 分歧归因 (14 条 → 3 个系统性模式)

模式 A: Q1 "规则 vs 真值" 边界 (S01/S02/S05/S09/S10, 5 条全在 sys)

- 分歧: 作者判 L2 (客观真值) , Rater B 判 L1 (确定性规则) 。
- 归因: 协议没澄清"题面给的方程"与"从 LLM 输出派生的规则"的区别——两者都可读成"规则"。
- **附带发现 (重要)** : S10 医疗裁判 (作者自造规则) 作者判 L2、Rater B 判 L1——按 VAL 哲学 **B 更对**: docs/o6 §7 自己写着"自造规则未验证", 未经验证的自造规则不构成客观真值锚。作者原判自相矛盾。
- 协议修复: 新增 **R9** (题面真值=答题前已存在→L2; LLM 派生规则=答题后提取→L1) + **R10** (自造未验证规则→L1) 。

模式 B: 混合系统报告粒度不一致 (L07/L09/L10/L12/L17, 5 条全在 lit17)

- 分歧: 作者给粗标签 (L0/L1、L1/L2、L2) , Rater B 按协议 R2/R3 严格拆分 (L0、L0+L2、L0+L1+L2) 。
- 归因: 协议 R2/R3 已规定"报告最弱锚", 但作者 docs/o7 的标签没严格执行——**作者的文献判级与自己的协议不一致。**
- 协议修复: 新增 **R11** (判定路径定义: 从证据到最终判定所经组件; 报告等级=路径最弱锚) , 并强化 R2/R3 的强制拆分。

模式 C: 基准 vs 运行时 (L11, 1 条)

- 分歧: 作者判 L2 (金标基准) , Rater B 判 N/A (纯基准, 非运行时验证器) 。
- 归因: 协议 R1 已写"判运行时不判基准", 作者对 L11 自己标注"非运行时验证器"却仍给 L2——执行不一致。
- 协议修复: R1 补明"纯评测基准 → N/A"。

低危项 (3 条, 非系统性)

- L08 (L3/L4 vs L4) 、 L13 (L1/L2 vs L2) : 作者 hedge 双标签 vs Rater B 承诺单标签——集合比较的规范化伪差, 实际语义接近。
- L02 (L0/L1 vs L0) : 作者对未知节点机制 hedge, Rater B 按最弱锚承诺 L0。

4. 协议修订 (v2.1, 2026-08-20)

1. **R9**: 题面真值 (答题前已存在) → truth/L2; LLM 派生规则 (答题后提取) → rule/L1;
2. **R10**: 设计者自造、未外部验证的规则 → L1 (输入客观≠规则有背书) ;

- 3. **R11**：判定路径 = 证据→最终判定的组件链；报告等级 = 路径最弱锚；
- 4. **R1 澄清**：纯评测基准 → N/A。

修订对作者判级的预测影响（待第二轮验证）：

- S01/S02/S05/S09 → 维持 L2（题面方程=R9 truth）；
- S10 → **L2 降为 L1**（R10 自造规则）——作者判级的一次协议驱动修正；
- L07/L17 → L0/L1、 L1/L2 按 R2/R11 应降为 L0（判定路径最弱锚）；
- L11 → L2 改为 N/A（R1）；
- L09 → 按 R11 认知层在判定路径 → 报告 L0+L2 中的最弱 L0（或明确"金标层=L2 证据组件"）。

5. 第二轮（v2.1 协议）

5.1 结果（Rater B2 盲评，全新实例）

对比	一致率	κ	说明
A vs B（第一轮 v2.0）	14/28 = 50.0%	0.445	基线
A vs B2（第二轮 v2.1）	17/28 = 60.7%	0.549	协议修订后 ↑
B vs B2（LLM 轮次漂移）	34/48 = 70.8%	0.647	lit17 稳定 82.4%；sys 按设计变动

- 分域（A vs B2）：lit17 52.9%、**sys 72.7%**（首轮 54.5% → 大幅提升——R9/R10 直接修复了 sys 分歧）。
- **结论：协议修订有效，但 κ=0.549 仍未达"标准"的强一致门槛（≥0.7）。确定性声称部分成立——修订把可复现性从"差"提到"中等"。**

5.2 协议修订的实证效果（预测全部兑现）

条目	v2.0 分歧	v2.1 后	兑现的规则
S01/S02/S05/S09	A=L2 vs B=L1	一致 L2	R9（题面方程=答题前真值）
S10 医疗裁判	A=L2 vs B=L1	B=L1（作者仍 L2）	R10（自造规则→L1）—— 作者判级需修正
L11 M3-SafetyBench	A=L2 vs B=N/A	B=N/A（作者仍 L2）	R1 澄清（纯基准→N/A）—— 作者判级需修正
L07/L17	A=L0/L1·L1/L2 vs B=L0	B=L0（作者仍粗标签）	R11（判定路径最弱锚）—— 作者判级需修正
L12 Output Drift	A=L1/L2 vs B=L0+L1+L2	一致 {L1,L2}	R11（自洽统计不单独成 L0 组件）

5.3 新暴露的协议缺口（第三轮待修）

- 1. **符号工具 = "规则"还是"可判定系统"？（So4）**：B2 把 sympy simplify 判 L1（规则），但论文 §3.4（preprint 第 171 行）明确"符号简化是合格客观锚"→ 应判 L2（用法降级）或 L3。
协议缺一条：**R12 提案——符号计算引擎（solveset/simplify/执行内核）= 可判定系统（Q1=decidable），用于正确性=L2（用法降级），用于完备性=L3。**

2. **判定路径的组件归属 (Lo9/L10)**：Dr. V 认知层、Proprietary Safety 的解释层是否在"判定路径"上？B2 对 L10 从 Lo+L2 漂移到 L2 (R5 分类器标签优先) ——需要明确"以哪个组件产出最终判定"。
3. **hedge vs 承诺约定 (Lo2/Lo8/L13)**：作者倾向双标签 hedge，Rater B 倾向承诺单标签。需规定：证据不足时是"取最弱锚承诺"还是"允许 Lx/Ly 双标签"。

5.4 对论文的实证反哺 (v2 待办)

1. **§3.3 确定性声称** (第 192 行"两个评分者对同一方案必须得到同一等级")：本研究显示 Q1/Q2 答案在评分者间系统性分歧 (首轮 50%)，**v2 应把该句改为引用本研究**："程序映射是确定的；Q1/Q2 输入的判定需评分者+证据包 ($\kappa=0.549$, docs/11)"——诚实化 + 顺手把"裁判的裁判"做成论文自己的实证。
2. **§5.3 医疗裁判等级** (第 279 行 "L2" vs 第 281 行 "L1/L2")：按 R10 修正为 **L1/L2** (手写规则)，与第 281 行自洽。
3. **docs/o6 §7 审计修正** (见下)。

5.5 第三轮 (v2.2 协议，完成)

协议 v2.2 增补：R12 (符号计算引擎=可判定系统)、R13 (判定路径归属：判定产出组件定级，证据/解释层仅标注)、R14 (hedge 承诺约定：证据不足也须承诺单标签，最弱锚为准)。

结果 (Rater B3 盲评，全新实例)：

对比	精确一致率	κ	最弱锚一致率	κ
A vs B (v2.0)	50.0%	0.445	—	—
A vs B2 (v2.1)	60.7%	0.549	—	—
A vs B3 (v2.2)	50.0%	0.434	67.9%	0.599
B2 vs B3 (协议收敛后)	83.3%	0.794	89.6%	0.865
B vs B3 (v2.0→v2.2 漂移)	72.9%	0.665	77.1%	0.693

5.6 三轮回溯：什么在起作用，什么不是

1. **协议收敛后，评分者间可复现性达到强一致 (核心结论)**。

B2 vs B3 (同一协议时代 v2.1/v2.2) 精确 $\kappa=0.794$ 、最弱锚 $\kappa=0.865$ ——远超 0.7 门槛。
论文 §3.3"两个评分者对同一方案必须得到同一等级"的声称，在**协议精化后**得到实证支持 (评分者间)；v1 版本协议 (v2.0) 做不到，是协议歧义，不是映射不确定。

2. **作者 vs LLM 的残余差距来自两个可解释因素，都不是协议缺陷**：

- **(a) 证据包过度清洗 (研究设计缺陷)**：So3/So9 的清洗把"穷尽性/真值锚"这类机制事实也删了，导致 B3 低估 (So3 L1 vs 作者 L3；So9 L1 vs 作者 L2)。教训：**清洗结论性措辞 ≠ 删除机制事实**。已修复证据包 (make_pack.py v1.1)。

- **(b) 作者 hedge vs R14 承诺**：作者 docs/o7 用 Lo/L1、L1/L2 双标签 hedge；B3 按 R14 承诺单标签。

用最弱锚口径比较，Lo/L1 vs Lo 即一致——最弱锚一致率 (67.9%) 显著高于精确一致率

(50.0%)。

结论：作者文档应按最弱锚口径统一输出承诺等级（或至少标注最弱锚）。

3. 作者判级的系统性偏差（协议驱动审计输出，全部已定位）：

- S10/S11（自造规则判 L2 → 应为 L1, R10）；L11（基准判 L2 → 应为 N/A, R1）；Lo9（判定路径未识别 → 报告 Lo, R13）；Lo7/L17（最弱锚 Lo）。
- 这些不是"评分者分歧"，是作者原始判级与精化协议的不一致——研究直接产出了 docs/o6 §7 的审计修正。

4. 剩余真边界（保留为开放问题，非协议缺陷）：

- N11/N13（LLM 生成测试的执行选择）：Lo（测试期望 LLM 声明，信任递归）vs L1（执行确定性）——取决于"测试 oracle 的生成者"是否算判定路径一部分；R12/R13 交叉处的真实模糊。
- L01/L12（VerifiAgent、Output Drift）：多组件弱锚排序约定（L0+L1 vs L0+L2）——最弱锚一致。

5.7 结论（三轮回溯）

"裁判的裁判"实证成立：VAL 判级程序在协议精化（v2.2）后达到评分者间强可复现（ $\kappa \approx 0.79-0.87$ ）；作者原始判级存在 6 处系统性偏差，全部定位并给出修正方向；剩余分歧可归因于证据包设计（已修）与作者 hedge 约定（待统一）。
论文 v2 的两处实证修正已确认（§3.3 确定性声称改为引用本研究；§5.3 医疗裁判 L2→L1/L2）。

5.8 第四轮（v2.2 + 修复后证据包，完成；含 agentsec 语料 N21–N26）

Rater B4 盲评 54 条（全新实例，协议 v2.2，证据包含修复后的 S03/S09）。

评分者间一致性（三个独立 LLM 评分者两两对比）：

对比	精确一致率	κ	最弱锚一致率	κ
B2 vs B3	83.3%	0.794	89.6%	0.865
B2 vs B4	85.4%	0.821	89.6%	0.866
B3 vs B4	85.4%	0.817	87.5%	0.837

结论强化：协议 v2.2 在三个独立评分者间稳定达到强可复现（ $\kappa \approx 0.82-0.87$ ）。

S09 证据包修复生效：B4 判 L2（题面条件=R9 真值），B2 亦 L2；但 B1/B3 判 L1——2:2 分歧，属 R9 应用方差（B1/B3 把"代入题面条件"读成统计规则），协议规则本身（R9）指向 L2。

S03 暴露真协议缺口（R12 需修正）：B2=L3、B3=L1、B4=L1。证据包修复（补"穷尽极值候选"）未消除分歧——R12 的判定测试（"现成形式引擎 vs 人写决策程序"）把定理支撑的穷举也归为 rule/L1，但闭区间极值定理使端点+驻点枚举完备（候选集穷尽性可证明）。提案 R12b：定理/证明支撑的穷举过程 = 可判定系统（Q1=decidable，完备用法→L3）；无完备性背书的启发式枚举 = 规则（L1）。待 v2.3 冻结后验证。

agentsec 语料 (N21–N26) 盲评验证全部命中 docs/12 预测：
PPMF=L1 (承诺最弱锚； docs/12 预判 L1/L2) · Content/Prompt Filter=L1 · Self-Ask=Lo · Gate-only=L1 · Handle/Marker guard=L1。

5.9 第五轮 (v2.3 协议，完成)

协议 v2.3 冻结： R12b (定理支撑的穷举=可判定 L3)、R15 (LLM 生成的验证期望→Lo)、R9 采样澄清 (对照题面条件→L2)、锚语义正交声明 (docs/13)。

Rater B5 盲评 54 条 (全新实例, v2.3)。 定向修订效果全部兑现：

条目	第四轮分裂	v2.3 后 B5	修订规则
So3 (区间极值枚举)	B2=L3 vs B3/B4=L1	L3 (与作者一致)	R12b: 定理支撑的穷举→L3
N11/N13 (LLM 生成测试)	B1/B3=Lo vs B2/B4=L1	Lo	R15: LLM 生成的期望→最弱锚 Lo
So9 (采样验证)	2:2 (L1/L2)	L2 (与作者一致)	R9 澄清: 对照题面条件→truth

评分者间一致性 (四个独立 LLM 评分者两两)：

对比	精确 κ	最弱锚 κ
B2 vs B3	0.794	0.865
B2 vs B4	0.821	0.866
B3 vs B4	0.817	0.837
B2 vs B5	0.766	0.811
B3 vs B5	0.813	0.835
B4 vs B5	0.837	0.857

结论 (五轮收束)： 协议 v2.3 在四个独立评分者间稳定达到强可复现 ($\kappa \approx 0.77-0.87$)；最后三个已知协议缺口 (So3/N11/N13/So9) 经 v2.3 定向修订后全部消除 (B5 与作者一致)。B4-B5 残差分歧为已知边界 (L10 分类器标签 vs LLM、L12 Output Drift、N16 断言 R10 vs R9)。
五轮迭代轨迹： v2.0 (50%, κ 0.445) → v2.1 (60.7%, 0.549) → v2.2/v2.3 (评分者间 $\kappa \approx 0.8+$, 缺口清零)。
协议 v2.3 可作为"标准"候选版本。

5.10 待办

- 作者补判 No1–N26 (可只判 6 条 N21–N26) → 全量 54 条人机对比；
- 作者判级按最弱锚口径重判 (消除 hedge) → A vs B 全量 κ ；
- 论文 v2 §3.3 已引用本研究 ($\kappa \approx 0.8$) ——补判完成后可将人机 κ 一并写入。

6. 诚实的边界

- Rater B 是 LLM：** 一致率是"可复现性"不是"效度"；真效度需人工评分者 (后续可招募)；
- 等级集合比较的粒度：** L3/L4 vs L4 这类子集差异被记为分歧，实际语义接近——第二轮可加"最弱锚等级"对比维度；

3. **作者判级本身是研究的一部分**：本研究发现的作者判级错误 (S10、L11、L07/L17) 是对 docs/o6 §7 / docs/o7 的审计输出，修正轨迹保留（追加版本，不改写历史）；
4. **语料 20 条新文献** (N01–N20) 尚无 Rater A 答案，待作者补判后纳入全量对比。

7. 数据与复现

- 语料：reliability/corpus.json (48 条，含作者判级)
- 证据包（盲评版）：reliability/evidence_pack.json
- 评分：reliability/ratings/rater_a.json · rater_b.json · agreement_report.json
- 分析：python reliability/analysis.py

13 · 锚语义类型学 (Anchor Semantics Typology)

动机来源：PPMF 论文 (arXiv:2607.29167) ——USER_CONFIRMED 标签保证"用户意图指向该目标",
不保证"动作将正确执行"。这是 VAL 没有显式化的一个维度：同一个 L2 锚，语义类型不同，判定含义就不同。
状态：概念稿 (2026-08-20)。这是 VAL 的可发表增量概念候选。

o. 一句话

VAL 判等级 (锚多强)，锚语义类型学判"这个锚到底认证了什么" (意图 / 真值 / 效果) ——两个维度正交。

1. 三种锚语义

语义类型	认证什么	典型例子	失败模式
Intent (意图锚)	意图指向目标: "用户/平台记录了这个动作指向这个目标"	PPMF 确认事件 (USER_CONFIRMED)、授权策略、用户请求	伪造/社工 : 假确认标记、社会工程骗真确认 (PPMF: 10% 伪造 → ASR 0.088)
Truth (真值锚)	事实一致: "答案/声明与客观真值一致"	金标答案、代回方程、solveset 解、实测值	过时/错误真值 : 金标错、数据过期、方程写错
Effect (效果锚)	执行结果: "动作实际发生/成功"	执行反馈、server-side state change、运行结果、测试执行	部分执行/假成功 : 副作用未发生但报告成功、执行成功但目标错误

2. 与 VAL 正交

等级和语义是两个独立维度：

- 同一个锚可以是 L2 的意图锚 (PPMF 确认事件——平台记录，客观)，也可以是 L2 的真值锚 (金标——客观)，或 L2 的效果锚 (执行结果——客观)；
- 等级回答"锚有多强/独立" ($Q_1 \times Q_2 \times Q_3$)，语义回答"锚认证的是什么"——**后者决定判定含义**：
 - 意图锚的"正确" = 授权正确 (动作该不该做) ——**不涉及结果对不对**；
 - 真值锚的"正确" = 答案正确 (与事实一致) ——不涉及该不该做；
 - 效果锚的"正确" = 执行正确 (确实发生了) ——不涉及是否该发生。

PPMF 的洞察翻译：门禁是意图锚门控的授权边界（L2 意图锚 + L1 策略规则）；它不验证真值（购买是否划算）也不验证效果（支付是否真的执行成功）。**"已确认"与"已正确执行"是两码事——把确认当效果验证用，就是锚语义混淆。**

3. 失败模式对照

语义混淆	后果	例子
意图锚当效果锚	授权了 = 以为执行了	确认购买，支付实际失败但 UI 显示成功
意图锚当真值锚	用户想要 = 以为事实正确	用户确认了错误的转账目标
真值锚当意图锚	事实正确 = 以为该做	金标答案正确但用户根本不想做这件事
效果锚当真值锚	执行了 = 以为结果对	测试通过但覆盖的是错误断言

这三种混淆里，PPMF 修的正是第一种~~的时域版~~：来源洗白 = 外部记忆（低权威意图/真值）被重写成用户历史（高权威意图），平台没保留语义类型与权威。

4. 本项目自己的锚语义盘点

验证器	等级	锚语义
verify_root 代回	L2	Truth （题面方程=真值）
solution_set solveset	L3	Truth （完备解集）
answer_type 类型系统	L3	Truth（类型符合是事实）
医疗信息完整性裁判	L1/L2	Truth （lab 缺失是事实；规则是自造的）
行为统计检测器	L1/L2	Truth（偏差是测量的；阈值是自造的）
终局验证 final_parameter_set（规则派生）	L2	Truth （采样对照题面真值）
终局验证（LLM 规格回退）	L0	无锚 （已降级 UNSURE, docs/12 §4.1）
第四章执行反馈（mypy/测试）	L2/L3	Effect （执行/类型检查结果）
PPMF 门禁	L1/L2	Intent （确认事件+策略）

观察：本项目几乎全是 Truth/Effect 锚，没有 Intent 锚——因为数学/诊断域不做授权决策。**PPMF 补上了 Intent 锚这个象限；**方向 3（Agent 安全）正好需要三个象限全用：Intent（授权）+ Truth（事实核查）+ Effect（工具执行结果）。

5. 论文落点（v2）

- §3.2（锚定三问）后补一段：L2 锚可按语义类型细分（intent/truth/effect），判定含义随类型而变；
- §4（完备性盲区）用 PPMF 作安全域实例时，顺带点出：意图锚的完备性盲区 = 伪造/社工路径无法穷尽（与真值锚的漏解盲区同构）；
- docs/13 全量作为补充材料索引。

6. 开放问题

1. 锚语义应作为判级程序的 **Q4** (编码进 `val_standard.py`) , 还是保持**正交标注** (判级时附注语义类型) ?

倾向后者: 语义是判定的语义解释, 不是等级判据; 但可加一个 `semantics` 字段
(`intent/truth/effect/mixed`) 。

2. 混合锚 (一个判定同时依赖 `intent+truth+effect`) 如何标注? ——按最弱锚 + 多语义标注。
3. 语义类型能否帮助 ODD 划定? (意图锚的 ODD = 确认机制可信; 真值锚的 ODD = 真值未过时; 效果锚的 ODD = 执行环境受控) ——候选研究方向。

第五部・Agent 安全（方向 3）

12 · VAL × Agent 安全：PPMF 案例（方向 3 开篇）

来源：Jinghan Xu 等, *Memory Provenance Laundering in LLM Agents: A Non-Amplification Firewall for Persistent Memory* (arXiv:2607.29167v1, 2026-07) 。

收到途径：Paperbrief 研究摘要按本文 arXiv 论文关键词画像推送。

状态：案例初读完成 (2026-08-20) ； 判级待盲评/作者复核。

定位：方向 3 (Agent 安全) 的第一块实证砖——一个真实、新鲜的 Agent 安全防御，其核心机制与 VAL 高度同构。

o. 论文一句话

LLM 代理用长期记忆复用偏好，但 LLM 压缩记忆时会把"外部网页的低信任来源"重写成"用户历史的高信任来源"——动作触发器保留、来源权威被洗掉（**memory provenance laundering / 记忆来源洗白**）。作者提出 **PPMF**：记忆携带平台维护的来源元数据，工具调用前用确定性门禁按"动作风险 ≤ 来源权威"授权。

1. VAL 同构映射（为什么这篇论文是 VAL 的天然案例）

PPMF 概念	VAL 对应	说明
信任阶梯 U<E<T<H<C<S	Q1 锚定轴	来源按权威排序 = 锚定层级的 Agent 版 (USERHISTORY≈L1 声明、USER_CONFIRMED≈L2 客观事件、SYSTEM≈L3/L4)
non-amplification : "记忆可以保留信息，但不能放大来源权威"	VAL 不对称性 : "保证只能向下退化，不能向上抬升"	同一条原理的两种表述
洗白攻击 = 压缩把 L0/L1 来源重写成 L2 外观	信任递归 / 伪造锚	判定依赖"文本看起来像什么" (Lo) 而非"锚是什么" (平台元数据)
"LLM 只做规划，平台强制授权"	"LLM 只做翻译，确定性代码做裁判"	项目核心教义在 Agent 安全的落地
门禁只读平台元数据，不读生成文本	"验证规格锚定在不依赖 LLM 声明的地面真值"	第 3 章医疗裁判同款设计哲学
确认伪造 (confirmation-spoofing)	完备性盲区	门禁验证了"正确的部分"，没验证"没被伪造的部分"

2. VAL 判级 (reliability 协议 v2.2)

2.1 PPMF 门禁本体

- **Q1**: 门禁 = 设计者指定的风险策略 (READ≥EXTERNAL、NAV≥USERHISTORY、EFFECT/PURCHASE/CREDENTIAL≥USER_CONFIRMED) → 规则 (R10) ; 输入 = 平台维护的确认/来源/风险元数据 → 客观 (L2)
- **Q2**: 正确性 (拦截未授权动作) ; 无完备性
- **报告等级**: **L1/L2** (L2 输入 + L1 策略规则) , 正确性, ODD = 平台元数据完整

完备性盲区的量化证据 (论文自己的数字) : 元数据完整时 ASR=0.000 (95% Clopper-Pearson 上界 0.005, 仅覆盖评估分布) ; **10% 伪造确认** → **ASR 0.088±0.016** ; 10% 风险降级 → ASR 0.112±0.006。这就是"正确性探针, 非完备性保证"的实证。

2.2 基线判级对比 (VAL 分界预测实验胜负)

防御	VAL 等级	论文数据 (ASR / 良性成功)	VAL 解读
Content Filter	L1 (词表规则)	0.200–0.933 / 不稳定	表面规则, 改写即破
Prompt Filter	L1 (模式规则)	0.933–1.000 / 镜像 ND	洗白后无操作点
Action-handle/Confirm-marker	L1 (启发式)	0.818–1.000 / 部分	伪造标记可绕过
Self-Ask provenance	Lo (LLM 自评)	permissive 0.984 / 0.979	信任递归: 文本说啥信啥
Gate only	L1 (规则无 L2 输入)	0.000 / 0.000 (良性全杀)	有门无锚, 误杀
PPMF	L1/L2 (元数据锚+确定性门)	0.000 / 1.000	唯一同时达标

结论: VAL 的锚定分界 (Lo 文本自评 vs L2 平台元数据) 恰好预测了这组实验的胜负——这是 VAL 框架在 Agent 安全域的第一次外部实证。

3. VAL 能为 PPMF 增加什么

1. **Q2/Q3 语义缺失**: PPMF 的信任阶梯只有 Q1 轴 (来源权威排序) , 不携带保证语义。USER_CONFIRMED 保证的是"用户意图指向该目标" (意图锚) , **不是**"动作将正确执行" (效果锚) ——两个锚不同, VAL 能把这层区分显式化: 门禁验证的是授权边界, 不是动作正确性。
2. **ODD 形式化**: PPMF 的保证边界 (平台元数据完整) 就是 VAL 的 ODD; 元数据被伪造/降级 → 自动退回 L2 之外, 诚实的边界声明。VAL 提供语言: L1/L2 判定 + ODD 内成立 + 超出即失效。
3. **失败模式归类**: 来源洗白 = 信任递归的时域版本 (规格声明者=LLM 压缩器, 被信任的输出=记忆) ; 补进失败模式分类学可作为"信任递归在持久层"的第 9 个模式或子类。

4. **抬级路径** (L2→L3)：若确认事件用**密码学签名/区块链式 append-only 日志**（防伪造），确认锚就从"平台声明"升为"可验证的客观真值"，门禁仍是正确性 (L2) ——完备性盲区依然在（无法证明"没有伪造路径"）；若要完备，需要形式化"授权路径"性质并编码进可判定系统——大概率不可判定（开放世界），诚实封顶 L2。

4. 对方向 3 的意义（测试床起点）

- **主线失败模式**：来源洗白（authority laundering）= 信任递归在记忆层的实证，与第二/三章的隐匿执行、信息不足下结论并列；
- **对照锚**：PPMF 是现成的 L1/L2 防御，可作方向 3 判级测试床的"标准答案"候选（连同 N22–N26 基线，已入 reliability 语料 N21–N26）；
- **方法复用**：判级协议 v2.2 (R1/R9/R10/R13) 直接适用；Agent 安全声称（"我们的安全过滤器很安全"）全部可判级；
- **文献入口**：PPMF 引用了 AgentDojo/BrowseSafe/IFC/CaMeL/Progent 等——方向 3 的相关工作地图可从它展开。

2.3 盲评验证（2026-08-20, reliability 第四轮 Rater B4）

docs/12 的判级预测经独立盲评（协议 v2.2）**全部命中**：

条目	预测	B4 盲评	结果
N21 PPMF	L1/L2	L1 （承诺最弱锚）	✓
N22 Content Filter	L1	L1	✓
N23 Prompt Filter	L1	L1	✓
N24 Self-Ask	Lo	Lo	✓
N25 Gate only	L1	L1	✓
N26 Handle/Marker guard	L1	L1	✓

PPMF 的 L1/L2 判级（客观平台元数据输入 + 设计者策略规则 + 正确性保证）得到独立评分者确认；
其完备性盲区（10% 伪造确认 → ASR 0.088）是"正确性探针非完备性保证"的量化证据，判级口径一致。

4.1 自审计：我们自己的洗白面（PPMF 启示的第一次真正使用）

用 PPMF 透镜审计本项目管道（final_check_deriver.py + main.py），发现并修复了残余信任递归：

审计发现：

1. 规则派生覆盖率 **6/20 = 30%**——只认"区间+条件"模式；11 题无区间、3 题有区间无匹配条件；
2. 其余 70% 终局规格要么回退 LLM 声明（fc_source="llm", Lo 锚），要么无规格；
3. **回退路径是 fail-open**：LLM 声明规格与规则派生规格同权验证，PASS 同样认证答案——这正是 PPMF 说的"来源权威被洗掉后仍被信任"。

修复 (main.py 审计修正第 4 波)： LLM 声明规格 (Lo 锚) 下，终局验证 **PASS 降级为 UNSURE** (fail-closed, PPMF 式)；FAIL 保留 (自相矛盾是真实信号)；规则派生 (L1 锚) PASS 维持。新增 final_check_anchor 字段记录规格锚等级。

修前/修后对比 (reliability/audit_fallback.py, 历史数据 + 确定性重建, 未重跑 LLM)：

指标	修前	修后
终局验证 PASS	6	5
终局验证 UNSURE	13	14
answer_verified	6	5
被降级条目	—	#18 (llm 规格 + 原 PASS → UNSURE)

巧合但意味深长： #18 正是 docs/o2 模式 8 的"假 FAIL (误杀正确答案)"条目——它的终局规格因无法用规则表达 (零点类条件) 而回退 LLM 声明，历史运行给了 PASS (伪认证)。**信任递归的残余恰好藏在已知问题条目上。**

诚实边界： 对比基于历史运行数据的确定性变换 (重建 fc_source + 应用降级规则)，不是新 LLM 运行；真实新运行的降级数可能不同，但结构性结论不变——凡 LLM 声明规格的 PASS 一律不再认证。

5. 诚实边界

- 1. 只读了 arXiv v1 (2607.29167v1)，判级依据摘要+正文机制描述，未全文精读附录细节；
- 2. PPMF 判级 (L1) 已获盲评确认 (B4)，但仍待作者 (用户) 复核定稿；
- 3. 论文实验环境为合成/沙箱，ASR 数字不可外推到生产；我们引用它们只为说明"完备性盲区被量化"这一结构性事实；
- 4. 邮件交流已起草 (promo/email-to-ppmf-team.md)，等对方回应后再深化合作层面。

6. 相关文件

- 语料条目：reliability/corpus.json N21–N26
- 邮件草稿：promo/email-to-ppmf-team.md
- 判级协议：reliability/protocol_v2.md

14 · VAL→ASR 预测协议：把判级变成可证伪的预测器（方向 3 方法论）

目的：方向 3（Agent 安全）的研究设计——用 VAL 判级协议**预测**安全防御的成败（ASR/良性），
再对照实验数据验证。这是第四章"反向验证"（预测先写死、证据后撞）在 Agent 安全域的应用。
状态：预测规则已从 PPMF 一轮数据归纳（N=6 防御，post-hoc 演示）；前向闭环待方向 3 测试床执行。

o. 核心主张

判级协议的输出（L0–L2 等）可以翻译成可证伪的防御行为预测——因为等级决定了"防御的锚在哪"，而锚决定了"什么攻击能突破它"。一个防御被击败的方式，是其锚的失效方式。

1. 预测规则表（已归纳，N=6 防御，来自 PPMF 数据）

VAL 等级	锚的特征	预测的行为模式	PPMF 实测（ASR / 良性）	命中
L0（LLM 自评/文本推断权威）	无确定性锚；判定=LLM	可被改写/伪造击穿，无稳定操作点；permissive 时高 ASR	Self-Ask permissive 0.984 / 0.979 ；strict 全杀	✓
L1 词表/模式过滤	表面形式规则	改写（洗白/paraphrase）即失效，ASR 高且不稳定	Content 0.200–0.933；Prompt 0.933–1.000	✓
L1 句柄/确认标记启发式	表面形式规则	伪造标记绕过（confirmation-spoofing 专用）	AG/CG/HH 0.267–0.850，伪造场景 0.818	✓
L1 门禁（无 L2 输入）	规则但无客观输入	ASR 低但良性全杀（有门无锚）	Gate-only 0.000 / 0.000	✓
L2 元数据锚 + 确定性门	客观输入 + 规则	ODD 内 ASR≈0 + 良性保留； ODD 外（元数据伪造）突破	PPMF 0.000 / 1.000 ；10% 伪造 → 0.088	✓
L1/L2 混合（L2 输入 + L1 策略）	客观输入 + 自造策略	同 L2，但策略本身无外部背书（策略错=系统性误判）	PPMF 同格（策略固定）	✓

第一轮归纳结论：等级与防御行为模式强对应——失败的防御全部是 L0/L1（锚在文本/表面形式），成功的防御是 L1/L2（锚在平台元数据）。VAL 的分界预测了 PPMF 全部 6 个防御的胜负。

2. 前向预测协议（真正的前瞻闭环）

对一个**新的** Agent 安全防御声称（论文/产品/自研），执行：

1. 收集机制描述（不含结果）→ 入语料（evidence pack 格式）
2. 按协议 v2.2 盲评判级（≥2 评分者，R13 判定路径、R14 承诺）
3. 写预测卡（先于看数据）：
等级 → 预期 ASR/良性模式 + 失效条件（什么攻击能突破：改写/伪造/社工/缺失元数据）
预测卡签名冻结（反向验证纪律：预测先于证据，事后不改动）
4. 再看论文数据或跑实验（ASR / 良性 / 伪造压力）
5. 对照：命中 → 等级-行为映射+1 样本；未命中 → 框架或协议的边界发现（比命中更值钱）

预测卡模板：

```
{
  "defense": "<名称>",
  "level_prediction": "Lx (weakest anchor)",
  "predicted_behavior": "ASR 区间/良性模式",
  "predicted_attack_that_breaks_it": "<改写/伪造/社工/元数据缺失...>",
  "signed_date": "...",
  "observed": null
}
```

3. 方向 3 测试床的执行计划

1. **扩充语料**：收集 10–20 个 Agent 安全防御声称（越狱防护、提示注入过滤器、工具授权、记忆防护、RAG 引用验证……），全部按 evidence-pack 格式入语料（N21–N26 已入 6 个）；
2. **判级**：v2.2 协议盲评（B 系列评分者）+ 作者补判；
3. **预测卡**：逐防御冻结预测（等级→行为）；
4. **验证**：逐防御找实验数据/跑小实验，对照 ASR/良性；
5. **产出**：等级-行为映射表（N 样本）、预测命中率、框架边界（未命中案例的归因）。

3.1 P1+P2 进展（2026-08-20）

P1 测试床（完成）：语料 54 → **70 条**，新增 16 个 Agent 安全防御（N27–N42，reliability/corpus.json），覆盖护栏/分类器/权限/IFC/记忆防御/沙箱/形式验证/基准全谱系。

P2 判级（完成）：Rater B6 盲评 70 条（v2.3 协议）。**B5 vs B6 一致性破纪录：κ=0.882（精确）/ 0.926（最弱锚）**，agentsec 6/6、sys 11/11 全一致——协议 v2.3 可复现性再创新高。

等级图（Agent 安全防御的 VAL 全景）：

等级	防御	保证性质
L0	系统提示加固 (N34)、ToolEmu (N42)、A-MemGuard (N32)、SafeAgent 计划层 (N41 弱锚)	无确定性锚, LLM 自评
L1	NeMo Guardrails (N28)、Progent (N30)、白名单 (N35)、Perplexity 过滤 (N36)、PPMF 门禁 (N21)、各类文本过滤器 (N22/N23/N26)	确定性规则, 可改写/绕过
L2	Llama Guard 分类器 (N27 组件)、RAG 引用核对 (N33)、执行效果核对 (N39)、统计检测器 (N19)	客观锚正确性, 分布内有效
L3	IFC (N31)、CaMeL (N29)、沙箱 (N37)	构造性单性质完备: 信息流/隔离/confine
L4	智能合约形式验证 (N38)	形式内核域内完备
N/A	AgentDojo (N40)	评测基准 (R1)

关键发现 (docs/12 开放问题的答案) :

Agent 安全的 L3 抬级路径 = confinement/信息流性质 (构造性保证: 隔离、污点传播、执行域限制——由机制保证而非统计), **而语义安全 (开放世界) 封顶 L2**——与论文 §4 二分法完全同构。授权策略本身 (PPMF/Progent/白名单) 是 L1 设计者规则; **结构性的授权 (IFC) 才到 L3**。这个"L3 在隔离不在语义"的结论是方向 3 的核心理论产出。

预测卡 (已冻结, 先于验证数据) : reliability/prediction_cards.json——16 条新防御逐条等级→行为预测 + 预测的突破攻击方式。observed 字段待 P3 回填。

3.2 P3 验证 (第一轮, 部分)

PPMF 家族 (N21–N26) : 已在第四轮 (docs/12 §2.3) 全量验证——**6/6 命中** (高置信, 数据来自 PPMF 论文本体) 。

新卡 (N27–N42) 第一轮回填 (reliability/prediction_cards.json, 网络不可用, 数字来自收集代理报告, **未核实**) :

卡	等级	预测	现有数据 (未核实)	结果
N27 Llama Guard	L1+L2	分布内有效/OOD 下降	内部准确率 0.9007, 外部基准下降	命中
N34 系统提示加固	L0	强攻击有残余 ASR	RA-LLM: ASR 80-90%→10-25%	命中
N36 Perplexity	L1	单独效果有限	Jain: 仅小幅降低 ASR	命中
N42 ToolEmu	L0	审计有效/非运行时防御	144 场景发现 31 风险 (90% 确认真风险)	命中
N40 AgentDojo	N/A	基准不适用	无防御 ASR 70-90%	基准上下文

第一轮命中率: 4/4 (有行为的防御全命中) + PPMF 6/6 = 10/10 (N40 为 N/A 不计数) 。

其余 11 张卡 (N28–N33/N35/N37–N39/N41) 待网络可用后收集 ASR 数据回填——**预测卡已冻结, 验证不受卡冻结影响**。

P3 诚实边界: 4 条新命中基于收集代理的文献数字 (未在线核实), 只能作为"方向正确"的低置信信号; PPMF 6/6 是高置信证据。全量验证需要网络或用户提供论文。

3.3 P3 验证 (摘要级确认, 2026-08-20, arXiv API 直连)

网络恢复 (Google 被墙但 arxiv 可直连) , 抓取 6 篇论文摘要核实机制与评估数字:

卡	等级	摘要级证据	结果
N28 NeMo Guardrails	L1	可编程 rails, 用户定义、独立于 LLM、可解释; 无公开 ASR	机制确认 (数字待全文)
N29 CaMeL	L3	AgentDojo: 77% 任务 可证明安全 (无防御 84%) ; "untrusted data can never impact program flow"	✔ 命中 (L3 结构保证语言确认)
N30 Progent	L1	摘要揭示 SMT 单调限制 (action space 只能收缩) ——门禁 L1 + confinement L3 风味	⚠ 机制修正 (B6 基于不完整证据判 L1; SMT confine 应记为 L3)
N31 IFC/Fides	L3	动态 taint-tracking 确定性强制 + 形式模型刻画可强制的性质类 ; AgentDojo 评估	✔ 命中 (L3 确认)
N32 A-MemGuard	Lo	共识验证 (多记忆 LLM 路径比较) + 双记忆; ASR 降 95%+ (基准内)	✔ 部分命中 (Lo 判对; 95% 为基准内经验数字)
N33 RARR	L2→Lo	机制=LLM 检索+归因比较+修订 (非机械存在性检查) ——R3 判定路径最弱锚 Lo	⚠ 机制修正 (B6 判 L2 过宽)

验证循环的自我修正价值: N30/N33 两处修正不是"评分者错误", 是**证据包不完整**——摘要级机制细节 (SMT 单调限制、LLM 驱动归因) 补入后, 判级自然修正。这正是 docs/11 §5.6 发现的"证据清洗影响判级"的正面镜像: **证据完整度 → 判级准确度**。

累计验证: PPMF 6/6 (高置信) + 新卡摘要级 4/6 命中 + 2 机制修正 + 1 机制确认 + 1 N/A。 **命中率 10/10 (行为预测层面) + 2 判级修正 (证据驱动)**。

4. 诚实边界

- 1. 预测规则表是 **N=6 归纳的假设**, 不是定律——前向预测 (先预测后看数据) 才构成验证; 当前 PPMF 对照是 post-hoc 演示;
- 2. ASR 数字来自 PPMF 的合成/沙箱评估, 不可外推到生产; 预测对照只用相对模式 (高/低/全杀) 不用绝对值;
- 3. 等级预测的是"锚的失效方式", 不是"攻击是否发生"——同一等级可因威胁模型不同而有不同绝对 ASR;
- 4. 判级本身有可复现性问题 (docs/11: $\kappa \approx 0.8$ 需精化协议) ——预测的可靠性受判级可靠性约束。

5. 相关文件

- 判级协议: reliability/protocol_v2.md (v2.2)
- 语料: reliability/corpus.json (N21–N26 已入 6 个 Agent 安全防护)
- 案例: docs/12 (PPMF 判级 + 自审计)
- 反向验证先例: docs/08 (第四章 4/5 预测命中)

第 18 章

15 · 方向 3 综合：锚定层级预测 Agent 安全防御的成败（论文级草稿骨架）

定位：把 docs/12（PPMF 案例）、docs/13（锚语义）、docs/14（预测协议）收束成一篇实证论文的核心骨架。

状态：骨架 + 数据已齐（P1–P3 完成），待成文。核心主张一句话：

一个 Agent 安全防御能保证什么，由它的锚定层级决定——防御被击败的方式，是其锚失效的方式。

o. 论文标题候选

- *The Anchor Decides: Verification Autonomy Levels Predict the Success of LLM-Agent Security Defenses*
- *Grading the Defenses: Predicting Agent-Security Outcomes with Verification Autonomy Levels*
- 中文立场：延续主论文 *Grading the Graders*——“别人给答案分级，我们给‘分级’分级”；方向 3 是“给防御分级，并预测其成败”。

1. 问题

Agent 安全文献充满防御声称：“我们的护栏/过滤器/门禁是安全的”。但没有一个框架回答**前置问题**：

这个防御的判定锚定在什么真值上？它保证什么（正确性/完备性）？ 部署者无法预先判断该信哪个防御，只能靠事后 benchmark 试错。

本文主张：VAL 锚定层级（主论文 arXiv:2608.19009）是一个**可证伪的预测器**——防御的等级决定其行为模式（ASR 高低、可否绕过、良性是否被误杀），等级预测可以先于实验数据。

2. 方法

1. **判级**：v2.3 协议 (reliability/protocol_v2.md) 对 22 个 Agent 安全防御盲评（70 条语料中的 agentsec 子集，Rater B6，评分者间 $\kappa \approx 0.88$ ）；
2. **锚语义** (docs/13)：intent（意图）/ truth（真值）/ effect（效果）正交标注——判定含义随语义而变；
3. **预测卡** (docs/14)：等级 → 行为预测 + 突破攻击方式，冻结于验证数据之前；
4. **验证**：PPMF 论文数据（高置信）+ arXiv 摘要级证据（N28–N33）。

3. 等级图：22 个防御的 VAL 全景

等级	防御	锚/语义	预测行为
L0	系统提示加固 (N34)、ToolEmu (N42)、A-MemGuard (N32)、SafeAgent 计划层 (N41)、Self-Ask (N24)	无确定性锚 / LLM 自评	强攻击有残余 ASR；无稳定操作点
L1	PPMF 门禁 (N21)、Progent 门 (N30)、白名单 (N35)、rails (N28)、过滤器 (N22/N23/N26)、Perplexity (N36)	设计者规则 / intent	改写/伪造可绕过；粒度粗
L2	Llama Guard 分类器 (N27)、RAG 引用 (N33*)、执行效果核对 (N39)、统计检测 (N19)	客观锚 / truth-effect	分布内有效，OOD/污染失效
L3	IFC/Fides (N31)、CaMeL (N29)、沙箱 (N37)、Progent SMT confine (N30 注解)	可判定系统 / 结构保证	构造性保证：隔离/信息流/限制性质不可绕过 (ODD 内)
L4	智能合约形式验证 (N38)	形式内核	已编码性质域内完备
N/A	AgentDojo (N40)	基准 (R1)	不适用

*N33 RARR 按 R3 修正为 L0 (LLM 判定路径)，表中标注。

4. 核心发现（三条）

1. **Agent 安全的 L3 抬级路径 = 隔离/信息流/限制性质，不是语义安全。**
IFC (污点传播)、CaMeL (数据-控制分离)、沙箱 (执行域限制)、SMT 单调限制 (Progent) ——
这些是**构造性保证** (性质由机制保证，不可绕过)，而语义安全 ("这个动作危险吗") 是开放世界问题，封顶 L2。
与主论文 §4 二分法完全同构：**完备性只对可判定的性质可达。**
2. **授权策略是 L1，除非结构化。** PPMF 门禁、Progent 权限规则、白名单都是设计者规则 (L1) ——
可被策略覆盖外攻击穿透；但当授权附上**结构** (IFC 的格策略、SMT 的单调限制)，升到 L3。
一句话：**"规则"不升层，"机制"才升层。**
3. **LLM 判定器 (L0) 的有效性是基准内经验数字。** ToolEmu (LLM 裁决)、A-MemGuard (共识验证)、
SafeAgent (计划评估) 报告的高拦截率 (如 ASR 降 95%) 是分布内测量——按 VAL，它们无确定性锚，
对抗改写可绕过；其价值在**审计/辅助**而非**保证**。

5. 锚语义的贡献 (docs/13 落点)

- intent 锚 (PPMF 确认、Progent 策略)：授权"该不该做"——失败模式=伪造/社工；
- truth 锚 (分类器、引用核对)：验证"对不对"——失败模式=过时/污染；
- effect 锚 (执行核对、沙箱外副作用)：确认"做没做"——失败模式=部分执行。

- **三类锚不可互换**："已确认"≠"正确"≠"已执行"——这是 PPMF 的 USER_CONFIRMED 教训的框架化。

6. 验证结果

- PPMF 家族：**6/6 命中**（高置信，论文本体数据）；
- 新卡摘要级：4/6 命中 + 2 **机制修正**（N30 Progent SMT confine 补充、N33 RARR 按 R3 降 Lo）+ 1 机制确认 + 1 N/A；
- **验证循环的自我修正**：证据完整度 → 判级准确度（docs/14 §3.3）——预测协议不仅预测，还能通过失配修正判级。

7. 局限（诚实）

1. 判级者为 LLM 评分者（Lo/L1 锚）：一致性测可复现性，效度待真人评分（作者补判）；
2. 验证数据：PPMF 为论文本体（高置信），其余为摘要级（未全文核实 ASR 细节）；
3. 评估均来自合成/基准环境（AgentDojo 等），不可外推生产；
4. N30/N33 的判级修正说明证据完整度敏感——全文级验证是后续必做。

8. 下一步

1. **全文级验证**：抓 N28–N33 全文（arXiv HTML），核实 ASR/utility 数字，补全预测卡；
2. **判级复核**：更新后的证据包 → B7 重评 N30/N33（验证修正是否收敛）；
3. **自研防御实验**：按"L3 在限制性质"的发现，做一个最小 IFC/沙箱式防御原型，跑 AgentDojo 对照（方向 3 的实验贡献）；
4. **成文**：本文骨架 + 主论文关系（作为 v2 的安全域实证章节 或 独立短论文）。

9. 相关文件

- 语料：reliability/corpus.json（70 条，agentsec 22 条）
- 判级：reliability/ratings/rater_b6.json、agreement_report_rater_b5_rater_b6.json
- 预测卡：reliability/prediction_cards.json
- 摘要证据：reliability/validation/abstracts/
- 前置：docs/12（PPMF）、docs/13（锚语义）、docs/14（预测协议）

第六部・收束

第 19 章

评估报告：拆解-验证-纠错架构 vs 直接 LLM 求解

- 日期：**2026-08-18
- 模型：**deepseek-chat（同一模型跑两个对照）
- 测试集：**5 道一元二次函数综合题（含纯代数、含参分类讨论、判别式不等式）
- 系统：**入口分类 → 拆解 → 自查 → 逐个子任务（求解 + SymPy 验证 + 反馈纠错）→ 组合
- 基线：**同样的题直接问 DeepSeek（单次思维链输出，无验证）

一、结论摘要

指标	系统（拆解-验证-纠错）	基线（直接求解）
最终答案正确率	5 / 5	5 / 5
子任务验证状态	11 PASS / 5 UNSURE / 0 FAIL	—（黑箱，无中间验证）
纠错机制触发	未触发（本测试集无 FAIL）	—

诚实结论：在"简单题集"上两者准确率无差异——因为题目都在 LLM 的舒适区内（记忆权重高）。

架构的价值不在简单题更准，而在三点：

- 可验证性：**系统的每个中间结果都有符号引擎盖章，基线的每一步都是黑箱；
- 诚实的失败模式：**裁判不懂时会 UNSURE（弃权）而不是硬答；基线错了就是"自信地错"；
- 难题潜力：**按"记忆 vs 推导"的规律，题越新、越综合，验证-纠错回路的优势才会显现

——
这正是下一阶段要用难题集（MATH/竞赛题）验证的假设。

二、系统结果明细

#	题目	最终答案	标准答案	判定	子任务验证
1	求 $f(x)=x^2-4x+7$ 在 $[1,4]$ 上的最小值及 x	最小值 3, $x=2$	最小值 3, $x=2$	✓	4 PASS / 1 UNSURE
2	$f(x)=x^2-2ax+1$ 在 $[0,2]$ 上最小值为 -1, 求 a	$a=\sqrt{2}$ (含完整分类讨论)	$a=\sqrt{2}$	✓	2 PASS / 1 UNSURE
3	求 $f(x)=x^2-2x+3$ 在 $[0,3]$ 上的最大/最小值	最小 2 ($x=1$), 最大 6 ($x=3$)	同上	✓	4 PASS / 0 UNSURE
4	解 $x^2-5x+6=0$, 求较大根	$x=3$	$x=3$	✓	1 PASS / 1 UNSURE
5	$f(x)=x^2-kx+1$ 与 $y=2x$ 无交点, 求 k	$-4<k<0$	$-4<k<0$	✓	0 PASS / 2 UNSURE

关键亮点 (第 2 题)：含参分类讨论题。子任务 3 用 interval_extreme + subs 模式——LLM 求出 $a=\sqrt{2}$ 后，裁判把 a 代回原函数，在 $[0,2]$ 上重新计算最值，**确认确实等于 -1 才放行**。这正是"拆解→加权匹配→验证"架构对付"LLM 自信地说错"的核心武器。

三、基线结果明细 (直接求解)

#	标准答案	基线输出	判定
1	最小值 3, $x=2$	最小值 3, $x=2$	✓
2	$a=\sqrt{2}$	$a=\sqrt{2}$ (含分类讨论)	✓
3	最小 2, 最大 6	最大 6, 最小 2	✓
4	$x=3$	3	✓
5	$-4<k<0$	$(-4,0)$	✓

基线原始输出保存在 data/baseline_results.json。

四、UNSURE 弃权分析 (诚实记录裁判的盲区)

5 次 UNSURE，全部源于**裁判还缺少"不等式/区间答案"的验证能力**：

题目	弃权子任务	原因	修法
#1	判断驻点在区间内	链式不等式 $1<x<=4$ 无法解析	拆成 $x>=1$ 且 $x<=4$ 两个条件
#2	分情况写最小值表达式	LLM 一个答案里塞了多个值	拆解 Prompt 加强"一子任务一量"约束
#4	选较大根 $\max(2,3)=3$	验证类型不匹配	用 satisfies 验证 $3>2$ 且 3 是根
#5	判别式表达式	答案 $k \in (-4,0)$ 无法解析	新增 verify_inequality 验证类型
#5	解不等式	sympy 不等式比较报错	同上

注意：UNSURE ≠ 答案错误。弃权后系统继续流程（标记待人工确认），最终答案由后续子任务的验证冗余兜住——这也是设计的一部分，不是 bug。

五、已知边界与下一步（已更新 2026-08-18）

已完成的改进：

1. ☒ **verify_inequality 已实现**：SymPy 实数域 solveset 对比解集，支持区间/链式/并集/离散点形态（裁判测试 23→25 用例全过）；
2. ☒ **satisfies 支持链式不等式**（ $1 < x <= 4$ ）；
3. ☒ **equality 支持 表达式=0 形式**——堵住了"拆解错→后续自治地错"的穿透路径；
4. ☒ **反馈纠错（FAIL 路径）首次实战成功**：复测题3 时，LLM 声称 $\Delta=16m+12$ ，裁判当场 FAIL，一次反馈即修正为 $8m-4$ ，最终 $m>1/2$ 正确。

仍待做：

1. **多值答案支持**： $a=1$ ， $b=-2m-2$ ， $c=m^2+2$ 这类列表答案仍会 UNSURE——让 answer 支持列表逐个验证；
2. **拆解完备性强化**：题1 曾漏解 $a=0$ （分类讨论不对称）、回溯后曾出现"常数项 m^2-4 穿透"，需要在自查环节对"每种情况解出的候选是否代回复核"加强约束；
3. **难题集**：把测试集扩展到 MATH/GSM8K 抽样或高考真题，验证"题越新，验证-纠错优势越大"的核心假设；
4. **经验库（P8）**：logs/run_log.jsonl 已在积累，下一步做"题目特征→成功拆解方案"的检索复用。

六、如何复现

```
# 跑系统
python main.py -f data/problems.json

# 跑基线
python baseline.py

# 裁判单元测试（无需 API）
python test_verifier.py
```

所有原始数据：logs/run_log.jsonl（系统）、data/baseline_results.json（基线）。

七、对抗性预测实验（更新）

另做了一轮"预测→实跑→复盘"的对抗性实验（prediction_experiment.py），
意外发现：LLM 生成的 5 个"标准答案"有 3 个是错的（SymPy 审计确认，见 audit_standard_answers.py），
且复盘 LLM 把对的判成错的、把错的判成对的——三层"缺裁判"连锁错误，实证了本架构的核心主

张。

完整报告: [data/prediction_review.md](#)。

Grading the Graders: Verification Autonomy Levels (Lo–L5) for LLM Reasoning

Yajie Yin

Email: 1549080929@qq.com · ORCID: 0009-0001-6168-2530

Preprint: arXiv:2608.19009 (<https://arxiv.org/abs/2608.19009>)

Code & data: https://github.com/1549080929-debug/math_agent

License: CC-BY 4.0

Abstract

Large language models (LLMs) are increasingly paired with "verifiers"—step checkers, self-consistency filters, tool-based fact checkers, and formal proof assistants—that claim to detect the model's errors. Yet the verification literature uses the word *level* to mean at least five different things: verification granularity, concept abstraction, risk tier, system-stack layer, and the epistemic source of the ground truth. We propose **Verification Autonomy Levels (VAL)**, a meta-standard that classifies any verification scheme along a single axis: *where does the verification spec come from, and what does the verdict guarantee?* VAL ranges from Lo (LLM self-declaration; no deterministic anchor) through L2 (objective ground truth; correctness only) to L3/L4 (decidable systems with single-property or domain-level completeness), with L5 shown to be impossible in the unrestricted case. Central to VAL is the **completeness blind spot**: substitution- and sampling-based verifiers can confirm that proposed candidates hold, but cannot prove that no candidate was missed. We further identify a dichotomy the literature has not stated: completeness is reachable only for *formally specifiable* properties, whereas empirical open-world verification (fact-checking, diagnosis) caps at anchored correctness (L2). We document this gap empirically across four domains—symbolic mathematics, behavior monitoring, medical diagnosis, and code generation (the fourth a *reverse validation*: framework predictions stated before evidence)—and in the strongest formal-verification baseline in our survey, whose own authors note the verifier "focuses on the correctness of each step." We further show that the levels of granularity, concept hierarchy, risk, and system stack are orthogonal to VAL, resolving a systematic conflation across 17 surveyed papers. We release a runnable classifier (`val_standard.py`) and the full literature assessment as supplementary material.

Keywords: LLM verification, verification autonomy, completeness, ground truth, trustworthy AI

1. Introduction

Large language models (LLMs) produce fluent, confident, and frequently wrong reasoning. The dominant mitigation is *verification*: attach a second mechanism that checks the model's claims. The 2023–2025 literature has produced a large and rapidly growing body of verification proposals—trained step verifiers [3], self-checking schemas [5], tool-augmented fact checkers [17], graph-structured verification [2], and formal proof assistants [4]—each claiming to catch the errors the model cannot self-report. This proliferation raises a question the literature has not explicitly asked: **what can a given verification scheme actually guarantee, and where does that guarantee come from?**

Classical verification theory explains how to verify a program against a *given* specification—the specification is assumed exogenous and trusted. LLM-based systems introduce a prior problem that this assumption excludes: **whether the specification, the evaluator, and the verification authority are themselves appropriately anchored.** We argue that this prior problem has a natural graded structure (VAL, Sec. 3), and that the LLM-verification literature has been answering it implicitly, without naming it or giving it an operational decision procedure.

We further identify a dichotomy the literature has not stated: completeness is achievable only for *formally specifiable* properties—those with a decidable fragment (mathematical, syntactic, rule-scoped)—whereas empirical open-world verification (fact-checking, diagnosis, legal analysis) has no such fragment and, for the open-world question, caps at anchored correctness (L2); at best it reaches rule-scoped completeness over a formalized sub-fragment, which remains silent about the open world. This is not a limitation of our framework but a property of the domain, and it is a central finding of this paper.

Answering this question is harder than it appears, because the field uses the word *level* to mean at least five different things:

1. **Granularity**—how finely the output is decomposed for checking: claim → sentence → document [7]; atomic step → paragraph [2].
2. **Concept abstraction**—the mathematical sophistication at which verification operates: foundational elements → high-level concepts [8].
3. **Risk/disposition**—what the verdict triggers: safe/unsafe/conditional tiers [10]; output-determinism tiers [12].
4. **System stack**—which component is audited: model → workflow → system [13]; data → base → execute → service [14].
5. **Epistemic anchoring**—the source of the ground truth a verdict rests on, and the strength of the guarantee it delivers.

Axes 1–4 answer *what* to check, *how finely*, *at which layer*, and *what to do with the result*. None answers the question that determines whether a verifier can ever be trusted: **on what ground truth does the verdict rest, and does it guarantee correctness, completeness, or neither?**

We argue that the fifth axis is the one that matters for trust, that it has a natural six-level structure, and that the literature's conflation of the five axes has obscured it. Three observations motivate the framework.

First, the anchor is what fails. In our four-domain study (Sec. 5), every verification failure we catalogued was traceable to the anchor, not to the judge: a well-calibrated verifier (42/42 unit cases) was fed wrong subproblems by the LLM it was checking (decomposition contamination), and never saw the final answer it should have checked (combination tampering), or was asked to verify a condition declared by the very model under test (trust recursion). A perfectly calibrated instrument pointed at the wrong object is not a bug; it is a specification problem.

Second, correctness is not completeness. The most common verifiers—substitution, sampling, statistical thresholds—can confirm that *proposed* candidates hold, but cannot prove that *no* candidate was missed. We call this the **completeness blind spot** and document it empirically in symbolic mathematics (a missed root that substitution cannot see), in behavior monitoring (a covert-execution attack that leaves no trace in the output layer), and in medical diagnosis (confident conclusions from insufficient evidence). The blind spot is not a tuning failure; it is a property of the verification paradigm (Sec. 4).

Third, the strongest existing verification concedes the same point. The most rigorous scheme in our survey—Lean 4 step-level formal verification [4]—states that its formal verifier "focuses on the correctness of each step": the kernel checks proofs of LLM-generated statements, but the statements themselves, and the case coverage they encode, remain LLM declarations. The blind spot does not disappear at the top of the ladder; it is pushed to the statement layer.

Contributions. We make four:

1. **Verification Autonomy Levels (VAL)**, a six-level epistemic taxonomy (L0–L5) classifying any verification scheme by its anchor source and guarantee, together with a deterministic decision procedure (Sec. 3) and a runnable classifier (`val_standard.py`).
2. **A disambiguation of five confounded "level" axes**, showing that granularity, concept abstraction, risk, and system-stack are orthogonal to the VAL axis, and locating 17 representative papers in the resulting space (Sec. 2).
3. **A formal and empirical treatment of the completeness blind spot**, including a statement of why universal completeness is impossible in the unrestricted case (Sec. 4.5) and why completeness is always relative to an operational design domain (Sec. 4).
4. **Four cross-domain case studies**—symbolic mathematics, behavior monitoring, medical diagnosis, and code generation, the last a *reverse validation* with predictions stated before evidence (Sec. 5)—that exercise the framework end-to-end and report honest negative results.

Why this matters. For a deployer, VAL is a pre-purchase checklist: ask *where the spec comes from* before trusting any verification claim, and know that an L2 verdict is a correctness probe, not a completeness guarantee. For a researcher, VAL separates the five questions hidden inside "hierarchical verification," preventing category errors such as claiming that finer granularity

implies stronger grounding. For a reviewer, VAL supplies a vocabulary for interrogating any claim of the form "our system verifies X": *at what level, within what ODD, with what abstention behavior?*

Positioning. We present VAL as a conceptual contribution. The four case studies of Sec. 5 are exercises of the framework, not claims of empirical superiority over baselines—indeed, two report clean negative results (the verification architecture does not improve accuracy in mathematics or code generation). The empirical content is intentionally modest; the claim we defend is structural: that one axis—the source of the verification spec—is measurable, ordinal, and currently untheorized in the LLM-verification literature.

2. Related Work: Five Axes, One Word

We reviewed 17 representative papers spanning what the literature calls "layered" or "hierarchical" verification, and classified each along five axes. The full assessment—per-paper evidence, confidence levels, and a versioned re-verification trail—is released as supplementary material (docs/07); here we report the axis structure and the anchor-axis classifications used throughout.

2.1 Granularity axis: *how fine*

Graph of Verification [2] adapts verification granularity from atomic steps (formal tasks) to whole paragraphs (informal narratives) via a "node block" architecture, trading precision against robustness. Factcheck-GPT [7] annotates factuality at three granularities—claim, sentence, document—with a GPT-4-based annotation scheme and a gold-labeled benchmark. Dr. V [9] decomposes video-hallucination diagnosis into perceptual, temporal, and cognitive levels, grounding the first two in 10k spatial-temporal gold annotations. These are ladders of *decomposition fineness*. They say nothing about the anchor: the same granularity ladder can be implemented with LLM judgment (Lo) or with objective ground truth (L2).

2.2 Concept axis: *how abstract*

Hierarchical Attention [8] regularizes LLM attention toward a five-level hierarchy of mathematical concepts to improve proof generation in formal theorem proving (miniF2F, ProofNet). Its proofs are checked by a formal kernel—an L4 anchor—but the paper's contribution is a generation-time regularizer, and its "levels" are concept-abstraction levels, orthogonal to the anchor.

2.3 Risk/disposition axis: *what to do*

A safety-response framework [10] classifies inputs into four disposition tiers (Safe, Unsafe, Conditionally Safe, Focused Attention) via a supervised fine-tuned classifier, reporting 99.3% recall. LLM Output Drift [12] tiers models by output determinism for risk-adapted deployment in finance, combining consistency measurement with invariant checking and SEC-citation validation. M³-SafetyBench [11] evaluates models across content- and functional-safety dimensions with a 170k-item benchmark. These are ladders of *consequence*—what the verdict should trigger. A "four-tier

safety classifier" and a "three-level fact-checking pipeline" are frequently cited together, yet one is a disposition ladder and the other a granularity ladder; neither is an epistemic ladder.

2.4 System-stack axis: *which layer*

Standard Benchmarks Fail [13] proposes stress-testing financial LLM agents at model, workflow, and system layers, arguing that standard accuracy benchmarks "provide an illusion of reliability." Prompting Frameworks Survey [14] organizes prompting tooling into data, base, execute, and service layers. These are ladders of *audit scope*. [13] is notable as the one paper in our survey that makes completeness of evaluation coverage an explicit thesis.

2.5 Anchor axis: *on what ground truth (this paper)*

Classified by the procedure of Sec. 3.3:

- **L0** — SelfCheck [5]: four-stage regenerate-and-compare, explicitly "without resorting to external resources"; the checker itself scores 66.7% verification accuracy and the authors concede "the checks are themselves imperfect." LM² [6]: a verifier language model, fine-tuned on GPT-4 annotations and coordinated with the decomposer and solver via policy learning.
- **L0/L1 + tools** — VerifiAgent [1]: meta-verification of completeness and consistency is LLM-based with some deterministic rules (L0/L1); tool-based adaptive verification delegates factual and computational checks to a Python interpreter, a search engine, and symbolic computation—deterministic execution serving as *silver ground truth*, a reference weaker than gold (L1), as the authors confirm (personal communication).
- **L0/L1** — Factcheck-GPT [7]: runtime verdicts delegated to a GPT-4-based annotation scheme; the gold labels in its benchmark anchor the benchmark, not the runtime verdict (corrected from L1/L2 at abstract-level re-verification, Sec. 2.6).
- **L1/L2** — FACTOOL [17]: tools (Google Search, Google Scholar, code interpreters) gather evidence, but the final factuality verdict is LLM reasoning over that evidence. LLM Output Drift [12]: consistency measurement plus invariant checking with SEC-citation validation.
- **L2** — DiVERSE [3]: a DeBERTa-v3 step verifier trained on step labels derived by matching against ground-truth answers. M³-SafetyBench [11]: a gold-labeled evaluation benchmark. Dr. V [9]: hallucination diagnosis grounded in gold spatial-temporal annotations.
- **L3/L4** — Hierarchical Attention [8]: proofs checked by a formal kernel. Safe [4]: Lean 4 step verification—the kernel is L4, but the theorem statements are LLM-generated, i.e., L0 at the statement layer.

Axis	Question it answers	Representative papers
Anchor (VAL)	What ground truth does the verdict rest on, with what guarantee?	SelfCheck Lo; LM ² Lo; VerifiAgent Lo/L1+tools; DiVERSE L2; FACTOOL L1/L2; Safe L4+Lo
Granularity	How finely is the output decomposed for checking?	GoV, Factcheck-GPT, Dr. V
Concept	At what abstraction level does verification operate?	Hierarchical Attention
Risk/Disposition	What response does the verdict trigger?	SafetyResponse, OutputDrift, M ³ -SafetyBench
System stack	Which component of the system is audited?	BenchmarksFail, PromptingSurvey

Observation. Across all 17 papers, none formalizes the anchor as a ladder, none treats the completeness of a verification scheme itself as an object of study, and the single most explicit acknowledgment of the gap comes from the strongest formal baseline [4]. The conflation is not harmless: it lets "layered verification" papers inherit each other's credibility across axes that do not entail one another. The rest of this paper develops the anchor axis.

2.6 Adjacent fields: formal methods, risk frameworks, epistemology

Readers from program analysis and formal methods will recognize the correctness/completeness distinction as decades old: Hoare logic distinguishes partial from total correctness; abstract interpretation formalizes the soundness and completeness of abstractions; type theory defines completeness via logical relations. We agree, and state precisely what VAL adds.

In traditional program verification the division of labor is fixed: a human writes the specification, and the verifier proves the program against it. The specification is ground truth *by construction*—it does not depend on the program under analysis. In LLM verification this division collapses: the "program" (the model's reasoning) and the "specification" (what should be checked) are both produced by LLMs. The anchor-source axis—whether the spec is LLM-declared (Lo), derived from the problem text (L1), grounded in objective truth (L2), or encoded in a decidable system (L3/L4)—is the new object of study. Hoare logic tells us how to verify a program against a given specification; VAL asks *who gave the specification and whether it can be trusted*. The former is soundness theory; the latter is an accountability taxonomy for a class of artifacts in which the spec is no longer exogenous.

The distinction from risk-management and maturity frameworks (NIST AI RMF, CMMI) is different: those are disposition ladders—they classify what to do about risk (Sec. 2.3's risk axis), not the epistemic status of a verdict. A system can be process-mature and still run Lo verification; maturity and anchor strength are orthogonal. Epistemic philosophy (reliabilism, justification theory) and LLM interpretability frameworks [16] ask whether a belief is *justified* or how it was formed; VAL does not adjudicate truth—it classifies the structural source of a claim's checkability. Adjacent surveys of trustworthy evaluation [15] map this territory without isolating the anchor axis.

We do not claim to resolve epistemology; we claim that one axis—spec source—is ordinal, operationally decidable (Sec. 3.3), and currently absent from the LLM-verification literature.

Closest in the security-assurance tradition are the Common Criteria Evaluation Assurance Levels (EAL1–EAL7, grading the assurance of the evaluation *process*), DO-178C Design Assurance Levels (DAL A–E, grading by failure *consequence*), and the trusted computing base (TCB, delimiting which components must be *trusted*). These grade process maturity, consequence, or trust scope; none grades the *source of the verification specification*. VAL's axis is the fourth question: given a verification claim, who declared what is to be verified, and does that declaration carry an independent anchor? This is the question the security-assurance ladders do not answer, and it is the one that matters most when the verifier and the verified share a generator (Sec. 3.2).

Auditability of this assessment. The literature assessment in this paper is itself versioned and auditable (supplementary material, docs/o7): initial judgments (V1.0), abstract-level verification (V1.2), and full-text verification of the six anchor-axis papers (V1.3) are recorded as a change log. One judgment was corrected at abstract-level re-verification (V1.2): a paper initially classified L1/L2 was found to delegate its verdict to a GPT-4-based judge, L0/L1; full-text verification of the six anchor-axis papers (V1.3) subsequently upheld every remaining judgment. We report this not as an embarrassment but as the framework applied to itself: assessments are claims, claims require anchors, and the correction trail is the anchor.

3. The VAL Framework

3.1 Levels

Level	Anchor source	Guarantee	Completeness	Driving analogy
L0	LLM self-declaration	none	none	full manual (the driver "confidently errs")
L1	deterministic rules derived from the problem text/code	deterministic matching	no	lane keeping (single-function assist)
L2	objective ground truth / gold labels / <i>independent</i> oracle (not a sibling generator)	correctness	no	partial automation (human must take over)
L3	definitional/provable (property encoded in a decidable system)	single-property completeness	yes (within ODD)	conditional automation (system owns liability in ODD)
L4	domain-level proof systems	domain-wide completeness	yes (within domain)	high automation (no takeover in ODD)
L5	universal completeness	any property	impossible (unrestricted)	full automation—does not exist

Each level is defined by two properties: the **anchor source** (who or what supplies the ground truth the verdict rests on) and the **guarantee** (what a PASS commits to). Three consequences follow.

First, *the anchor, not the judge, determines the level*. A perfectly calibrated verifier at L2 is still L2: substitution checking that a candidate satisfies an equation proves the candidate holds; it says nothing about candidates not proposed. Improving the *implementation* of an L2 check (denser sampling, a better threshold) moves the system horizontally within L2; only changing the anchor source moves it vertically.

Second, *the guarantee degrades downward but not upward*. A judge built for L3 (e.g., a symbolic solver) can be *used* at L2 (substitution only)—we call this a *usage-degraded* anchor, and the classifier flags it as upgradeable. A judge built for L2 cannot be promoted to L3 by more data: no amount of sampling closes a completeness gap (Sec. 4). This asymmetry is why "more data" is not a level-raising operation.

Third, *every level has a characteristic abstention behavior, and it is diagnostic*. Lo systems deny error—they "confidently err." L1/L2 systems are silent about what they did not check. L3/L4 systems return a decidable abstention when a property falls outside their ODD (Safe's "failed formalization" state is exactly this [4]). L5 does not exist. In our experience, a verifier's abstention behavior is a faster diagnostic of its level than any benchmark score.

 Figure 1: The VAL ladder and four orthogonal axes

Figure 1. The VAL ladder (vertical) and four orthogonal axes (horizontal) that the literature conflates with it. Granularity, concept, risk, and system-stack answer *what / how fine / what response / which layer*; VAL answers *on what ground truth, with what guarantee*.

3.2 The three decisive questions

Any verification scheme is classified by answering three questions in order:

- **Q1 (spec source).** Who declares the verification condition—the thing the verdict is about? The answer is one of: (a) the LLM under test itself ("I checked it," "this is verified"); (b) a deterministic rule derived from the problem or code text (regex, parser, schema); (c) an objective source independent of both the problem and the generative mechanism under test (gold answer, measured value, execution result); (d) a property encoded in a decidable system (a symbolic solver, a type system, a decision rule with validated thresholds).

Anchor independence. "Objective" in L2 does not mean merely "external to the model under test." An anchor qualifies only if it is *independent of the generative mechanism being verified*—its error mechanism must not be correlated with the verified system's. An answer produced by another LLM is external but not independent: it shares the error-generating mechanism, and is Lo masked as L2. Execution results, symbolic simplification, and physical measurements qualify because their "error mechanism" is the artifact itself, not a correlated generator. This is the operational form of trust-recursion termination (Sec. 6.1): the anchor chain stops at definitional or conventional anchors, never at a sibling generator. Objective anchors are not all equal; they split by what they certify. A *gold-label anchor* (gold ground truth)—a known-correct reference—certifies the answer itself and is L2. A *silver anchor* (silver ground truth)—deterministic re-computation that serves as a reference—certifies the computation, not the answer, and sits at the L1/L2 boundary: objective

about whether the computation reproduces a result, silent about whether that result is correct. The term *silver ground truth* is due to the authors of VerifiAgent, whose tool component is exactly this—it re-derives with a Python interpreter or symbolic engine and compares—which is why the author places it at L1 rather than L2 (Sec. 2.5).

Anchor semantics. Gold anchors further split by *what they certify*, and the distinction changes what a verdict means. An **intent anchor** certifies a recorded decision event—that a user or platform authorized an action toward a target (a confirmation record, an approval). A **truth anchor** certifies a fact—that an answer matches objective ground truth (gold labels, measured values, a verified equation). An **effect anchor** certifies an execution outcome—that an action actually occurred or succeeded (server-side state change, test execution). The same L2 level can host all three, but they are not interchangeable: an intent anchor authorizes; it says nothing about whether the outcome was correct. Agent-security practice makes this explicit—a provenance-based gate treats a confirmation as an intent anchor and refuses to let it certify effects (Sec. 4.3, [18]). Conflating the three is a category error of the same family VAL prevents: it mistakes "authorized" for "correct" or "executed."

- **Q2 (guarantee).** What does a PASS commit to? *Correctness*—"every proposed candidate satisfies the condition"—or *completeness*—"no candidate was missed." The distinction is the entire substance of Sec. 4; most deployed verifiers offer the former while being read as the latter.

- **Q3 (scope).** If the guarantee is completeness, over what domain does it hold: a single property (this equation's solution set), a whole class (all programs' memory safety), or claimed universality?

3.3 Decision procedure

The classification is deterministic: given the answers to Q1–Q3, the level is the first rule that fires.

1. completeness + universal scope → L5 (rejected: impossible, unrestricted)
2. completeness + domain scope → L4
3. completeness + single-property scope → L3
4. correctness + gold-label anchor (known-correct reference) → L2
(decidable anchor used only for correctness → flag "usage-degraded, upgradeable")
5. correctness + silver anchor (deterministic re-computation, no gold reference)
→ L1/L2 boundary, classified L1
(objective about the computation, silent about the answer; VerifiAgent's tool)
6. correctness + problem-derived rule → L1
7. otherwise (LLM-declared / no anchor) → L0

The procedure is implemented in `val_standard.py` (11 self-tests) and documented in `docs/06`. Its determinism is deliberate: it is a *standard*, so two raters applying it to the same scheme must obtain the same level—the only legitimate disagreements concern the Q1/Q2 answers, not the mapping. We stress-tested this claim directly (`docs/11`): the Q1/Q2 judgments are reproducible across raters only once the protocol's boundary rules are explicit—three independent blind raters reach $\kappa \approx 0.8$ on 48–54 schemes under the refined protocol, versus 50% agreement under the underspecified v1. The mapping is deterministic by construction; input reproducibility is an empirical property the protocol earns, which is why the protocol ships frozen with an evidence pack. The unit of classification is the verification *mechanism* (component), not the monolithic

system: a mixed architecture is classified per-component, so a label such as "Lo/L1" in our survey denotes two components at different levels, not an indeterminate scalar. For a mixed system, the system-level level is *claim-relative*: a system is L4 with respect to the claim "the theorem is proven" and Lo with respect to the claim "the answer is correct." There is no single scalar system level; VAL composes per claim and per component.

3.4 Operational Design Domain (ODD)

Borrowing from autonomous-driving regulation, an L3/L4 guarantee holds only within an **ODD**—here, the *decidable domain* in which the property can be encoded and the verdict computed. An ODD must be a *pre-declared syntactic predicate* over inputs, not a post-hoc description of success. Examples of the correct form: `solveset` over "polynomial equations of degree ≤ 4 with real coefficients" (checkable before solving); an Alvarado score over "a record with all eight MANTRELS fields present and numeric/boolean"; a borrow checker over "a term parseable in the surface grammar of the type language." These syntactic ODDs are narrow by construction—beyond degree 4 there is no closed form, and a rule-scoped ODD is complete over its own inputs but silent about the open world—and the framework's honesty is that this narrowness is made explicit rather than hidden.

Two corollaries. First, **completeness is never absolute; it is relative to an ODD**. A verifier that is complete inside its ODD is, by construction, silent about anything outside it—the completeness claim is only as strong as the ODD is honestly specified. Second, **raising a level means enlarging the ODD, not intensifying sampling**. The L2→L3 move for "find all solutions" is achieved by re-encoding the task as solution-set equality, making the property decidable; no sampling density achieves this. The L3→L4 move is achieved by covering a whole class of properties under one decidable system.

ODD auditability. ODD honesty is an *audit target*, not an assumption VAL silently guarantees. An ODD claim has its own anchoring ladder—self-declared (Lo), tool-documented (L1), empirically tested (L2), formally characterized (L3)—and a completeness claim is only as strong as its ODD's own anchor. An ODD is meaningful only if it is a *pre-declared decidable predicate* over inputs (syntax, grammar, input shape): "polynomial equations of degree $\leq d$ over the reals" is checkable before invoking the solver, whereas "whatever the solver happens to solve" is vacuous—completeness then holds for "the cases we completely verified," which is circular. The engineering test of ODD honesty is a boundary probe: adversarial instances just outside the claimed ODD must break the completeness claim.

3.5 What VAL is not

Three non-claims, to preempt misreading:

1. **VAL is not a reliability measure.** A level states the epistemic status of an anchor; it does not state the probability that a verdict is correct. An L2 verifier can be more *accurate* than an L3 verifier on in-ODD cases (our own 42/42 unit tests span L2 and L3); level and reliability are orthogonal.

2. **VAL is not a utility claim.** We do not argue that higher is always better, nor that L3/L4 should be pursued everywhere. Sec. 6 governs *when* climbing is worth its cost; for low-stakes tasks an honest L1/L2 check may be the correct engineering choice.
 3. **VAL does not certify the anchor.** An L3 anchor guarantees that, *given the encoded property*, the verdict is decidable and complete. It does not guarantee that the property is the right one—that question is itself an anchor question, one level up (Sec. 6). This is the framework's own form of the regress it describes.
 4. **VAL's completeness ladder applies only to formalizable properties.** L3/L4 completeness requires a decidable fragment—mathematical, syntactic, or rule-scoped. Empirical open-world propositions (fact-checking, diagnosis, legal analysis) have no such fragment and therefore cap at L2 (anchored correctness), never L3/L4. VAL is not a universal completeness standard: its L3/L4 are empty for open-world verification, and its contribution there is the L0–L2 distinction plus honesty, not completeness. For empirical domains, L0–L2 are therefore not merely "the lower three levels" but a complete classification in their own right: L0 (self-claim), L1 (rule matching), L2 (anchored correctness)—each with a distinct failure mode (confident error; format-correct but content-wrong; tested but incomplete).
-

4. The Completeness Blind Spot

The central theoretical claim of this paper:

Substitution- and sampling-based verification (L2) can prove that proposed candidates hold; it cannot prove that no candidate was missed. Completeness is achievable only by re-encoding the property into a decidable system (L3/L4)—or not at all.

We state the above as a *definitional* fact—it follows from what substitution and sampling are. Its significance is not the fact itself but the operational gap it exposes: the correctness/completeness distinction is known in verification theory, yet the LLM-verification literature provides no operational procedure for marking which guarantee a given scheme delivers. VAL's contribution is to make that distinction executable (a decision procedure and tools), not to claim novelty for the distinction. The open empirical questions (when does this incompleteness produce actual errors, and what is the marginal L2→L3 gain) are left to future work rather than answered here.

4.1 Correctness is not completeness

Let $P(x)$ be the property of interest (e.g., " $a = x$ makes the maximum of f on $[0, 2]$ equal to 3"), and let C be the set of candidates the system proposes. A correctness verifier establishes

$\forall c \in C : P(c)$ — every proposed candidate satisfies the property.

A completeness verifier establishes the converse direction:

$\forall x : P(x) \Rightarrow x \in C$ — every value satisfying the property was proposed.

These are logically independent. The first is easy for a verifier and says nothing about the second; the second is what users implicitly assume. A verifier that performs the first while being read as the second is the mechanism behind systems that are "confident but wrong" yet still report "verified."

4.2 Substitution blindness

Substitution checking evaluates P on proposed candidates only. The missed candidate is invisible by construction: the verifier never receives it. In our math experiments, the system proposed $a = 2$ for a problem whose true answer is $\{0, 2\}$; substitution returned PASS, and the omitted $a = 0$ was undetectable. This is not an implementation deficiency; the verifier literally cannot ask a question it was not given. Re-encoding the same problem as solution-set equality—`solveset` over the equation, compared set-wise with the claimed answer—makes the property decidable and converts the blindness into a machine-checkable FAIL (claimed "2" vs. true $\{0, 2\}$). The L2→L3 move is a re-encoding, not an intensification.

4.3 The sampling ceiling

Sampling-based verification evaluates P at a finite set of points. It can falsify (find a counterexample in the sample) but cannot certify (a gap between samples is undetectable by construction). In our 20-problem run with sampling-based final-parameter verification, all three false passes shared one cause: the missed candidate or interior void was not in the sample. Denser sampling narrows the ceiling's shadow but never removes it; the ceiling is the defining property of the L2 paradigm, not a tuning parameter.

The blind spot is not confined to mathematics. In agent security, a provenance-based authorization gate (PPMF [18]) blocks every *evaluated* unauthorized tool call (0.000 attack-success rate) while platform metadata stays intact, yet a 10% rate of forged confirmation records yields 0.088 ASR. The gate is a correctness probe over the evaluated attack distribution: it cannot certify that no attack path was missed—the un-enumerated forgery is invisible to it by construction, exactly as the unsampled candidate is to substitution. The completeness question in security ("is there an unauthorized action we failed to enumerate?") is formally open in the same way the sampling ceiling is open in math.

4.4 The statement layer: where the blind spot hides

Every verification scheme draws a line between what is checked and what is assumed, and the completeness blind spot always lives on the assumed side. For L2 schemes, the assumption is "the proposed candidate set is the right one." For L3/L4 schemes, the assumption moves up but does not vanish: the kernel checks proofs, but the theorem statement—and the case-split coverage it encodes—is a declaration. The strongest formal baseline in our survey illustrates the boundary: Safe's verifier checks each step's proof correctness—complete *with respect to the stated theorem*—while the theorem statements, and the case-split coverage they encode, are LLM-generated and unchecked [4]. We cite this as an architectural fact about where the completeness question lives, not as the authors admitting incompleteness. The same pattern recurs in our own system: when the

verification condition was declared by the LLM under test, the judge faithfully verified a possibly wrong condition—trust recursion pushed one level up, not resolved.

4.5 Why universal completeness is impossible

The L5 claim is a single verifier complete over *arbitrary* properties and inputs. Its impossibility does not rest on any single theorem; it rests on the fact that "arbitrary properties" bundles three classes, each unverifiable-universally for a different reason: (i) program semantic properties—undecidable by Rice's theorem; (ii) empirical propositions about the world—whose truth is not a formal decidable question at all (open-world, natural-language); (iii) subjective properties with no ground truth. Empirical propositions cannot be re-encoded into a complete decidable fragment, because their truth depends on an open world; the best they achieve is anchored correctness against evidence (L2), or rule-scoped completeness over a formalized sub-fragment (L3 within the rule's inputs). Notably, proof checking—verifying a *given* proof against derivation rules—is decidable (it is the L4 kernel); this is precisely why completeness is achievable for the restricted decidable class (L3/L4) but not for the unrestricted one (L5). The consequence is architectural, not pessimistic: completeness is available locally (within an ODD, at L3/L4) and unavailable globally (L5); the engineering question is always *which ODD*, never *whether*.

4.6 Summary

The blind spot has three stable properties: (i) it is invisible to the verifier that suffers it; (ii) it is not fixed by more data or denser sampling; (iii) it can be moved up the ladder (from candidate sets to theorem statements) but not eliminated. The framework's prescription follows directly: when a property is encodable, raise the anchor (L3/L4); when it is not, the verifier is a correctness probe by necessity and should be labeled and deployed as one.

5. Empirical Case Studies

We exercised the framework across four domains—symbolic mathematics, behavior monitoring, medical diagnosis, and code generation. Full archives: RESULTS.md (math), behavior/REPORT.md, medical/REPORT.md, docs/08 (code). Each study followed the same protocol: audit first (build a labeled test bed), measure the value window (baseline comparison), classify failure modes, then test the judge itself. We report honest negative results where they occurred.

5.1 Symbolic mathematics

A decompose–solve–verify–combine agent was built on a deterministic SymPy verifier with nine verification types, calibrated to **42/42** unit cases (`test_verifier.py`). The types span the anchor ladder: correctness-only types (root, extreme, equality, satisfies, final_parameter_set) are L2; completeness-bearing types (interval_extreme, inequality, solution_set, answer_type) are L3; ANCHOR_LEVELS in `verifier.py` records each type's level.

The headline result is a clean negative one: on a 20-problem test set, the full architecture scored **16/15/13** across three runs, while a raw-LLM baseline scored **20/20** in all three. Accuracy was a random variable dominated by the LLM's decomposition; the verification stack did not improve it and sometimes hurt it. We do not claim otherwise.

The architecture's measurable value was *error reportability*, not accuracy. Three episodes illustrate it. First, a feedback loop caught a discriminant error (claimed $\Delta = 16m+12$, true $8m-4$) and corrected it in one iteration. Second, final-parameter verification caught a tampered answer (claimed $[-1,1]$, true $[0,1]$) and corrected it—a failure mode invisible to per-step verification. Third, and most relevant to this paper, the L3 `solution_set` verifier catches missed solutions that the L2 substitution verifier cannot see, by construction (Sec. 4.2).

The value window for *accuracy* in this domain is empty (the LLM is too strong on in-distribution problems); the value window for *completeness* is exactly the L3 re-encoding we describe. The two windows must not be conflated; preventing that conflation is the framework's purpose.

5.2 Behavior monitoring

Chapter 2 monitored LLM behavior under prompt injection using a statistical deviation detector (L2 by construction: its thresholds are learned from a baseline distribution, so no completeness is possible). Five iterative rounds were required to reach a deployable operating point. A global-statistics detector's apparent success (holdout AUC 0.82) collapsed to a **92% false-positive rate** under realistic input diversity; probe-conditioning (comparing each response only to its own probe's baseline) cut FPR to **30%** while preserving strong-attack TPR 1.0; character n-grams recovered weak-attack TPR at the cost of doubling FPR; real semantic embeddings (a local sentence encoder) recovered weak-attack TPR to 0.75 with FPR unchanged.

The residual failure is the paper's example of a structural L2 ceiling: "covert execution" attacks—the model executes an injected instruction but leaves no trace in its output—were detected at TPR 0.60 across all feature sets. The property cannot be encoded into the *output-layer* anchor. Whether internal-activation anchors (mechanistic interpretability, activation monitoring) could detect it is an open question this study does not address; our claim is bounded to output-layer statistical detection, not to verification in general.

5.3 Medical diagnosis

Chapter 3 studied LLM diagnostic reasoning as a research object (fictional cases; not a diagnostic tool). The raw LLM was near-perfect on 10 textbook cases (10/10) and acceptable on 8/10 hard cases; the two failures shared a single mode: a confident conclusion from insufficient evidence (diagnosis at confidence 0.7 with no lab results). Confidence calibration and information completeness were decoupled.

A deterministic post-hoc judge—an information-completeness rule (L1/L2: a hand-written rule anchored in the case's objective lab fields; the L3 upgrade is documented below)—flagged **3/3 true over-confidence failures at zero false positives** after calibration, *including one (#105) that*

human review had missed. The judge's value here is discovery, not re-checking: it found a failure the human annotator had labeled correct. This is the empirical analogue of Sec. 4.4: the judge is only as good as its anchor, and a cheap rule anchored in objective fields outperformed human review at one narrow but critical task—catching "conclusion beyond evidence."

A documented L2→L3 upgrade. To exercise the ladder's prescription (Sec. 6.3), we replaced the hand-written information-completeness heuristic with an imported, validated clinical decision rule—the Alvarado score (MANTRELS) for suspected appendicitis—as the judge's anchor. The rule's eight objective inputs are parsed by deterministic keyword/regex extraction from the case text (not by an LLM); when inputs are missing, the rule returns a reachable score *range* rather than a fabricated point value. This semantics does real work: for the over-confident case #103 the rule returns range 3–8 (indeterminate) and refuses to support a definitive diagnosis, re-flagging the failure through its own decision boundary; for case #109 the range is 7–9 (high risk) *despite* a missing neutrophil count, because the lower bound already clears the threshold—so the rule does not over-flag a case with genuinely high risk. The flagged set is unchanged (3/3); what changed is the anchor: a hand-written threshold (L1/L2) replaced by a validated rule with a decidable boundary and an honest missing-data policy (L3, within the appendicitis ODD).

5.4 Cross-domain synthesis

Across the four studies the pattern is identical. LLMs are too strong for accuracy windows to be non-empty on problems near their training distribution (math: baseline 20/20; medical: textbook 10/10; code: HumanEval 164/164 and six hand-written hard problems all solved in the bare condition); the non-empty windows are all *verification* windows: error reportability (math), ODD-bounded detection (behavior), over-confidence catching (medical), and L3-anchor repair (code). The framework states this: an L2 judge adds reportability, not accuracy; an L3 judge adds completeness within its ODD; nothing adds completeness outside it.

5.5 Code generation (Chapter 4): reverse validation

The fourth study reverses the direction of evidence. Chapters 1–3 induced the framework from data; Chapter 4 states predictions *before* gathering evidence (the protocol, with dates, is in the supplementary material), then tests them against external literature and one controlled experiment. Four of five expectations were supported:

- **P1 (test-oracle ceiling).** Test-based selection improves ranking (AlphaCode; CodeT; LEVER) but is bounded by test quality—LEVER's authors note that obtaining test cases is itself the hard step.
- **P2 (self-review: utility without guarantee).** Self-repair helps but is not a silver bullet (Olausson et al.); without external feedback, self-correction frequently fails (Huang et al.); Self-Refine's gains are task-dependent.
- **P4 (trust recursion at the spec layer).** LLM-generated tests are themselves suspect: Meta's production tool TestGen-LLM runs its generated tests through verification filters before acceptance.

- **P5 (execution feedback dominates self-review).** Execution-feedback agents (Reflexion, CodeAct, Self-Debugging) consistently outperform feedback-free generation, consistent with L2/L3 anchors beating L0 self-declaration.

The open prediction, **P3 (type information as the highest-ROI L3 anchor)**, was tested experimentally. We stripped type hints from HumanEval prompts and generated solutions under three conditions: bare (no types), typed, and typed-plus-type-checker-feedback (one mypy round). The result is a clean empty window: **164/164 HumanEval problems and all six hand-written, trap-laden novel problems were solved by the bare condition**—the model leaves no accuracy headroom for type information to improve. The single failure in the typed condition (a novel string-escaping problem) carried a mypy error that the test suite did not catch; the type-checker-feedback condition repaired it in one round.

Evaluation protocol and caveat. The 164/164 figure is single-sample pass@1 at temperature 0.2 (low-temperature sampling, not greedy decoding), checked against HumanEval's own tests. Three caveats bound its interpretation. First, HumanEval's test suites are weak—most problems carry only 3–5 assertions and little adversarial coverage—so "passes the tests" has a nontrivial false-positive rate. Second, the figure is not comparable to published pass@1 numbers, which use varied decoding and test harnesses. Third, it is reported only to establish that the accuracy window is empty *on this suite*; it is not a claim about general code ability. Full protocol and per-problem outputs are in the repository (`chapter4/p3_scan.json`).

The significance of Chapter 4 is twofold, and weaker than a first reading suggests. First, the framework's central empirical generalization—*LLMs are too strong for accuracy windows to be non-empty on problems near their training distribution*—is now observed in a fourth domain, including problems we designed to defeat it. Second, the "reverse validation" is a *weak* one: the four literature-supported predictions are framework-implied expectations checked against existing work, not novel a-priori hypotheses, and the fifth (P3) was *inconclusive*—the test set left no headroom, which is a failed experiment design, not evidence for the framework. We report this honestly: P3 remains open, and the reverse-validation claim is correspondingly modest.

6. Discussion

6.1 Trust recursion and its termination

Every judge needs a judge; an ungrounded chain is *trust recursion*. The framework shows where recursion can terminate: at the kernel of a decidable system (L4), at a definitional anchor (L3), or at a conventional primary standard (metrology, where calibration chains terminate at definitionally fixed units). L5 is the claim that recursion can be terminated universally, which Rice's theorem rules out for program-semantic properties (Sec. 4.5). The practical reading: the question is never "who verifies the verifier?" in the abstract, but "which decidable kernel, definitional anchor, or conventional standard terminates *this particular* chain?" In our own system, the chain terminates at `solveset` for solution-set checks, at the case data for the medical judge, and at the probe baselines for the behavior detector—three different termination points, each honest about what it covers.

6.2 Division of labor

VAL implies a canonical architecture: the LLM translates (decomposes, renders, declares), deterministic code judges within its ODD, and a human anchors the spec and audits the judge. Each component is best at what the others cannot do: the LLM cannot prove, the judge cannot generalize, the human cannot scale. Failures occur when the roles are crossed—when the LLM declares its own verification spec (Lo), when the judge is asked to certify outside its ODD, or when a human is expected to re-derive what a deterministic system should decide. In agent settings the anchoring role is increasingly *institutionalized rather than personal*: the anchor is platform-maintained metadata—provenance, confirmation events, risk labels—written outside the LLM and read only by a deterministic gate [18]. The design principle is the memory analogue of the division of labor above: the LLM may consolidate and plan, but it cannot write the authority that gates execution; memory may preserve information, but it cannot amplify source authority.

6.3 When to climb

Raising a verifier's level is governed by two factors: the cost of silent errors and the encodability of the property.

	Encodable (has an ODD)	Not encodable
Low error cost	climb to L3 if cheap	stay L1/L2; do not overspend
High error cost	L3/L4 is mandatory	L2 ceiling + human review; label the risk

The framework is a deployment checklist, not a competition ladder: an Lo self-check is the right answer for a low-stakes draft summarizer and an indictment for a discharge summary generator. The same verdict text ("verified") carries radically different weight at different levels; the framework's entire point is to make that difference visible.

A compact five-item deployment checklist follows:

1. **State the property and its ODD.** What exactly is verified, and by what decidable tool (or is it not encodable at all)?
2. **Identify the spec source (Q1).** LLM-declared → escalate immediately; rule/truth/decidable → proceed.
3. **Declare the guarantee (Q2).** Correctness probe or completeness claim? State it in the system's own documentation.
4. **Set the abstention contract.** What happens when the input falls outside the ODD—UNSURE, escalation, or silence? Silence is a decision.
5. **Terminate the trust recursion.** What audits the judge, and what is the audit's own anchor?

An Lo self-check may be acceptable for a low-stakes summarizer; it is not for a clinical decision aid.

6.4 What the honest negative result teaches

Our accuracy result—the architecture is no better than the raw baseline—is frequently read as a failure of verification. It is not; it is a precise measurement of the value window. Accuracy is the LLM's axis; reportability and completeness are the verifier's. The value of verification is not accuracy; it is error reportability and completeness awareness. A deployer who needs accuracy should not buy this architecture; a deployer who needs to know *when the answer is not trustworthy* should. Papers that conflate the two axes (Sec. 2) inherit exactly this confusion—and that conflation is precisely the category error this paper diagnoses. Our four chapters are, in effect, a controlled experiment showing the two axes come apart.

7. Limitations

1. **Literature assessment is abstract-level for 11 of 17 papers;** 6 anchor-axis papers were verified against full text (*docs/07 V1.3*). The classification of the remainder may shift with full-text review, though the axis structure is unaffected.
2. **Case studies use a single model family** (deepseek-chat) and synthetic data (medical cases are fictional; no patient data). The framework's claims are structural, but its empirical illustrations are single-model.
3. **VAL classifies, it does not measure:** a scheme's level states its anchor's epistemic status, not the probability that its verdict is correct. Level and reliability are orthogonal; an L2 verifier can be more accurate than an L3 verifier on in-ODD cases.
4. **ODD boundaries are fuzzy in practice:** whether a property is "encodable" depends on the solver, the type system, or the rule's inputs, and can change with tooling. ODD honesty is an audit target, not an enforced guarantee (Sec. 3.4): ODD claims carry their own anchor ladder (self-declared → formally characterized), and boundary probes are the engineering check—but neither is automatic.
5. **VAL itself is unvalidated as a measurement:** we have not tested inter-rater reliability of the decision procedure at scale, nor shown that independent raters reach the same levels on a common corpus. The procedure is deterministic by construction; whether its Q1/Q2 judgments are reproducible is an open empirical question we plan to address with a larger corpus.
6. **The mathematics negative result is bounded by a ceiling effect:** a 20/20 baseline leaves no accuracy headroom, so the result supports only "no headroom on in-distribution problems," not "verification cannot improve accuracy in general." We do not compare against published verification architectures, so an implementation-quality confound cannot be excluded. The 42/42 calibration tests and the #105 adjudication were author-designed/adjudicated, not independently validated.
7. **The 17-paper survey omits verification families without clean objective truth—** calibration and uncertainty estimation, red-teaming, RAG citation verification, and reward-model verification—which VAL classifies only as L0/L1. The anchor axis does not cover anchor-less verification; that is a scope boundary, not a claim of coverage. These are not edge cases but mainstream LLM-verification practice, so VAL's completeness ladder is

inapplicable to them; the framework's guidance there is the L0–L2 distinction and honesty, not completeness.

8. **The value of "error reportability" is asserted but not quantified.** We do not report the judges' true-positive/false-positive rates, the cost of false alarms, or a cost–benefit against the accuracy drop (20–35% in the mathematics case). A verifier that raises many false positives for one true catch is net-negative; we have not shown this is not the case, and the reportability claim should be read as a hypothesis, not a measured benefit.
-

8. Conclusion

The verification literature for LLMs is a tower of levels built on different questions. We have argued that one question—*where does the ground truth come from, and what does the verdict guarantee?*—underlies all claims of verification strength, and we have provided a six-level taxonomy (VAL, L0–L5), a deterministic decision procedure, and a runnable classifier. Our empirical case studies and literature review support a single headline claim:

Others grade answers. We grade the graders—and the highest grade any grader can earn is a guarantee it can actually deliver: correctness within its domain, completeness within its ODD, and honesty when it must abstain.

Three directions follow. First, validating the standard itself: measuring inter-rater agreement of the decision procedure on a larger corpus of verification schemes, and stress-testing its boundary cases (the L1/L2 borders of RAG-grounded and tool-augmented checkers). Second, extending the L2→L3 upgrade already demonstrated in the medical domain (Alvarado within the appendicitis ODD, Sec. 5.3) to further decision rules and ODDs (e.g., PERC/Wells for pulmonary-embolism and chest-pain presentations). Third, exporting the framework beyond verification—the same "anchor source × guarantee × scope" structure applies to any claim of automated assurance, from unit testing to regulatory attestation.

Acknowledgments

We thank Jiuzhou Han for a clarifying discussion of VerifiAgent's verification mechanism, and for the apt term *silver ground truth*—deterministic re-computation that serves as a reference but certifies the computation rather than the answer—which sharpened the L2 sub-distinction of Sec. 3.2.

References

1. Han, J., Buntine, W., Shareghi, E. *VerifiAgent: A Unified Verification Agent in Language Model Reasoning*. EMNLP 2025. arXiv:2504.00406.
2. Fang, J., Zhang, B., Wang, C., et al. *Graph of Verification (GoV): Structured Verification of LLM Reasoning with Directed Acyclic Graphs*. arXiv:2506.12509, 2025.

3. Li, Y., Lin, Z., Zhang, S., et al. *Making Large Language Models Better Reasoners with Step-Aware Verifier (DiVERSE)*. ACL 2023. arXiv:2206.02336.
 4. Liu, C., Yuan, Y., Yin, Y., et al. *Safe: Enhancing Mathematical Reasoning in LLMs via Retrospective Step-aware Formal Verification*. ACL 2025. arXiv:2506.04592.
 5. Miao, N., Teh, Y.W., Rainforth, T. *SelfCheck: Using LLMs to Zero-Shot Check Their Own Step-by-Step Reasoning*. ICLR 2024. arXiv:2308.00436.
 6. Juneja, G., Dutta, S., Chakraborty, T. *LM²: A Simple Society of Language Models Solves Complex Reasoning*. EMNLP 2024. arXiv:2404.02255.
 7. Wang, Y., Gangi Reddy, R., et al. *Factcheck-GPT: End-to-End Fine-Grained Document-Level Fact-Checking and Correction of LLM Output*. arXiv:2311.09000, 2023.
 8. Chen, J., Li, C., Yuan, Y., Yao, A.C. *Hierarchical Attention Generates Better Proofs*. ACL 2025. arXiv:2504.19188.
 9. Luo, M., Wu, S., et al. *Dr. V: A Hierarchical Perception-Temporal-Cognition Framework to Diagnose Video Hallucination*. arXiv:2509.11866, 2025.
 10. Li, Q., Xu, J., et al. *A Proprietary Model-Based Safety Response Framework for AI Agents*. arXiv:2511.03138, 2025.
 11. Yang, W., Cheng, H., Zhou, B., et al. *M³-SafetyBench: 多领域多场景多维度的大语言模型安全评估体系* [M³-SafetyBench: a comprehensive benchmark for evaluating the safety of large language models across multiple domains, scenarios, and dimensions]. 中国科学: 信息科学 [Scientia Sinica Informationis], 55:2923–2940, 2025.
 12. Khatchadourian, R., Franco, R. *LLM Output Drift: Cross-Provider Validation & Mitigation for Financial Workflows*. arXiv:2511.07585, 2025.
 13. Chen, Z., Chen, J., et al. *Standard Benchmarks Fail—Auditing LLM Agents in Finance Must Prioritize Risk*. arXiv:2502.15865, 2025.
 14. Liu, X., Wang, J., et al. *Prompting Frameworks for Large Language Models: A Survey*. ACM Computing Surveys. arXiv:2311.12785, 2023.
 15. Liu, Y., Yao, Y., et al. *Trustworthy LLMs: A Survey and Guideline for Evaluating Large Language Models' Alignment*. arXiv:2308.05374, 2023.
 16. González, J., Nori, A.V. *Beyond Words: A Mathematical Framework for Interpreting Large Language Models (HEX)*. arXiv:2311.03033, 2023.
 17. Chern, I.-C., Chern, S., Chen, S., Yuan, W., Feng, K., Zhou, C., He, J., Neubig, G., Liu, P. *FacTool: Factuality Detection in Generative AI—A Tool Augmented Framework for Multi-Task and Multi-Domain Scenarios*. ICLR 2024. arXiv:2307.13528.
 18. Xu, J., Xiao, Y., Shao, W., Liu, H., Li, X. *Memory Provenance Laundering in LLM Agents: A Non-Amplification Firewall for Persistent Memory*. arXiv:2607.29167, 2026.
-

Appendix A: The decision procedure (runnable)

The classification procedure of Sec. 3.3 is implemented in `val_standard.py` (11 self-tests, all passing). Representative outputs:

LLM self-check ("I checked it")	→ L0 self-declaration / no deterministic anchor
Problem-derived rule (final_check)	→ L1 deterministic rules from problem text
Substitution (verify_root)	→ L2 correctness + gold-label anchor
Solution-set equality (solveset)	→ L3 single-property completeness (decidable)
Rust borrow checker	→ L4 domain-level proof system
Universal complete verifier	→ L5 impossible (unrestricted), rejected

Full standard with usage criteria and stop conditions: `docs/06`.

Appendix B: The 17-paper classification

Full versioned assessment with per-paper evidence: `docs/07`. Compact summary (anchor axis only):

Level	Papers
Lo	SelfCheck, LM ²
Lo/L1 (meta) + L1 (tool, no gold labels)	VerifiAgent
Lo/L1	Factcheck-GPT (corrected from L1/L2 at V1.2)
L1/L2	FACTOOL, LLM Output Drift
L2	DiVERSE, M ³ -SafetyBench, Dr. V
L3/L4	Hierarchical Attention, Safe (L4 kernel + Lo statement layer)
Non-verification (taxonomy)	Trustworthy LLMs, HEX, Prompting Survey
L1/L2 (audit)	Standard Benchmarks Fail
Not anchor-classified (granularity axis)	GoV
Not anchor-classified (risk axis)	SafetyResponse

Appendix C: Experiment archives

- Chapter 1 (math): `RESULTS.md`, `EVALUATION.md`, `data/testset_results.json`, `test_verifier.py` (42/42), `verifier.py` (ANCHOR_LEVELS)
- Chapter 2 (behavior): `behavior/REPORT.md`, `behavior/analyze*.py`, `behavior/data_raw*.json`
- Chapter 3 (medical): `medical/REPORT.md`, `medical/verify_diag.py`, `medical/alvarado.py`, `medical/cases*.json`
- Chapter 4 (code): `docs/08` (predictions & protocol), `chapter4/p3_experiment.py`, `chapter4/hard_problems.py`, `chapter4/p3_*.json` (results)

附录

math_agent — LLM 多步推理的失败模式与验证框架 (VAL)

一句话：一个仓库、三个章节、一条脉络——研究"LLM 多步推理的失败模式与验证框架"。
三章实验的元结论已升华为 **验证自主等级 (VAL, L0–L5)** ——给"验证方案"本身分级——
并沉淀为预印本 **Grading the Graders**。

核心主张 (可原样引用)：
每一层都必须挂接可验证的裁判，而裁判必须理解"完备性"而不只是"正确性"。
LLM 只做翻译 (拆解/声明/渲染)，确定性代码做裁判；验证规格必须锚定在
"不依赖 LLM 声明"的地面真值上。

贯穿三章的元教训：**"裁判的裁判也需要裁判"** (验证器自身也要被校准/审计)。

论文 (arXiv 预印本)

Grading the Graders: Verification Autonomy Levels (L0–L5) for LLM Reasoning

- 一句话：**别人给答案分级，本项目给"分级"分级。
- 状态：**已发布 — arXiv:2608.19009 (2026-08-20 公布, cs.CL 主类目)：
<https://arxiv.org/abs/2608.19009>
- 文件：**paper/preprint.md (源稿) · paper/preprint.pdf (20 页, 可上传版) ·
paper/submission.md (arXiv 表单提交包)

VAL 验证自主等级 (跨三章的元结论)

等级	锚定来源	保证	完备性
L0	LLM 自证 ("我检查过了")	无	无
L1	题面/代码确定性规则	确定性匹配	无
L2	客观真值 / oracle / 金标	正确性	无 (完备性盲区)
L3	可判定系统 (解集等价 / 类型 / 决策规则)	单性质完备	有 (ODD 内)
L4	领域级证明系统 (类型系统 / 证明内核)	领域完备	有 (域内)
L5	通用完备验证	任意性质	不可判定 (Rice 定理)

- 判级工具: val_standard.py (判级三问的可运行版)
- 抬级工具: val_raise.py (L2→L3 重编码处方——框架从诊断走向生成)
- 追问层: val_interrogate.py (buzzword 不豁免, 不清楚就不猜)
- 判级标准: docs/06-判级标准.md · 等级体系: docs/05-锚定层级.md · 文献对照: docs/07-文献评述.md
- 抬级可行性: docs/09-抬级器可行性.md

工具链（一条命令）

声称 → [追问层 问] → [判级器 判] → [抬级器 开处方] → [审计报告 体检]
不豁免buzzword 你在哪一级 怎么上去/上不去为什么 整条验证链健康

python val_chain.py # 一条命令：33/33 自测 + 演示走查（全仓库含压力测试共 38/38）

工具	作用	自测	命令
val_interrogate.py	追问层：自由文本声称→判级参数, buzzword 穿透	5/5	python val_interrogate.py --auto (交互：无参数运行)
val_standard.py	判级器：L0–L5 分类（Q1 规格来源/Q2 保证/Q3 范围）	10/10	python val_standard.py
val_raise.py	抬级器：六模式重编码处方 + 可判定域扫描 + 审计报告	18/18	python val_raise.py
val_demo.py	演示走查：判级→抬级→处方→体检四步	—	python val_demo.py
stress_test.py	压力测试：多智能体投票/SymPy 用法/主观性质/混合栈/区块链 buzzword	5 题	python stress_test.py

诚实边界（工具会说的"不"）：性质不可编码 → 无法抬级；可判定性未知 → 要求人工确认；输入是声称（Lo），工具不验证其真实性。

三章总览

章	领域	状态	结论
一	数学验证 (LLM+SymPy)	✅ 封存	准确率窗口空 (16/15/13 vs baseline 20/20) ； 价值=错误可报告性； L3 解集等价抓漏解
二	行为观测 (prompt injection)	✅ 封存	五轮迭代：FPR 92%→30%、强攻击 TPR 1.0、弱攻击 0.75； 边界=隐匿执行 0.60 (L2 天花板)
三	医疗诊断	🔄 进行中	LLM 教科书 10/10、硬病例 8/10； 失败模式="信息不足仍下结论"； 信息完整性裁判 3/3 (含人工漏判 #105) ； L3 抬级成功：导入 Alvarado 决策规则 (阑尾炎 ODD，区间缺数据语义)

快速开始

```
# 1. 配置密钥: local_config.json (不入 git) 或环境变量 DEEPSEEK_API_KEY
pip install -r requirements.txt

# 2. 裁判单元测试 (无需 API)
python test_verifier.py          # 42/42

# 3. VAL 判级工具自测
python val_standard.py          # 10/10

# 3b. VAL 抬级器自测
python val_raise.py             # 18/18

# 3c. VAL 追问层自测 (buzzword 穿透)
python val_interrogate.py --auto # 5/5

# 4. 跑一道数学题
python main.py -p "求函数 f(x)=x^2-4x+7 在区间 [1,4] 上的最小值"

# 5. 医疗裁判实跑 (无需 API)
python medical/verify_diag.py
```

文档导航

文档	内容
docs/01-研究叙事.md	项目故事主线 (投名状第一页, 四结论版)
docs/02-失败模式分类学.md	8 个失败模式, 全部有数据实证
docs/03-架构设计备忘.md	设计决策的"为什么" + 已知边界
docs/04-实验档案.md	8 轮实验全记录与数据索引
docs/05-锚定层级.md	VAL 等级体系 (可迁移结论)
docs/06-判级标准.md	判级三问 + 决策程序 + 使用清单
docs/07-文献评述.md	17 篇"分层验证"论文 × VAL 判级 (V1.3, 版本化追踪)
RESULTS.md / EVALUATION.md	结果总账 / 早期评估
HANDOFF.md	项目交接文档 (新 AI 接手的全部上下文)

目录结构

```
math_agent/
├─ main.py                # 第一章主流程（分类→拆解→验证→纠错→组合→终局验证）
├─ verifier.py            # 裁判：9 种验证类型（含 L3 solution_set），ANCHOR_LEVELS 标注等级
├─ decomposer.py / solver.py / combiner.py / final_check_deriver.py
├─ llm_client.py / config.py      # DeepSeek 客户端 / 配置（密钥外置）
├─ test_verifier.py        # 裁判 42 用例
├─ val_standard.py        # VAL 判级工具（10 用例）
├─ behavior/              # 第二章：行为观测（5 轮迭代，REPORT.md 完整报告）
├─ medical/               # 第三章：医疗诊断（cases*.json 病例 + verify_diag.py 裁判 + REPORT.md）
├─ data/ logs/            # 全部实验数据 / 运行痕迹
├─ docs/                  # 01-07 方法论文档（含 VAL 体系/判级标准/文献评述）
├─ paper/                 # arXiv 预印本（preprint.md/pdf、submission.md 提交包）
└─ README.md / HANDOFF.md / RESULTS.md / EVALUATION.md
```

已知边界（诚实版）

1. **完备性盲区**：代回/采样验证无法证明“没有遗漏”——原理性开放；L3 解集等价仅在 ODD（可判定域）内闭合；
2. **准确率窗口**：数学章架构不优于 baseline（16/15/13 vs 20/20）——价值在错误可报告性，不在准确率；
3. **行为章天花板**：隐匿执行攻击（输出层无痕迹）是 L2 结构性边界（TPR 0.60）；
4. **VAL 自身未验证**：判级程序是确定性的，但其 Q1/Q2 判断的可复现性（inter-rater）尚未在规模语料上检验——judge 的 judge 也需要 judge；
5. **医疗章伦理边界**：合成虚构病例、无真实患者数据、非诊断工具、不构成医疗建议。

更多上下文见 [HANDOFF.md](#)——新 AI 接手项目的完整交接文档。

附录 B

项目交接文档（HANDOFF）

用途：在全新 AI 对话中继续本项目。本文档包含全部必要上下文——把全文粘贴给新 AI 即可继续。

最后更新：2026-08-18

o. 一句话项目定位

一个仓库、三个章节、一条脉络：研究"LLM 多步推理的失败模式与验证框架"。
核心主张（可原样引用）：

每一层都必须挂接可验证的裁判，而裁判必须理解"完备性"而不只是"正确性"。
LLM 只做翻译（拆解/声明/渲染），确定性代码做裁判；验证规格必须锚定在
"不依赖 LLM 声明"的地面真值上。

贯穿三章的元教训：**"裁判的裁判也需要裁判"**（验证器自身也要被校准/审计）。

1. 仓库与运行环境

- **GitHub**： https://github.com/1549080929-debug/math_agent（已推送，main 分支）
- **本地路径**： C:\Users\殷雅杰\Desktop\math_agent
- **Python**： C:\Users\殷雅杰\AppData\Local\Programs\Python\Python313\python.exe
- **API Key**： DeepSeek， 存在 local_config.json（已被 .gitignore 排除，不入库）
- **推送**： 已配置 SSH over 443（ ~/.ssh/config → ssh.github.com:443， git core.sshCommand 已设）。
命令： git add -A && git commit -m "..." && git push
- **模型**： deepseek-chat； 本地嵌入模型 BAAI/bge-small-zh-v1.5（第二章用，已缓存）

2. 方法论主线（三章共同）

审计先行（先建带标签测试床）→ 测价值窗口（baseline 对比/可分性）→ 失败模式分类
→ 裁判可行性（先测裁判本身）→ 诚实负结果如实报告

失败模式分类学（第一章产出，8 个模式，全部有数据实证）

#	模式	状态
1	自治地错（错误答案有干净推导路径）	✅ 审计先行制度化
2	伪验证（文本声称已验证但验证器没干活）	✅ combiner 诚实化
3	拆解层污染（错误公式写进子任务）	⚠️ 终局验证兜底
4	组合层篡改（推理对、终局答案错）	✅ 类型系统+终局验证
5	完备性盲区（验证不了“漏掉的候选”）	❌ 原理性开放
6	信任递归（验证规格由 LLM 声明）	✅ 规则派生
7	预测编造（失败模式预测是幻觉归因）	✅ 方法论修正
8	验证器极限（开放问题无裁判）	⚠️ 架构边界

元结论（跨三章）：裁判的价值 = 规格锚定在哪一层。已抽象为**验证自主等级 VAL-Lo~L5**（Lo LLM 声明 < L1 题面派生 < L2 客观真值 < L3 单性质完备 < L4 领域完备 < L5 不可判定）。完备性不是“无解”，是相对 ODD（可判定域）的——在 ODD 内可达完备（解集等价、类型检查、已验证决策规则），ODD 外退回 L2。详见 docs/05-锚定层级.md；verifier.py 的 ANCHOR_LEVELS/anchor_level() 已把等级编码进代码。**判级标准**（三问+程序+清单）见 docs/06-判级标准.md，可运行工具 val_standard.py（10/10 自测）。

3. 第一章：数学验证（已封存）

架构（math_agent/，42/42 裁判测试）：入口分类→拆解→自查→子任务(求解→SymPy验证→反馈纠错→回溯)→组合(结构化输出)→终局验证(规格规则派生)→诚实失败出口。
验证类型：
root/extreme/interval_extreme/equality/satisfies/inequality/solution_set/answer_type/final_parameter_set
（三态：PASS/FAIL/UNSURE）。

关键结果：
- 裁判单元测试 **42/42**（test_verifier.py）；新增 solution_set（L3 完备性锚：solveset 解集等价，抓漏解/压缩篡改）
- 20 题三级测试集跑 3 轮：系统 16/15/13 vs **baseline 恒 20/20——准确率是随机变量，架构不优于 baseline**
- 压轴 2 题（竞赛/新题）：baseline 2/2，系统 0/2（拆解污染+组合篡改）→类型系统+终局验证后 B 题 [-1,1] 当场抓获并修正为 [0,1]
- 对抗预测实验：LLM 生成的标准答案 **3/5 是错的**（SymPy 审计证实）
- **结论：数学域价值窗口为空**（LLM 太强，验证无增量）

产出： docs/01-研究叙事.md、 docs/02-失败模式分类学.md、 docs/03-架构设计备忘.md、 docs/04-实验档案.md、 RESULTS.md

4. 第二章：行为观测（已封存）

领域： prompt injection 下 LLM 行为完整性观测（固定探针，标准答案未知）。
方法： 行为统计特征 + 无向偏差（马氏距离风格）； probe-conditioned（per-probe 基线）。

五轮迭代结果 (behavior/REPORT.md) :

轮	动作	关键数字
1	全局统计	留出 AUC 0.82 (窄测试集假象)
2	真实分布检验	**FPR 92%** (窗口闭合) ; evasiveB 对抗规避 AUC 0.987→0.653
3	探针化+操作点修正	**FPR 30%**, 强攻击 TPR 1.0/AUC 1.0 (发现无向z分数中心≈√特征数, 阈值必须从正常分布定)
4	字符 n-gram	弱攻击 TPR 0.50→0.80 但 FPR 翻倍 0.65 (同义词改写噪声)
5	真语义嵌入(bge-small-zh)	**FPR 30%、强攻击 1.0、弱攻击 0.75** (w=0.5) ; 隐匿执行 evasiveB 0.60 (原理性边界)

结论: 行为统计检测达到**可部署窗口**; 唯一边界=输出层无痕迹的隐匿执行攻击。

5. 第三章：医疗诊断 (进行中)

定位: LLM 诊断推理失败模式研究, **非诊断工具**。伦理边界: 合成虚构病例、无真实患者数据、不构成医疗建议。

测试床:

- medical/cases.json: 10 例教科书式病例 (常见病/红旗征/罕见致命/易漏/鉴别陷阱/多系统/急诊/疼痛定位)
- medical/cases_hard.json: 10 例硬病例 (不典型×2/信息不足×2/竞争×3/合并症×1/误导主诉×2)

基线结果:

- 教科书 10 例: **10/10 全对** (置信度 0.9+, 红旗征全识别) ——LLM 太强, 准确率窗口为空
- 硬病例 10 例: **8/10 可接受, 2/10 失败**——失败模式="信息不足仍下结论" (#103 阑尾炎 conf 0.7、#104 心绞痛 conf 0.7, lab 均为"暂无检查结果") ; 置信度校准与信息完整性脱钩

裁判原型 (medical/verify_diag.py, 三个结构性检查) :

1. 信息完整性: lab 完全缺失 + 确定性高置信度诊断 → 过度自信 FAIL
2. 红旗征: 急诊红旗 → 必须 urgent=True
3. 鉴别完整性: 确诊在鉴别列表 (含同义词/大小写归一)

校准后结果: **只标记 #103/#104/#105, 其余 17 例全过; 精确率/召回率 3/3 (金标修正后)**。

关键事件: **裁判发现了人工漏判的失败 (#105)** ——裁判价值不只是复核, 是发现。
裁判自身的校准 (同义词/大小写/字段语义) 是"裁判的裁判也需要裁判"第三次上演。

第三章价值窗口形态: 不是提高准确率, 而是"**信息完整性裁判**" (廉价后置规则, 针对医疗最危险的失败模式"结论超出证据") 。

6. 三章对比与当前状态

章	领域	状态	窗口结论
一	数学验证	✅ 封存	空 (LLM 太强)
二	行为观测	✅ 封存	达成 (探针化+语义嵌入)
三	医疗诊断	🔄 进行中	形态=信息完整性裁判 (原型 3/3)

7. 当前待办 (第三章, 三选一)

- 1. 扩"信息不足"病例 (10+ 例) : 验证裁判的稳定精确/召回, 变成可汇报指标;
- 2. 裁判接入"诊断-检查"闭环: LLM 输出 → 裁判拦截过度自信 → 强制输出 UNSURE+检查建议 (三态裁判在医疗的落地形态) ;
- 3. 收束第三章: 写进 docs/ 叙事 (o1-研究叙事 升级为三章版) 。

更远的方向 (可选) : 新领域第四章 (如代码生成验证/Agent 安全)、或把三章方法论提炼成正式文章/投名状。

8. 关键文件索引

文件	内容
main.py / verifier.py / decomposer.py / solver.py / combiner.py / final_check_deriver.py	第一章架构
test_verifier.py	裁判 42 用例
behavior/	第二章 (REPORT.md 为完整报告, probes.json 探针, data_raw.json 数据, analyze*.py 分析)
medical/	第三章 (cases.json 病例, data_baseline.json 基线, verify_diag.py 裁判, REPORT.md)
docs/o1~o6	第一章四份核心文档 (o5=锚定层级/可迁移结论, o6=VAL判级标准)
docs/o7- 文献评述.md	17 篇"分层验证"论文 × VAL 判级标准 (含低置信标注与待办)
val_standard.py	VAL 判级工具 (classify()) + 10 用例自测, 判级三问的可运行版)
paper/	arXiv 预印本 (README=状态清单; preprint.md=英文骨架稿, 标题/摘要/正文各节草稿)
RESULTS.md / EVALUATION.md	结果总账
HANDOFF.md	本文档

9. 给新 AI 的起点建议

接到本交接文档后，推荐先读 `medical/REPORT.md` 和 `medical/verify_diag.py` 了解第三章现状，然后与用户确认走待办 1/2/3 哪条。执行纪律保持不变：
审计先行、测窗口再建架构、诚实负结果、每步提交推送。

附录 C

验证栈审计报告

验证器/指标	等级	保证	风险	抬级建议
answer_type	L3	低 (ODD 内完备)	无	
equality	L2	中 (正确性探针)	接入抬级器 (val_raise)	
extreme	L2	中 (正确性探针)	接入抬级器 (val_raise)	
final_parameter_set	L2	中 (正确性探针)	接入抬级器 (val_raise)	
inequality	L3	低 (ODD 内完备)	无	
interval_extreme	L3	低 (ODD 内完备)	无	
root	L2	中 (正确性探针)	接入抬级器 (val_raise)	
satisfies	L2	中 (正确性探针)	接入抬级器 (val_raise)	
solution_set	L3	低 (ODD 内完备)	无	
medical 信息完整性裁判	L2	中 (正确性探针)	接入抬级器 (val_raise)	
behavior 统计检测器	L2	中 (正确性探针)	接入抬级器 (val_raise)	
LLM 自评(基线)	Lo	高 (无保证)	接入抬级器 (val_raise)	

生成: val_raise.audit_report() — 评测指标的'等级体检', 自动标注哪些指标只是 L2 正确性探针、哪些达到完备保证。